

SENIOR BLOCKCHAIN EXPERT

20-DAY MASTERCLASS

150-200 Page Instructor-Grade Lecture Handbook

Blockchain Architecture | Solidity & EVM | Foundry | Smart Contract Security
DeFi & AMMs | Aave-Style Lending & Flash Loans | Oracles | Proxies
Web3 Backend | Indexing | Layer 2 | Cross-Chain | Solana & Anchor
System Architecture | Capstone | LinkedIn & Upwork Career Delivery

Course purpose: Build the technical depth, practical evidence and system-design habits needed to compete for senior-track blockchain development and freelance work. Completing twenty lectures is the start; seniority comes from repeatedly building, testing, debugging, reviewing and operating real systems.

Edition: August 2026

Recommended pace: 60-minute lecture + 60-120 minutes of hands-on practice per day

HOW TO USE THIS MASTERCLASS

What this handbook is designed to do

This edition expands the 20-day curriculum into an instructor-ready handbook. Each day is intentionally organized into seven teaching pages: lecture map, deep theory, architecture and trade-offs, code walkthrough, guided lab, security/testing/debugging, and interview/homework/portfolio work. The goal is to make each 60-minute lecture teachable without needing a separate textbook while still pushing the student into hands-on work after the lecture.

Reality check: Twenty lecture hours alone cannot create senior-level judgment. Use the handbook as the map, then repeat the labs, build the capstone, review open-source code, perform audits, and ship production-quality portfolio artifacts.

Recommended daily rhythm

1. Preview the objectives and key terms for 5 minutes before class.
2. Teach/follow the 60-minute lecture agenda.
3. Complete the guided lab immediately after the lecture while concepts are fresh.
4. Run the security/testing checklist and deliberately break the code.
5. Answer the interview questions aloud without reading the answer first.
6. Complete the homework and commit a portfolio artifact to GitHub.
7. Write one “what I misunderstood today” note; these notes become an interview study guide.

Minimum toolchain

Core development	Later modules
Git + GitHub	OpenZeppelin Contracts
VS Code or equivalent	Slither
Node.js LTS	TypeScript + viem/ethers
Python 3	SQL database or SQLite
Foundry: forge/cast/anvil	Solana CLI + Rust + Anchor
A wallet in a dedicated test profile	Docker/CI as needed

Safety and professional practice

- Use only local chains, testnets and small disposable test accounts for course exercises.
- Never paste seed phrases, private keys or production API secrets into code, screenshots or chat logs.
- Pin dependencies and network configuration for reproducibility.
- Treat third-party contract addresses, token decimals, oracle feeds and bridge endpoints as network-specific configuration that must be verified from official documentation.
- Use the capstone as evidence of reasoning, not as a claim that the code has undergone an independent professional audit.

COURSE COMPETENCY MAP

What you should be able to do by Day 20

Protocol reasoning - Describe transaction/state lifecycles, EVM execution, gas, consensus, finality and L2 settlement without relying on vague analogies.

Smart contracts - Design Solidity state models, interfaces, access control, token standards, external-call boundaries and upgrade-compatible storage.

Testing - Use unit, fuzz, invariant and fork testing to encode protocol properties and integration assumptions.

Security - Threat-model assets and roles, reproduce vulnerabilities locally, triage static analysis and write professional findings.

DeFi - Reason about AMM execution, lending risk, oracles, flash-loan atomicity, slippage and profitability constraints.

Infrastructure - Build RPC-backed services, transaction state machines, event indexers and reorg-aware data pipelines.

Scaling/interoperability - Explain rollups, sequencers, data availability, bridges and cross-chain validation boundaries.

Solana - Understand accounts, instructions, PDAs, Anchor constraints, CPI, SPL token flows and upgrade authority.

Architecture/operations - Design governance roles, monitoring, runbooks, deployment pipelines and incident controls.

Professional delivery - Convert vague client requests into discovery questions, milestones, acceptance criteria and evidence-backed proposals.

Portfolio outputs across the course

- Transaction-lifecycle architecture note
- EVM calldata/storage lab
- SimpleBank with invariants
- Treasury authority-surface review
- Production-style ERC-20 system
- Foundry unit/fuzz/invariant/fork suite
- Reentrancy exploit + remediation report
- Audit sample
- AMM math / DEX integration lab
- Flash-loan/arbitrage simulator
- Oracle-safe price consumer
- Upgradeable proxy lab
- Web3 transaction service
- Reorg-aware event indexer
- L2 deployment comparison
- Cross-chain receiver security lab
- Solana Anchor PDA lab
- Senior architecture pack
- DeFi Vault capstone
- LinkedIn/Upwork/GitHub professional package

Target outcome: By the end, your GitHub should show code + tests + architecture + threat models + deployment evidence. That combination is far more persuasive for senior-track work than a collection of token-generation tutorials.

CLICKABLE COURSE CONTENTS

Select a day below to jump to its first lecture page. PDF conversion preserves these internal links in most modern viewers.

[Day 01 - Blockchain Architecture and Cryptographic Foundations](#)

Build an engineering mental model of how a blockchain moves from signed user intent to replicated, finalized state, and learn the cryptographic primitives that make that model possible.

[Day 02 - Ethereum, EVM, Transactions and Gas](#)

Understand Ethereum as an execution environment: account state, EVM memory areas, ABI calldata, opcodes, gas accounting, transaction receipts and RPC methods.

[Day 03 - Solidity Fundamentals: State, Types and Functions](#)

Write clear, testable Solidity and understand how language choices map to blockchain state, authorization and external interfaces.

[Day 04 - Advanced Solidity and Contract Architecture](#)

Use inheritance, interfaces, libraries, modifiers, low-level calls and storage-aware design without creating brittle or unsafe abstractions.

[Day 05 - Token Standards, OpenZeppelin and Production ERC Design](#)

Implement standards-based assets with mature libraries, understand allowance and supply mechanics, and design roles, pause controls and token lifecycle features responsibly.

[Day 06 - Foundry, Hardhat, Testing and Deployment Workflows](#)

Operate a professional smart-contract development workflow with reproducible builds, unit tests, fuzzing, invariant testing, fork tests, deployment scripts and verification.

[Day 07 - Smart Contract Security I: Reentrancy, Authorization and External Calls](#)

Learn to identify and exploit common Solidity vulnerabilities in a controlled local environment, then redesign the contract around explicit invariants and safe interaction patterns.

[Day 08 - Smart Contract Security II: Auditing, Fuzzing and Invariants](#)

Perform a structured smart-contract review using threat modeling, manual code analysis, static analysis, fuzzing, invariant tests and professional finding reports.

[Day 09 - DeFi Fundamentals, AMMs and Uniswap-Style Market Design](#)

Understand automated market maker mathematics, liquidity-provider economics, slippage, price impact, routing and the architecture behind modern DEX integrations.

[Day 10 - Lending, Aave-Style Flash Loans, Arbitrage and MEV Risk](#)

Understand collateralized lending, liquidation, health factors, flash-loan execution and the engineering reality of arbitrage after fees, slippage, gas and transaction-ordering risk.

CLICKABLE COURSE CONTENTS - DAYS 11-20

Day 11 - Oracles, Price Feeds and External Data Integrity

Design contracts that consume external data safely, with explicit handling for decimals, staleness, sequencer health, fallback behavior and manipulation resistance.

Day 12 - Upgradeable Contracts, Proxies and Storage Safety

Understand proxy architecture, initialization, UUPS/transparent patterns, storage compatibility, upgrade authorization and operational governance for mutable smart-contract systems.

Day 13 - Web3 Backend Engineering, Signing and Transaction Reliability

Build reliable backend services that read chain state, construct transactions, manage nonces, sign securely, monitor receipts and expose safe APIs without leaking private keys.

Day 14 - Blockchain Indexing, Events, The Graph and Data Pipelines

Build reliable offchain data views from onchain events, handle reorgs and backfills, and understand when to use custom indexers versus Subgraphs.

Day 15 - Layer 2, Rollups, Data Availability and Scaling Architecture

Explain modern Ethereum scaling systems, compare optimistic and ZK rollups, reason about sequencers/bridges/finality, and deploy applications with L2-specific operational awareness.

Day 16 - Cross-Chain Messaging, Bridges and Interoperability Security

Design and review cross-chain systems by separating message verification, token movement, replay protection, finality assumptions, rate limits and emergency controls.

Day 17 - Solana, Rust and Anchor: Accounts, PDAs, CPI and Program Security

Build a second blockchain mental model beyond the EVM by learning Solana's account model, Rust program execution, Anchor validation, PDAs, SPL tokens and cross-program invocation.

Day 18 - Senior Blockchain System Architecture, Governance and Production Operations

Design end-to-end blockchain systems that integrate contracts, wallets, backends, indexers, oracles, governance, monitoring and deployment controls with explicit trust boundaries.

Day 19 - Capstone: Production-Style DeFi Vault from Requirements to Deployment

Integrate Solidity, ERC-4626 thinking, testing, oracles, access control, backend/indexing and production documentation into one portfolio-grade project.

Day 20 - Senior Blockchain Career: Upwork, LinkedIn, Interviews and Client Delivery

Turn technical capability into professional credibility through evidence-based positioning, discovery, scoping, proposals, estimates, interview reasoning and long-term skill development.

Day 01 - Blockchain Architecture and Cryptographic Foundations

60-minute goal: Build an engineering mental model of how a blockchain moves from signed user intent to replicated, finalized state, and learn the cryptographic primitives that make that model possible.

Learning outcomes

- Explain and apply: a blockchain is a replicated state machine.
- Explain and apply: hashing is integrity, not encryption.
- Explain and apply: keys and signatures prove authorization.
- Explain and apply: consensus solves ordering under adversarial conditions.
- Complete the guided lab and capture reproducible evidence.
- Answer the senior interview questions using architecture/security trade-offs, not memorized definitions.

Minute-by-minute lecture plan

Time	Teaching focus
0-8 min	Course orientation: what senior blockchain engineers are expected to reason about
8-18 min	Distributed ledgers, nodes, blocks, state, transactions and peer-to-peer propagation
18-30 min	Hash functions, keypairs, signatures, Merkle structures and addresses
30-42 min	Consensus, fork choice, finality and the difference between safety and liveness
42-52 min	Bitcoin UTXO model versus Ethereum account/state model
52-60 min	Lab: inspect a real transaction and write the end-to-end lifecycle

Key terms for today

Term	Term	Term
distributed ledger	full node	validator
block header	transaction	state transition
hash function	public/private key	digital signature
Merkle tree	consensus	finality

Instructor opening script

Today's topic is blockchain architecture and cryptographic foundations. The senior-level objective is not to memorize API names; it is to understand the underlying trust model, state transitions and failure modes well enough to make architecture decisions and debug unexpected behavior. During the lecture, repeatedly ask: what state is authoritative, who can change it, which external dependencies are trusted, and what property must remain true after every successful transaction?

DAY 01 | BLOCKCHAIN ARCHITECTURE AND CRYPTOGRAPHIC FOUNDATIONS

Deep Theory and Mental Models

A blockchain is a replicated state machine

The useful senior-level definition of a blockchain is not “a database that cannot be changed.” It is a deterministic state machine replicated across many nodes that agree on an ordered history of state transitions. A transaction proposes a transition; the protocol determines whether it is valid; consensus determines which valid history becomes canonical. Thinking in state transitions makes later subjects - smart contracts, reorgs, rollups, bridges, and indexing - much easier to reason about.

Hashing is integrity, not encryption

A cryptographic hash maps arbitrary input to a fixed-length digest. Engineers use hashes for content addressing, commitments, transaction identifiers, Merkle structures, and signature schemes. A hash is intentionally one-way and does not “hide” data the way encryption does. Collision resistance, preimage resistance, and second-preimage resistance are distinct properties and should not be casually conflated.

Senior lens: Translate this concept into a review question: identify the authoritative state, caller/dependency assumptions, and the invariant that would fail if the concept were implemented incorrectly.

Keys and signatures prove authorization

A private key authorizes actions by creating digital signatures; the corresponding public key allows others to verify those signatures. Wallet software protects and uses private keys, but the chain never needs to receive the private key itself. Senior engineers separate key custody, signature generation, transaction construction, and transaction broadcast as different security boundaries.

Consensus solves ordering under adversarial conditions

Distributed systems need a way to agree on the sequence of valid transitions despite network delay, node failures, and potentially malicious participants. Consensus does not make every fact true; it gives participants rules for choosing a canonical history. Finality describes when a transition becomes sufficiently difficult or impossible to reverse under the protocol assumptions.

Senior lens: Translate this concept into a review question: identify the authoritative state, caller/dependency assumptions, and the invariant that would fail if the concept were implemented incorrectly.

State models change application design

Bitcoin's UTXO model represents spendable outputs consumed and created by transactions. Ethereum's account model tracks balances, nonces, code, and storage associated with addresses. Neither model is simply “better”; each makes different trade-offs around parallelism, programmability, privacy, and state access.

Senior engineers reason about trust boundaries

Every blockchain system includes assumptions: validator honesty, client correctness, oracle correctness, bridge security, key custody, RPC availability, and application authorization. The habit to develop from Day 1 is to ask: what must be trusted, what can be verified, and what happens when an assumption fails?

DAY 01 | BLOCKCHAIN ARCHITECTURE AND CRYPTOGRAPHIC FOUNDATIONS

Architecture, Data Flow and Trade-offs

Reference architecture / execution flow

```

User intent
|
Wallet constructs transaction
| signs with private key
v
RPC / gateway node
| propagates over P2P network
v
Mempool / transaction pool
| proposer selects + orders
v
Block execution
| deterministic validation
v
Consensus / fork-choice
|
Finalized canonical state
  
```

Trade-off analysis

1. Decentralization increases fault tolerance but adds coordination cost and latency.
2. Public verifiability reduces the need to trust a database operator, but it also means application data and transaction metadata may become observable.
3. Finality is a protocol property; user-facing “confirmed” status is a policy decision that should reflect risk.
4. RPC providers improve convenience but introduce availability, privacy, and censorship dependencies.
5. Key ownership gives power; key-management mistakes are often more catastrophic than ordinary application bugs.

Failure-mode questions to ask in a design review

- What happens if the external dependency is unavailable, stale or malicious?
- Can one compromised key change user balances, upgrade logic or move treasury assets?
- Does the system fail closed, fail open, or enter an ambiguous partially completed state?
- Which data is authoritative: contract state, emitted events, an indexer, an RPC response or an offchain database?
- How would you detect this component behaving incorrectly in production?
- Can the operation be replayed, reordered, reentered or executed on the wrong chain/context?
- What is the smallest privilege or interface that satisfies the requirement?

Architecture habit: For client work, draw the flow before writing code. A five-minute diagram often reveals missing authorization, wrong trust assumptions or unavailable recovery paths earlier than a compiler can.

DAY 01 | BLOCKCHAIN ARCHITECTURE AND CRYPTOGRAPHIC FOUNDATIONS

Code Walkthrough and Implementation Notes

Python: derive an Ethereum-style address from a public key (concept demonstration)

```
from eth_hash.auto import keccak

def public_key_to_address(uncompressed_public_key: bytes) -> str:
    # Expected form: 0x04 || X(32 bytes) || Y(32 bytes)
    if len(uncompressed_public_key) != 65 or uncompressed_public_key[0] != 0x04:
        raise ValueError("expected 65-byte uncompressed public key")
    digest = keccak(uncompressed_public_key[1:])
    return "0x" + digest[-20:].hex()

# In production, use a well-reviewed wallet/crypto library.
# This example is intentionally limited to the address-derivation step.
```

Walkthrough

Ethereum-style externally owned account addresses are derived from the lower 20 bytes of the Keccak-256 hash of the uncompressed public key payload.

The private key is never needed for address derivation once the public key exists; its role is authorization through signatures.

Do not implement signing primitives yourself for production systems. Use audited libraries and hardware-backed key storage where appropriate.

A senior engineer distinguishes cryptographic primitives from the wallet UX that wraps them.

Senior implementation checklist

- Pin the compiler/framework/library versions used by the repository.
- Keep configuration such as chain IDs and dependency addresses outside business logic unless immutability is deliberate.
- Use typed interfaces instead of arbitrary calls whenever practical.
- Emit events for operationally important state changes.
- Write the failure-path test at the same time as the success-path implementation.
- Document what this example intentionally omits before anyone reuses it.

DAY 01 | BLOCKCHAIN ARCHITECTURE AND CRYPTOGRAPHIC FOUNDATIONS

Guided Lab: Build, Observe and Break It

Lab steps

1. Install Git, VS Code, Node.js, Python and a wallet extension in a dedicated development profile.
2. Create a repository named senior-blockchain-masterclass and a day01 folder.
3. Choose one Ethereum transaction from a block explorer and record sender, recipient, nonce, value, gas limit, effective gas price, status and block number.
4. Find at least one event log if the transaction called a contract. Record the emitting contract and topic count.
5. Draw the lifecycle from wallet construction to finality in your README.
6. Explain which parts are verified by cryptography and which depend on consensus rules.
7. Write three trust assumptions introduced if the user relies on a hosted RPC provider.
8. Commit the notes and diagram to Git.

Expected evidence

- A README that can explain the transaction lifecycle without saying that validators “decrypt” transactions.
- A clean distinction among hashing, encryption and signatures.
- A diagram that includes wallet, RPC, propagation, block execution, consensus and finality.

Troubleshooting guide

If you see a problem: If a transaction has no logs, it may be a native-value transfer or a contract call that emitted no events; choose another transaction for the event exercise.

If you see a problem: A “pending” transaction may disappear from a public mempool view after replacement, inclusion or provider policy changes.

If you see a problem: Different explorers label confirmation/finality states differently. Use protocol concepts rather than trusting UI labels blindly.

Lab rule: Do not “fix” a failing lab by removing validation or weakening the expected assertion. First identify whether the failure is environment, configuration, integration state or actual code behavior.

Security, Testing, Debugging and Production Readiness

Security review points

- Never paste a real seed phrase or private key into course code, browser consoles, screenshots or repositories.
- Treat RPC responses as untrusted external input; verify chain IDs and transaction targets before signing.
- Do not equate an address with a human identity; addresses are pseudonymous identifiers.
- Understand phishing at the signature layer: malicious approvals and typed-data signatures can be dangerous without moving native currency immediately.

Required test plan

- Can the student describe a transaction as a state transition?
- Can the student distinguish integrity, authorization and confidentiality?
- Can the student explain why a blockchain needs both execution rules and consensus?
- Can the student identify at least five trust boundaries in a wallet-to-chain workflow?

Debugging playbook

1. Reproduce deterministically: pin block, seed, caller, input and contract version.
2. Read the revert/call trace before changing gas limits or code.
3. Check chain ID, deployed address, ABI and dependency version together.
4. Inspect state immediately before the failing call.
5. Reduce the scenario to the smallest failing sequence.
6. Convert the discovered failure into a regression test.
7. Only then refactor or optimize.

Production gate: A feature is not complete when it compiles. It is complete when its permissions, failure modes, tests, deployment configuration, monitoring signals and rollback/emergency assumptions are understood.

DAY 01 | BLOCKCHAIN ARCHITECTURE AND CRYPTOGRAPHIC FOUNDATIONS

Senior Interview Practice, Homework and Portfolio Output

Interview questions - answer aloud first

Q: What is the best engineering definition of a blockchain?

A: A replicated deterministic state machine whose participants agree on an ordered history of valid state transitions under a consensus protocol.

Q: Does a digital signature encrypt a transaction?

A: No. It proves authorization/integrity for signed data; transaction data on public chains is generally observable.

Q: What is finality?

A: A property describing when a block or state transition is considered irreversible under the protocol's economic or cryptographic assumptions.

Q: Why do reorgs matter to applications?

A: An application that treats a merely included transaction as irreversible may credit value that later disappears from the canonical chain.

Q: What is a trust boundary?

A: A point where security assumptions change - for example from a user-controlled wallet to a hosted RPC service or from an onchain contract to an oracle.

Q: Why is "immutable database" an incomplete description?

A: It ignores execution, ordering, consensus, reorgs, protocol upgrades and the fact that application state can intentionally change through valid transactions.

Homework

- Write a 1,200-word technical article: "What happens after I click Confirm in a wallet?"
- Create a one-page comparison of UTXO and account models.
- Build a 25-term flash-card set from today's vocabulary.

Portfolio deliverable

Ship this: Publish Day 1 README with a transaction lifecycle diagram and a concise trust-boundary analysis. This becomes your first portfolio artifact showing architectural reasoning, not just code.

Official references for this day

<https://ethereum.org/en/developers/docs/>

<https://docs.soliditylang.org/en/latest/introduction-to-smart-contracts.html>

<https://ethereum.org/en/developers/docs/transactions/>

Day 02 - Ethereum, EVM, Transactions and Gas

60-minute goal: Understand Ethereum as an execution environment: account state, EVM memory areas, ABI calldata, opcodes, gas accounting, transaction receipts and RPC methods.

Learning outcomes

- Explain and apply: the evm is deterministic computation with metering.
- Explain and apply: storage is persistent and expensive.
- Explain and apply: the stack shapes low-level execution.
- Explain and apply: abi is the contract interaction boundary.
- Complete the guided lab and capture reproducible evidence.
- Answer the senior interview questions using architecture/security trade-offs, not memorized definitions.

Minute-by-minute lecture plan

Time	Teaching focus
0-10 min	Ethereum account state and transaction envelope
10-20 min	EVM stack, memory, storage, calldata and code
20-30 min	ABI encoding, function selectors and return data
30-42 min	Gas, EIP-1559 fee fields and storage cost intuition
42-52 min	JSON-RPC methods and transaction receipts
52-60 min	Lab with cast or an explorer: decode calldata and inspect storage

Key terms for today

Term	Term	Term
EOA	contract account	nonce
bytecode	opcode	stack
memory	storage	calldata
ABI	function selector	gas
base fee	priority fee	receipt
log		

Instructor opening script

Today's topic is ethereum, evm, transactions and gas. The senior-level objective is not to memorize API names; it is to understand the underlying trust model, state transitions and failure modes well enough to make architecture decisions and debug unexpected behavior. During the lecture, repeatedly ask: what state is authoritative, who can change it, which external dependencies are trusted, and what property must remain true after every successful transaction?

Deep Theory and Mental Models

The EVM is deterministic computation with metering

Every validating node must obtain the same result from the same pre-state and transaction. Gas is the metering mechanism that prices computational and state resources while bounding execution. Senior engineers design contracts around both semantic correctness and resource cost.

Storage is persistent and expensive

Contract storage survives across transactions and contributes to long-lived chain state. Memory exists only during a call frame and is cheaper for temporary computation. Calldata is read-only input supplied to external calls. Choosing the wrong data location can create unnecessary gas costs or accidental copies.

Senior lens: Translate this concept into a review question: identify the authoritative state, caller/dependency assumptions, and the invariant that would fail if the concept were implemented incorrectly.

The stack shapes low-level execution

The EVM is stack-based. Solidity hides most stack manipulation, but stack depth, local-variable pressure, and compiler-generated temporaries still matter. “Stack too deep” errors and assembly optimizations make more sense once the underlying machine model is understood.

ABI is the contract interaction boundary

External callers encode a four-byte function selector followed by ABI-encoded arguments. The selector is derived from the Keccak-256 hash of the canonical function signature. ABI encoding is a schema-based format; the bytes are not self-describing.

Senior lens: Translate this concept into a review question: identify the authoritative state, caller/dependency assumptions, and the invariant that would fail if the concept were implemented incorrectly.

Transaction inclusion and execution are separate concepts

A transaction can be included in a block and still fail. The receipt status, gas used, logs, and effective gas price describe the execution result. A reverted transaction consumes gas because validators still performed work before the revert.

RPC is an interface, not consensus

Methods such as `eth_call`, `eth_getLogs` and `eth_sendRawTransaction` expose node functionality. A hosted RPC can return stale, filtered or unavailable data. Critical systems should understand provider behavior, block tags, retries, rate limits and chain-ID validation.

Architecture, Data Flow and Trade-offs

Reference architecture / execution flow

```

Signed transaction
| nonce, to, value, data, gas settings
v
Node validation -> transaction pool
|
Block builder / proposer
v
EVM call frame
+-- calldata (read-only input)
+-- stack (word operations)
+-- memory (temporary)
+-- storage (persistent state)
v
Return / revert
v
Receipt + logs + gas used

```

Trade-off analysis

1. Storage writes are expensive but sometimes necessary; optimize state layout without sacrificing clarity or security.
2. `eth_call` simulates execution against a chosen state and does not create a transaction.
3. Gas estimation can fail when the simulated call reverts; the fix is usually to understand the revert, not blindly raise gas.
4. A transaction nonce orders transactions from one EOA and prevents replay within the same chain context.
5. Fee settings affect inclusion incentives but do not change contract semantics.

Failure-mode questions to ask in a design review

- What happens if the external dependency is unavailable, stale or malicious?
- Can one compromised key change user balances, upgrade logic or move treasury assets?
- Does the system fail closed, fail open, or enter an ambiguous partially completed state?
- Which data is authoritative: contract state, emitted events, an indexer, an RPC response or an offchain database?
- How would you detect this component behaving incorrectly in production?
- Can the operation be replayed, reordered, reentered or executed on the wrong chain/context?
- What is the smallest privilege or interface that satisfies the requirement?

Architecture habit: For client work, draw the flow before writing code. A five-minute diagram often reveals missing authorization, wrong trust assumptions or unavailable recovery paths earlier than a compiler can.

Code Walkthrough and Implementation Notes

Solidity + ABI: inspect a simple storage contract

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.36;

contract Counter {
    uint256 public count;
    event Incremented(address indexed caller, uint256 newCount);

    function increment(uint256 amount) external {
        require(amount > 0, "zero amount");
        count += amount;
        emit Incremented(msg.sender, count);
    }
}

// Canonical signature: increment(uint256)
// calldata = 4-byte selector || 32-byte ABI encoded amount
```

Walkthrough

The public state variable creates an automatically generated count() getter; the stored value occupies a storage slot.

The external function reads its argument from calldata.

The require creates a revert path before the storage write.

The event writes data to the transaction log; indexed parameters become topics and non-indexed values are ABI encoded in log data.

count += amount causes a persistent state update and therefore materially affects gas.

Senior implementation checklist

- Pin the compiler/framework/library versions used by the repository.
- Keep configuration such as chain IDs and dependency addresses outside business logic unless immutability is deliberate.
- Use typed interfaces instead of arbitrary calls whenever practical.
- Emit events for operationally important state changes.
- Write the failure-path test at the same time as the success-path implementation.
- Document what this example intentionally omits before anyone reuses it.

Guided Lab: Build, Observe and Break It

Lab steps

1. Use an Ethereum development tool such as Foundry cast, an explorer decoder, or a local script.
2. Compute or obtain the selector for increment(uint256).
3. Encode increment(7) and separate selector bytes from the 32-byte argument.
4. Deploy Counter to a local chain or test environment.
5. Read count with eth_call before sending a transaction.
6. Send increment(7), wait for a receipt, and record gas used.
7. Read count again and inspect the Incremented log.
8. Call increment(0) with eth_call and record the revert behavior.
9. Use eth_getStorageAt or cast storage to inspect the first storage slot.

Expected evidence

- The calldata ends with a 32-byte integer encoding of 7.
- The receipt for the successful transaction has status 1 and contains the event log.
- The reverting simulation produces an error/revert reason and does not change state.
- The first storage slot reflects the current count for this simple layout.

Troubleshooting guide

If you see a problem: If a local chain address differs after redeployment, update your client configuration and ABI together.

If you see a problem: A failed transaction can still have a transaction hash and receipt; do not mistake “mined” for “successful.”

If you see a problem: If a public RPC blocks debug/storage methods, use a local Anvil node or another provider.

If you see a problem: If selector results differ, check canonical type names: uint means uint256 in the signature.

Lab rule: Do not “fix” a failing lab by removing validation or weakening the expected assertion. First identify whether the failure is environment, configuration, integration state or actual code behavior.

Security, Testing, Debugging and Production Readiness

Security review points

- Validate chainId before signing to avoid sending to the wrong network.
- Treat raw calldata decoders as convenience tools; verify critical targets and values independently.
- Do not assume a successful eth_estimateGas guarantees execution - state can change between estimation and inclusion.
- Be careful with arbitrary eth_call "from" addresses when testing authorization; simulations can impersonate callers conceptually but do not prove you control a key.

Required test plan

- Unit test increment success and zero-amount revert.
- Test that the event includes the correct caller and new value.
- Fuzz amount over a bounded range.
- Measure gas for first write versus later nonzero-to-nonzero writes and explain the difference conceptually.

Debugging playbook

1. Reproduce deterministically: pin block, seed, caller, input and contract version.
2. Read the revert/call trace before changing gas limits or code.
3. Check chain ID, deployed address, ABI and dependency version together.
4. Inspect state immediately before the failing call.
5. Reduce the scenario to the smallest failing sequence.
6. Convert the discovered failure into a regression test.
7. Only then refactor or optimize.

Production gate: A feature is not complete when it compiles. It is complete when its permissions, failure modes, tests, deployment configuration, monitoring signals and rollback/emergency assumptions are understood.

Senior Interview Practice, Homework and Portfolio Output

Interview questions - answer aloud first

Q: What are the main EVM data areas?

A: Stack, memory, persistent storage, calldata, code and return data; each has different lifetime and access properties.

Q: Why can a reverted transaction cost gas?

A: Nodes still execute instructions until the revert and must meter that work.

Q: What is a function selector?

A: The first four bytes of Keccak-256 over the canonical function signature.

Q: Does eth_call send a transaction?

A: No. It locally simulates a call against node state and returns the result or revert.

Q: What does a receipt tell you?

A: Execution status, gas used, logs, block inclusion and other transaction execution metadata.

Q: Why is storage optimization a senior topic?

A: Persistent storage affects gas and state growth, but aggressive packing can harm readability or cause upgrade-layout mistakes.

Homework

- Write a diagram of an EVM call frame including calldata, stack, memory and storage.
- Capture one successful and one reverted transaction receipt from your lab.
- Explain in 500 words why raising gasLimit does not fix a logical require failure.

Portfolio deliverable

Ship this: Publish a small “EVM calldata decoder” note or script that shows selector extraction, ABI argument decoding and receipt inspection.

Official references for this day

<https://docs.soliditylang.org/en/latest/abi-spec.html>

<https://ethereum.org/en/developers/docs/evm/>

<https://ethereum.org/en/developers/docs/gas/>

<https://ethereum.org/en/developers/docs/apis/json-rpc/>

Day 03 - Solidity Fundamentals: State, Types and Functions

60-minute goal: Write clear, testable Solidity and understand how language choices map to blockchain state, authorization and external interfaces.

Learning outcomes

- Explain and apply: state variables define the durable contract model.
- Explain and apply: mappings are lookup structures without built-in enumeration.
- Explain and apply: visibility expresses part of the trust boundary.
- Explain and apply: mutability keywords are semantic promises.
- Complete the guided lab and capture reproducible evidence.
- Answer the senior interview questions using architecture/security trade-offs, not memorized definitions.

Minute-by-minute lecture plan

Time	Teaching focus
0-8 min	Contract anatomy and compiler versioning
8-20 min	Value types, reference types, mappings, arrays and structs
20-30 min	State variables, constants, immutables and constructors
30-40 min	Function visibility, mutability, parameters and return values
40-50 min	Errors, events and checks-effects-interactions
50-60 min	Lab: implement SimpleBank with deposits, withdrawals and events

Key terms for today

Term	Term	Term
pragma	state variable	mapping
struct	enum	constructor
immutable	constant	visibility
view	pure	payable
custom error	event	msg.sender
msg.value		

Instructor opening script

Today's topic is solidity fundamentals: state, types and functions. The senior-level objective is not to memorize API names; it is to understand the underlying trust model, state transitions and failure modes well enough to make architecture decisions and debug unexpected behavior. During the lecture, repeatedly ask: what state is authoritative, who can change it, which external dependencies are trusted, and what property must remain true after every successful transaction?

Deep Theory and Mental Models

State variables define the durable contract model

Every state variable adds to the contract's persistent state model. Before writing functions, senior developers define invariants: what must always be true about balances, totals, permissions and lifecycle states? Solidity syntax is easy; preserving invariants under every call sequence is the difficult part.

Mappings are lookup structures without built-in enumeration

mapping(address => uint256) is ideal for balances but does not provide a list of keys. If an application needs enumeration, it must maintain additional data structures or rely on indexed events/offchain indexing. This design decision affects gas, data availability and upgrade complexity.

Senior lens: Translate this concept into a review question: identify the authoritative state, caller/dependency assumptions, and the invariant that would fail if the concept were implemented incorrectly.

Visibility expresses part of the trust boundary

external and public expose callable interfaces, while internal and private constrain Solidity-level access. private does not make onchain data secret. Authorization must be explicit; visibility alone is not an access-control system.

Mutability keywords are semantic promises

view promises not to modify state; pure promises not to read or modify contract state. These modifiers improve clarity and tooling, but they do not create an offchain environment. A view function called from another transaction still executes as part of that transaction.

Senior lens: Translate this concept into a review question: identify the authoritative state, caller/dependency assumptions, and the invariant that would fail if the concept were implemented incorrectly.

Custom errors are precise and gas-conscious

require with strings is useful for readability, but custom errors provide typed failure information and can reduce deployment/runtime data costs. Senior contracts use meaningful error types that help tests, frontends and auditors distinguish failure modes.

Events are an application data interface

Events do not update contract storage and cannot be read by contracts after emission in the same way as storage. They are designed for external observers. Good event design supports monitoring, indexing, incident response and user-facing history.

DAY 03 | SOLIDITY FUNDAMENTALS: STATE, TYPES AND FUNCTIONS

Architecture, Data Flow and Trade-offs

Reference architecture / execution flow

SimpleBank state model

```
address -> user balance mapping
|
+-- deposit(): balance += msg.value
|
+-- withdraw(amount):
    check balance
    update balance
    transfer value
    emit event
```

Invariant: sum of recorded user balances
should never exceed contract-controlled assets.

Trade-off analysis

1. A mapping is gas-efficient for direct access but not enumerable.
2. An immutable is assigned once during construction and embedded efficiently for later reads.
3. A payable function can receive native currency; receiving value does not automatically update your internal accounting.
4. External calls during withdrawal can transfer control to another contract; state ordering matters.
5. Events should contain enough indexed identity fields for common queries without over-indexing dynamic data.

Failure-mode questions to ask in a design review

- What happens if the external dependency is unavailable, stale or malicious?
- Can one compromised key change user balances, upgrade logic or move treasury assets?
- Does the system fail closed, fail open, or enter an ambiguous partially completed state?
- Which data is authoritative: contract state, emitted events, an indexer, an RPC response or an offchain database?
- How would you detect this component behaving incorrectly in production?
- Can the operation be replayed, reordered, reentered or executed on the wrong chain/context?
- What is the smallest privilege or interface that satisfies the requirement?

Architecture habit: For client work, draw the flow before writing code. A five-minute diagram often reveals missing authorization, wrong trust assumptions or unavailable recovery paths earlier than a compiler can.

DAY 03 | SOLIDITY FUNDAMENTALS: STATE, TYPES AND FUNCTIONS

Code Walkthrough and Implementation Notes

Solidity: SimpleBank with custom errors and CEI ordering

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.36;

contract SimpleBank {
    mapping(address => uint256) public balances;

    error ZeroDeposit();
    error InsufficientBalance(uint256 available, uint256 requested);
    error TransferFailed();

    event Deposited(address indexed user, uint256 amount);
    event Withdrawn(address indexed user, uint256 amount);

    function deposit() external payable {
        if (msg.value == 0) revert ZeroDeposit();
        balances[msg.sender] += msg.value;
        emit Deposited(msg.sender, msg.value);
    }

    function withdraw(uint256 amount) external {
        uint256 bal = balances[msg.sender];
        if (amount > bal) revert InsufficientBalance(bal, amount);

        balances[msg.sender] = bal - amount; // effect before interaction
        (bool ok, ) = msg.sender.call{value: amount}("");
        if (!ok) revert TransferFailed();

        emit Withdrawn(msg.sender, amount);
    }
}
```

Walkthrough

The contract updates the user balance before the external value transfer, which follows checks-effects-interactions.

The transfer failure reverts the entire transaction, restoring the previous storage value automatically.

The contract's native balance and the accounting mapping are related but not identical abstractions; forced ETH and direct transfers can complicate naïve invariants.

A production bank-like protocol would need more careful accounting, reentrancy protection, pause/emergency design and extensive tests.

Senior implementation checklist

- Pin the compiler/framework/library versions used by the repository.
- Keep configuration such as chain IDs and dependency addresses outside business logic unless immutability is deliberate.
- Use typed interfaces instead of arbitrary calls whenever practical.
- Emit events for operationally important state changes.
- Write the failure-path test at the same time as the success-path implementation.
- Document what this example intentionally omits before anyone reuses it.

Guided Lab: Build, Observe and Break It

Lab steps

1. Create a Foundry project or use Remix only for the first compile if needed.
2. Implement SimpleBank from scratch without copying the example.
3. Add a receive() function and decide whether direct ETH transfers should count as deposits. Document the choice.
4. Deploy locally and deposit from two accounts.
5. Attempt an over-withdraw and capture the custom error.
6. Withdraw a partial amount and inspect state plus emitted event.
7. Write one invariant you believe the contract should satisfy and identify a scenario that challenges it.
8. Refactor one require string into a custom error and compare readability/testing.

Expected evidence

- Two accounts maintain independent balances.
- An over-withdraw reverts without changing state.
- A successful withdrawal decreases accounting before sending ETH and emits the event.
- Your README clearly explains how receive() behaves.

Troubleshooting guide

If you see a problem: If direct ETH transfers are accepted without updating balances, contract balance can exceed the sum of recorded balances. That is not automatically a bug if documented.

If you see a problem: If withdraw to a contract reverts, the whole withdrawal reverts. Consider whether pull-based or alternative recovery flows are needed in a real system.

If you see a problem: If your frontend displays balances by summing events, reorgs or missed events can create drift; source of truth remains state plus canonical history.

Lab rule: Do not “fix” a failing lab by removing validation or weakening the expected assertion. First identify whether the failure is environment, configuration, integration state or actual code behavior.

Security, Testing, Debugging and Production Readiness

Security review points

- Never use tx.origin for user authorization.
- External calls can reenter; checks-effects-interactions reduces risk but does not replace comprehensive reasoning.
- Private state is publicly observable onchain. Do not store secrets.
- Avoid unbounded loops over user-controlled arrays in state-changing functions.
- Define invariants before adding features; feature accumulation without invariants produces fragile contracts.

Required test plan

- Unit test deposit, zero deposit, partial withdrawal and over-withdraw.
- Test events with exact indexed user and amount.
- Test withdrawal to a receiver contract.
- Fuzz deposit values within practical bounds.
- Add a balance-conservation invariant that accounts for your documented receive behavior.

Debugging playbook

1. Reproduce deterministically: pin block, seed, caller, input and contract version.
2. Read the revert/call trace before changing gas limits or code.
3. Check chain ID, deployed address, ABI and dependency version together.
4. Inspect state immediately before the failing call.
5. Reduce the scenario to the smallest failing sequence.
6. Convert the discovered failure into a regression test.
7. Only then refactor or optimize.

Production gate: A feature is not complete when it compiles. It is complete when its permissions, failure modes, tests, deployment configuration, monitoring signals and rollback/emergency assumptions are understood.

DAY 03 | SOLIDITY FUNDAMENTALS: STATE, TYPES AND FUNCTIONS

Senior Interview Practice, Homework and Portfolio Output

Interview questions - answer aloud first

Q: Why is private not secret in Solidity?

A: private restricts Solidity-level access from other contracts, but blockchain storage is publicly inspectable.

Q: When should you use immutable?

A: For a value set during construction that will never change, such as a trusted dependency address, when upgrade architecture does not require otherwise.

Q: What is checks-effects-interactions?

A: Perform validation, update internal state, then make external calls so reentrant execution sees the updated state.

Q: Why are events important?

A: They provide an efficient offchain observation interface for UIs, indexers, monitoring and incident analysis.

Q: What is the danger of unbounded loops?

A: Gas can grow with user-controlled state until the function becomes impossible to execute within block limits.

Q: What should be designed before functions?

A: State model, actors, permissions, invariants, failure modes and external dependencies.

Homework

- Extend SimpleBank with an administrator-controlled pause using a minimal custom implementation or OpenZeppelin on Day 5.
- Write five invariants for a custodial vault contract.
- Document three differences between contract native balance and internal accounting.

Portfolio deliverable

Ship this: Publish SimpleBank with tests, a state-model diagram and a README section titled "Invariants and known limitations."

Official references for this day

<https://docs.soliditylang.org/en/latest/types.html>

<https://docs.soliditylang.org/en/latest/contracts.html>

<https://docs.soliditylang.org/en/latest/control-structures.html>

Day 04 - Advanced Solidity and Contract Architecture

60-minute goal: Use inheritance, interfaces, libraries, modifiers, low-level calls and storage-aware design without creating brittle or unsafe abstractions.

Learning outcomes

- Explain and apply: interfaces reduce coupling when used well.
- Explain and apply: inheritance is powerful but increases hidden coupling.
- Explain and apply: libraries centralize reusable logic.
- Explain and apply: low-level calls expose raw evm behavior.
- Complete the guided lab and capture reproducible evidence.
- Answer the senior interview questions using architecture/security trade-offs, not memorized definitions.

Minute-by-minute lecture plan

Time	Teaching focus
0-10 min	Interfaces, abstract contracts and dependency boundaries
10-20 min	Inheritance, override rules and composition trade-offs
20-30 min	Libraries, using-for patterns and custom value types
30-40 min	call, staticcall, delegatecall, fallback and receive
40-50 min	Storage layout, packing and assembly awareness
50-60 min	Lab: modular treasury with role interface and safe external execution

Key terms for today

Term	Term	Term
interface	abstract contract	inheritance
override	virtual	library
modifier	fallback	receive
call	staticcall	delegatecall
storage slot	assembly	

Instructor opening script

Today's topic is advanced solidity and contract architecture. The senior-level objective is not to memorize API names; it is to understand the underlying trust model, state transitions and failure modes well enough to make architecture decisions and debug unexpected behavior. During the lecture, repeatedly ask: what state is authoritative, who can change it, which external dependencies are trusted, and what property must remain true after every successful transaction?

Deep Theory and Mental Models

Interfaces reduce coupling when used well

An interface defines the external surface a dependency must provide. Contracts should depend on the smallest interface they need rather than importing an entire implementation. This improves testability, upgrade flexibility and security review because the trust boundary becomes explicit.

Inheritance is powerful but increases hidden coupling

Solidity supports multiple inheritance with linearization. Deep inheritance trees can make initialization, access control and overridden behavior difficult to audit. Senior engineers prefer inheritance for stable reusable behavior and composition for changeable business logic.

Senior lens: Translate this concept into a review question: identify the authoritative state, caller/dependency assumptions, and the invariant that would fail if the concept were implemented incorrectly.

Libraries centralize reusable logic

Libraries can hold pure calculations, validation helpers or logic that operates on structs. Reuse is valuable, but shared logic becomes a systemic dependency. Version pinning and test coverage matter because a subtle arithmetic or validation bug can affect many contracts.

Low-level calls expose raw EVM behavior

`address.call` sends an arbitrary call and returns success plus bytes. `staticcall` forbids state modification for the called frame. `delegatecall` executes another contract's code in the caller's storage context. These primitives are necessary for proxies and integrations but they bypass much of Solidity's type safety.

Senior lens: Translate this concept into a review question: identify the authoritative state, caller/dependency assumptions, and the invariant that would fail if the concept were implemented incorrectly.

Delegatecall makes storage layout part of the API

When code executes through `delegatecall`, storage reads and writes affect the calling contract's slots. A proxy and implementation therefore share a storage layout contract. Reordering variables or changing inheritance can corrupt state across upgrades.

Assembly is a precision tool, not a style preference

Inline assembly can access opcodes and memory directly. It is appropriate when the benefit is measurable and the logic can be tightly tested. It also bypasses compiler safety checks, so unexplained assembly is a security and maintenance cost.

Architecture, Data Flow and Trade-offs

Reference architecture / execution flow

```

Typed integration path
Treasury -> IERC20.transfer() -> Token

Generic execution path
Treasury -> target.call(data) -> arbitrary contract

Proxy path
User -> Proxy -> delegatecall -> Implementation code
      |
      +-- reads/writes Proxy storage
  
```

Trade-off analysis

1. Typed interfaces are safer and easier to audit than arbitrary calldata execution.
2. Generic executor functions are flexible but become high-value privileged attack surfaces.
3. Modifiers improve readability for common guards but should not hide complex state transitions.
4. Packing storage can save gas but can complicate upgrade layouts and increase read-modify-write behavior.
5. Assembly optimizations should be benchmarked; "shorter code" does not guarantee lower gas or greater safety.

Failure-mode questions to ask in a design review

- What happens if the external dependency is unavailable, stale or malicious?
- Can one compromised key change user balances, upgrade logic or move treasury assets?
- Does the system fail closed, fail open, or enter an ambiguous partially completed state?
- Which data is authoritative: contract state, emitted events, an indexer, an RPC response or an offchain database?
- How would you detect this component behaving incorrectly in production?
- Can the operation be replayed, reordered, reentered or executed on the wrong chain/context?
- What is the smallest privilege or interface that satisfies the requirement?

Architecture habit: For client work, draw the flow before writing code. A five-minute diagram often reveals missing authorization, wrong trust assumptions or unavailable recovery paths earlier than a compiler can.

Code Walkthrough and Implementation Notes

Solidity: typed treasury withdrawal with an intentionally separate generic executor

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.36;

interface IERC20Minimal {
    function transfer(address to, uint256 amount) external returns (bool);
}

contract Treasury {
    address public immutable admin;

    error NotAdmin();
    error CallFailed(bytes data);

    constructor(address admin_) {
        require(admin_ != address(0), "zero admin");
        admin = admin_;
    }

    modifier onlyAdmin() {
        if (msg.sender != admin) revert NotAdmin();
        _;
    }

    function transferToken(address token, address to, uint256 amount)
        external onlyAdmin
    {
        bool ok = IERC20Minimal(token).transfer(to, amount);
        require(ok, "token transfer failed");
    }

    function execute(address target, bytes calldata data)
        external onlyAdmin returns (bytes memory result)
    {
        (bool ok, bytes memory out) = target.call(data);
        if (!ok) revert CallFailed(out);
        return out;
    }
}
```

Walkthrough

transferToken is constrained by a typed interface; execute can call almost anything the target exposes.

The generic execute function effectively grants the admin broad authority over assets and interactions. In production this would usually sit behind a multisig/timelock, allowlist, policy engine or be omitted entirely.

A raw token transfer return value is historically inconsistent across tokens; later you will use OpenZeppelin SafeERC20 for better compatibility.

The immutable admin pattern is intentionally simple; upgradeable systems and operational key rotation often require a different access model.

Senior implementation checklist

- Pin the compiler/framework/library versions used by the repository.
- Keep configuration such as chain IDs and dependency addresses outside business logic unless immutability is deliberate.
- Use typed interfaces instead of arbitrary calls whenever practical.
- Emit events for operationally important state changes.
- Write the failure-path test at the same time as the success-path implementation.
- Document what this example intentionally omits before anyone reuses it.

Guided Lab: Build, Observe and Break It

Lab steps

1. Implement the Treasury contract and a MockToken.
2. Write a test proving non-admin callers cannot transfer assets.
3. Encode transfer(address,uint256) calldata manually and call it through execute.
4. Compare the typed transferToken path with the generic execute path from an audit perspective.
5. Create a mock target that reverts with a custom error and observe returned revert bytes.
6. Add a target allowlist to execute and write tests for allowed/disallowed targets.
7. Write a one-page threat model for the admin key.

Expected evidence

- Typed transfers succeed only for the admin.
- The generic executor can reproduce the token transfer when supplied equivalent calldata.
- The allowlist reduces arbitrary target risk but does not validate dangerous function selectors within an allowed target.

Troubleshooting guide

If you see a problem: If a token does not return bool in the expected way, the minimal interface can be incompatible; use SafeERC20 in production integrations.

If you see a problem: Revert bytes from low-level calls may contain Error(string), Panic(uint256), custom errors or arbitrary data.

If you see a problem: A target allowlist is not enough if the allowed contract itself can route calls or upgrade to malicious logic.

Lab rule: Do not “fix” a failing lab by removing validation or weakening the expected assertion. First identify whether the failure is environment, configuration, integration state or actual code behavior.

Security, Testing, Debugging and Production Readiness

Security review points

- Generic external-call capabilities should be treated like root privileges.
- `delegatecall` to user-controlled addresses is usually catastrophic because callee code writes caller storage.
- Fallback functions can make call routing difficult to reason about; document proxy/fallback behavior clearly.
- Inheritance can silently alter storage layout in upgradeable contracts.
- Do not optimize storage or assembly until tests and profiling establish a meaningful benefit.

Required test plan

- Test `onlyAdmin` on every privileged method.
- Fuzz arbitrary calldata to a mock target and ensure failures are surfaced safely.
- Test allowlist updates and privilege boundaries.
- Add a regression test for storage variable positions if preparing for future upgrades.

Debugging playbook

1. Reproduce deterministically: pin block, seed, caller, input and contract version.
2. Read the revert/call trace before changing gas limits or code.
3. Check chain ID, deployed address, ABI and dependency version together.
4. Inspect state immediately before the failing call.
5. Reduce the scenario to the smallest failing sequence.
6. Convert the discovered failure into a regression test.
7. Only then refactor or optimize.

Production gate: A feature is not complete when it compiles. It is complete when its permissions, failure modes, tests, deployment configuration, monitoring signals and rollback/emergency assumptions are understood.

DAY 04 | ADVANCED SOLIDITY AND CONTRACT ARCHITECTURE

Senior Interview Practice, Homework and Portfolio Output

Interview questions - answer aloud first

Q: What is the security difference between call and delegatecall?

A: call executes target code in the target context; delegatecall executes target code in the caller context, including caller storage and address semantics.

Q: Why are interfaces valuable?

A: They define a minimal typed dependency boundary and reduce coupling.

Q: What is a common danger of generic execute functions?

A: They convert one privileged function into arbitrary protocol authority and greatly expand the blast radius of key compromise.

Q: When is assembly justified?

A: When a measurable requirement cannot be met cleanly in Solidity and the assembly can be isolated, documented and thoroughly tested.

Q: Why can inheritance hurt auditability?

A: Behavior and storage may be distributed across parent contracts and affected by override/linearization rules.

Q: Why does storage layout matter before Day 12?

A: Even non-proxy design choices can become future upgrade constraints; understanding layout prevents later corruption.

Homework

- Refactor SimpleBank into an interface + implementation + separate read-only helper.
- Add role rotation to Treasury and discuss why two-step ownership transfer can be safer.
- Write a threat model comparing immutable single admin, Ownable2Step, multisig and timelock.

Portfolio deliverable

Ship this: Publish Treasury with an "Authority surface" table listing each privileged function, who can call it, assets affected and recommended production control.

Official references for this day

<https://docs.soliditylang.org/en/latest/contracts.html#interfaces>

<https://docs.soliditylang.org/en/latest/units-and-global-variables.html>

<https://docs.soliditylang.org/en/latest/assembly.html>

Day 05 - Token Standards, OpenZeppelin and Production ERC Design

60-minute goal: Implement standards-based assets with mature libraries, understand allowance and supply mechanics, and design roles, pause controls and token lifecycle features responsibly.

Learning outcomes

- Explain and apply: standards are behavioral contracts between systems.
- Explain and apply: allowance is delegated spending authority.
- Explain and apply: safeerc20 exists because real tokens are inconsistent.
- Explain and apply: role design is part of token economics and security.
- Complete the guided lab and capture reproducible evidence.
- Answer the senior interview questions using architecture/security trade-offs, not memorized definitions.

Minute-by-minute lecture plan

Time	Teaching focus
0-8 min	Why standards create composability
8-20 min	ERC-20 balances, allowances and transferFrom flow
20-30 min	ERC-721 and ERC-1155 conceptual differences
30-40 min	OpenZeppelin Contracts 5.x architecture and SafeERC20
40-50 min	AccessControl, pausing, caps, permit, vesting and ERC-4626 preview
50-60 min	Lab: production-style capped role-based token

Key terms for today

Term	Term	Term
ERC-20	allowance	approve
transferFrom	ERC-721	ERC-1155
SafeERC20	AccessControl	role admin
Pausable	Permit	EIP-712
vesting	ERC-4626	

Instructor opening script

Today's topic is token standards, openzeppelin and production erc design. The senior-level objective is not to memorize API names; it is to understand the underlying trust model, state transitions and failure modes well enough to make architecture decisions and debug unexpected behavior. During the lecture, repeatedly ask: what state is authoritative, who can change it, which external dependencies are trusted, and what property must remain true after every successful transaction?

Deep Theory and Mental Models

Standards are behavioral contracts between systems

A token standard is more than a list of function names. Wallets, DEXs, bridges, custodians and indexers depend on expected semantics and events. Small deviations can break integrations. Senior engineers begin from established implementations and add the minimum required customization.

Allowance is delegated spending authority

approve gives a spender permission to transfer up to an amount from the owner via transferFrom. This is a persistent authorization state, which is why stale approvals and malicious spenders matter. Frontends should help users understand both the asset transfer and the authorization they are granting.

Senior lens: Translate this concept into a review question: identify the authoritative state, caller/dependency assumptions, and the invariant that would fail if the concept were implemented incorrectly.

SafeERC20 exists because real tokens are inconsistent

Not every deployed ERC-20 behaves perfectly with boolean return values. SafeERC20 wraps common operations to handle contracts that return false or no value in historically common patterns. Production DeFi integrations should not casually assume ideal token behavior.

Role design is part of token economics and security

A MINTER_ROLE, PAUSER_ROLE or DEFAULT_ADMIN_ROLE can have more economic power than the Solidity code itself. Who controls these roles, how they rotate, whether actions are delayed, and what monitoring exists are governance questions as well as code questions.

Senior lens: Translate this concept into a review question: identify the authoritative state, caller/dependency assumptions, and the invariant that would fail if the concept were implemented incorrectly.

Permit changes approval UX, not authorization fundamentals

EIP-2612-style permit uses typed signatures to authorize allowance changes, allowing gasless approval submission by a relayer. It introduces nonce, domain-separator and signature-verification concerns. Signed intent still must be scoped to prevent replay.

ERC-4626 standardizes tokenized vault interfaces

Vault standards improve composability around deposits, shares, withdrawals and asset accounting. They also introduce subtle rounding and donation/inflation attack considerations. Day 19 will use this thinking in the capstone.

Architecture, Data Flow and Trade-offs

Reference architecture / execution flow

```

Owner tokens
  | approve(spender, N)
  v
Allowance[owner][spender] = N
  |
Spender calls transferFrom(owner, recipient, x)
  | checks x <= allowance and owner balance
  v
Balances updated + Transfer event
Allowance decreased (subject to implementation semantics)

```

Trade-off analysis

1. Use battle-tested implementations unless a requirement truly demands custom token mechanics.
2. A cap limits future minting but does not decide who is allowed to mint; access control is separate.
3. Pause powers are operational controls that must have clear governance and incident-response policy.
4. Token decimals affect display and arithmetic expectations but do not change integer storage fundamentals.
5. Permit improves UX but signature domains, chain IDs and nonces must be handled correctly.

Failure-mode questions to ask in a design review

- What happens if the external dependency is unavailable, stale or malicious?
- Can one compromised key change user balances, upgrade logic or move treasury assets?
- Does the system fail closed, fail open, or enter an ambiguous partially completed state?
- Which data is authoritative: contract state, emitted events, an indexer, an RPC response or an offchain database?
- How would you detect this component behaving incorrectly in production?
- Can the operation be replayed, reordered, reentered or executed on the wrong chain/context?
- What is the smallest privilege or interface that satisfies the requirement?

Architecture habit: For client work, draw the flow before writing code. A five-minute diagram often reveals missing authorization, wrong trust assumptions or unavailable recovery paths earlier than a compiler can.

Code Walkthrough and Implementation Notes

Solidity: capped role-based ERC-20 using OpenZeppelin Contracts 5.x

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.36;

import {ERC20} from "@openzeppelin/contracts/token/ERC20/ERC20.sol";
import {ERC20Capped} from "@openzeppelin/contracts/token/ERC20/extensions/ERC20Capped.sol";
import {ERC20Pausable} from "@openzeppelin/contracts/token/ERC20/extensions/ERC20Pausable.sol";
import {AccessControl} from "@openzeppelin/contracts/access/AccessControl.sol";

contract ProfessionalToken is ERC20, ERC20Capped, ERC20Pausable, AccessControl {
    bytes32 public constant MINTER_ROLE = keccak256("MINTER_ROLE");
    bytes32 public constant PAUSER_ROLE = keccak256("PAUSER_ROLE");

    constructor(address admin, uint256 cap_)
        ERC20("Professional Token", "PRO")
        ERC20Capped(cap_)
    {
        _grantRole(DEFAULT_ADMIN_ROLE, admin);
        _grantRole(MINTER_ROLE, admin);
        _grantRole(PAUSER_ROLE, admin);
    }

    function mint(address to, uint256 amount) external onlyRole(MINTER_ROLE) {
        _mint(to, amount);
    }

    function pause() external onlyRole(PAUSER_ROLE) { _pause(); }
    function unpause() external onlyRole(PAUSER_ROLE) { _unpause(); }

    function _update(address from, address to, uint256 value)
        internal override(ERC20, ERC20Capped, ERC20Pausable)
    {
        super._update(from, to, value);
    }
}
```

Walkthrough

OpenZeppelin 5.x consolidates many transfer/mint/burn customizations through `_update`, so multiple extensions may require an explicit override.

`DEFAULT_ADMIN_ROLE` can grant and revoke roles; it is exceptionally powerful and should not remain on an ordinary hot wallet in a high-value system.

A cap is denominated in smallest token units. Deployment scripts should calculate `cap_` using `10 ** decimals()` or `parseUnits-` style helpers.

Pause behavior should be documented for mint, burn and transfers and covered by tests rather than assumed.

Senior implementation checklist

- Pin the compiler/framework/library versions used by the repository.
- Keep configuration such as chain IDs and dependency addresses outside business logic unless immutability is deliberate.
- Use typed interfaces instead of arbitrary calls whenever practical.
- Emit events for operationally important state changes.
- Write the failure-path test at the same time as the success-path implementation.
- Document what this example intentionally omits before anyone reuses it.

Guided Lab: Build, Observe and Break It

Lab steps

1. Install OpenZeppelin Contracts in a Foundry or Hardhat project and pin the dependency version.
2. Implement ProfessionalToken and write deployment parameters for a 100 million token cap with 18 decimals.
3. Mint to two accounts and confirm totalSupply.
4. Attempt to mint above the remaining cap.
5. Pause and test transfer behavior.
6. Revoke MINTER_ROLE from one account and verify minting fails immediately.
7. Grant a separate pauser account and discuss separation of duties.
8. Use SafeERC20 in a small Treasury integration to transfer the token.
9. Document every privileged role and recommended production holder.

Expected evidence

- Minting stops at the cap.
- Non-minters cannot mint.
- Pause behavior matches your documented policy and tests.
- Role changes emit AccessControl events that an offchain monitor can observe.

Troubleshooting guide

If you see a problem: If compilation reports an override conflict, inspect the inheritance tree and current OpenZeppelin API rather than guessing the override list.

If you see a problem: If cap arithmetic overflows in deployment tooling, parse human-readable units with a bigint-safe library.

If you see a problem: If transfers fail only while paused, verify whether your test expected mint/burn exceptions that the extension intentionally does not provide.

Lab rule: Do not “fix” a failing lab by removing validation or weakening the expected assertion. First identify whether the failure is environment, configuration, integration state or actual code behavior.

Security, Testing, Debugging and Production Readiness

Security review points

- A token admin key is an economic control surface; protect it like treasury infrastructure.
- Avoid fee-on-transfer, rebasing or blacklist mechanics unless the product truly needs them because they reduce composability and add integration assumptions.
- Never claim “renounced ownership” if other roles, proxy admins or upgrade paths still retain control.
- Document supply policy: initial mint, future mint authority, cap, burn behavior and emergency controls.
- Use multisig/timelock patterns for high-value administrative roles.

Required test plan

- Test cap boundary exactly at cap and one unit above.
- Test mint authorization and role revocation.
- Test pause/unpause across transfers, minting and burning according to current extension behavior.
- Fuzz transfers and allowances.
- Add an invariant that totalSupply never exceeds cap.

Debugging playbook

1. Reproduce deterministically: pin block, seed, caller, input and contract version.
2. Read the revert/call trace before changing gas limits or code.
3. Check chain ID, deployed address, ABI and dependency version together.
4. Inspect state immediately before the failing call.
5. Reduce the scenario to the smallest failing sequence.
6. Convert the discovered failure into a regression test.
7. Only then refactor or optimize.

Production gate: A feature is not complete when it compiles. It is complete when its permissions, failure modes, tests, deployment configuration, monitoring signals and rollback/emergency assumptions are understood.

Senior Interview Practice, Homework and Portfolio Output

Interview questions - answer aloud first

Q: Why use OpenZeppelin instead of writing ERC-20 from scratch?

A: It provides widely reviewed, standards-oriented implementations and extensions, reducing unnecessary custom security surface.

Q: What does allowance represent?

A: Persistent delegated spending authority from an owner to a spender.

Q: Why is `DEFAULT_ADMIN_ROLE` dangerous?

A: It can manage other roles and often represents ultimate authority over minting, pausing or upgrades.

Q: What problem does `SafeERC20` address?

A: Compatibility with ERC-20 implementations that do not consistently return standard boolean values.

Q: What does a token cap guarantee?

A: Only that total supply cannot exceed the cap; it does not specify who may mint or when.

Q: What additional risks come with permit?

A: Signature replay/domain mistakes, nonce handling and unsafe user signing UX.

Homework

- Add ERC20Permit to a separate branch and write a test for a signed permit.
- Research ERC-4626 and summarize deposit, mint, withdraw and redeem semantics.
- Create a token-governance matrix: action, role, recommended holder, delay, monitoring event.

Portfolio deliverable

Ship this: Publish “Production ERC-20 System” with tests, role matrix, supply policy, deployment script and a security assumptions section.

Official references for this day

<https://docs.openzeppelin.com/contracts/5.x>

<https://docs.openzeppelin.com/contracts/5.x/erc20>

<https://docs.openzeppelin.com/contracts/5.x/access-control>

<https://docs.openzeppelin.com/contracts/5.x/api/token/ERC20>

Day 06 - Foundry, Hardhat, Testing and Deployment Workflows

60-minute goal: Operate a professional smart-contract development workflow with reproducible builds, unit tests, fuzzing, invariant testing, fork tests, deployment scripts and verification.

Learning outcomes

- Explain and apply: testing is executable protocol reasoning.
- Explain and apply: unit tests isolate expected behavior.
- Explain and apply: fuzzing searches input space automatically.
- Explain and apply: invariant testing searches call sequences.
- Complete the guided lab and capture reproducible evidence.
- Answer the senior interview questions using architecture/security trade-offs, not memorized definitions.

Minute-by-minute lecture plan

Time	Teaching focus
0-8 min	Foundry toolchain: forge, cast, anvil and chisel
8-18 min	Project structure, dependency pinning and remappings
18-30 min	Unit tests, cheatcodes, events and revert assertions
30-42 min	Fuzz testing, invariant testing and stateful handlers
42-52 min	Fork testing, mainnet state and integration confidence
52-60 min	Deployment scripts, verification and CI quality gates

Key terms for today

Term	Term	Term
forge	cast	anvil
cheatcode	fixture	fuzz test
invariant	handler	fork test
snapshot	gas report	deployment script
verification	CI	

Instructor opening script

Today's topic is foundry, hardhat, testing and deployment workflows. The senior-level objective is not to memorize API names; it is to understand the underlying trust model, state transitions and failure modes well enough to make architecture decisions and debug unexpected behavior. During the lecture, repeatedly ask: what state is authoritative, who can change it, which external dependencies are trusted, and what property must remain true after every successful transaction?

Deep Theory and Mental Models

Testing is executable protocol reasoning

A good smart-contract test suite does more than verify happy paths. It encodes assumptions about permissions, accounting, error boundaries, lifecycle states and integrations. In security-sensitive code, tests are part of the design process because writing assertions forces vague requirements into precise properties.

Unit tests isolate expected behavior

Unit tests are deterministic examples with controlled inputs and expected outputs. They are excellent for role checks, boundary conditions, events and exact state transitions. Their weakness is that humans choose the examples, so hidden combinations can remain unexplored.

Senior lens: Translate this concept into a review question: identify the authoritative state, caller/dependency assumptions, and the invariant that would fail if the concept were implemented incorrectly.

Fuzzing searches input space automatically

Property-based fuzz tests let the framework generate many inputs. The valuable skill is not enabling fuzzing; it is choosing properties and bounds that reflect the domain. A meaningless property can pass millions of cases while the protocol remains unsafe.

Invariant testing searches call sequences

Stateful invariant tests generate sequences of operations against one or more handlers. This is powerful for vaults, exchanges and accounting systems where bugs emerge only after a particular series of deposits, withdrawals, price changes or role actions. Foundry's current documentation treats invariants as first-class testing workflows.

Senior lens: Translate this concept into a review question: identify the authoritative state, caller/dependency assumptions, and the invariant that would fail if the concept were implemented incorrectly.

Fork tests validate assumptions against live interfaces

A fork reproduces chain state at a chosen block and allows local execution against real deployed contracts. It is ideal for integration ABI compatibility, decimals, token behavior and protocol interactions. A fork is not a production guarantee: governance, balances and state can change after the pinned block.

Deployment is a controlled state transition too

Production deployment should be scripted, reviewable and reproducible. Address derivation, constructor arguments, role grants, proxy initialization, ownership transfer and verification should be treated as a sequence with explicit preconditions and post-deployment assertions.

Architecture, Data Flow and Trade-offs

Reference architecture / execution flow

```

Developer change
|
forge fmt / compile
|
unit tests -> fuzz tests -> invariant tests
|
+----- fork integration tests
|
coverage / gas / static analysis
|
scripted deploy to test environment
|
post-deploy assertions + verification
|
reviewed production deployment

```

Trade-off analysis

1. Fast tests encourage frequent execution, but speed must not replace meaningful properties.
2. Fork tests improve integration confidence while increasing dependence on RPC availability and pinned state.
3. High line coverage can coexist with weak assertions; coverage is a signal, not proof.
4. Deployment scripts reduce human error but must themselves be reviewed and protected from wrong-network or wrong-key execution.
5. CI should fail closed on compiler/test failures and should never expose signing secrets to untrusted pull requests.

Failure-mode questions to ask in a design review

- What happens if the external dependency is unavailable, stale or malicious?
- Can one compromised key change user balances, upgrade logic or move treasury assets?
- Does the system fail closed, fail open, or enter an ambiguous partially completed state?
- Which data is authoritative: contract state, emitted events, an indexer, an RPC response or an offchain database?
- How would you detect this component behaving incorrectly in production?
- Can the operation be replayed, reordered, reentered or executed on the wrong chain/context?
- What is the smallest privilege or interface that satisfies the requirement?

Architecture habit: For client work, draw the flow before writing code. A five-minute diagram often reveals missing authorization, wrong trust assumptions or unavailable recovery paths earlier than a compiler can.

Code Walkthrough and Implementation Notes

Foundry: unit + fuzz example for a bounded vault-like counter

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.36;

import {Test} from "forge-std/Test.sol";
import {Counter} from "../src/Counter.sol";

contract CounterTest is Test {
    Counter counter;

    function setUp() public {
        counter = new Counter();
    }

    function testIncrement() public {
        counter.increment(2);
        assertEq(counter.count(), 2);
    }

    function test_RevertWhen_ZeroAmount() public {
        vm.expectRevert();
        counter.increment(0);
    }

    function testFuzz_Increment(uint96 amount) public {
        vm.assume(amount > 0);
        counter.increment(amount);
        assertEq(counter.count(), uint256(amount));
    }
}
```

Walkthrough

The setUp fixture creates a fresh contract before each test so unit tests do not leak state into one another.

Fuzz types can be deliberately narrower than uint256 when the application domain has practical bounds.

vm.assume filters generated inputs, but excessive assumptions can make fuzzing ineffective; bound values when appropriate.

Tests should assert both state and observable effects such as events, token balances and authorization changes.

Senior implementation checklist

- Pin the compiler/framework/library versions used by the repository.
- Keep configuration such as chain IDs and dependency addresses outside business logic unless immutability is deliberate.
- Use typed interfaces instead of arbitrary calls whenever practical.
- Emit events for operationally important state changes.
- Write the failure-path test at the same time as the success-path implementation.
- Document what this example intentionally omits before anyone reuses it.

Guided Lab: Build, Observe and Break It

Lab steps

1. Create a new Foundry repository with src, test and script folders.
2. Move ProfessionalToken from Day 5 into src and create a deployment script.
3. Write unit tests for mint, cap, pause and role changes.
4. Add at least two fuzz tests: transfer amount and mint amount bounded by remaining cap.
5. Add an invariant that totalSupply <= cap at all times.
6. Create a handler that alternates mint, transfer and burn if your token supports burn.
7. Fork a public EVM network at a fixed block and read a known ERC-20 balance/decimals to prove your fork works.
8. Run forge test -vvvv on one failing test and learn to read the call trace.
9. Generate a gas snapshot and record the most expensive state-changing method.
10. Add a GitHub Actions or local CI script that runs format check, build and tests.

Expected evidence

- All deterministic tests pass from a clean checkout.
- The invariant runner executes multiple sequences without violating total supply constraints.
- The fork test pins a block so results can be reproduced later.
- Deployment script refuses or clearly reports an unexpected chain ID.

Troubleshooting guide

If you see a problem: If a fork test becomes flaky, pin both block number and RPC source behavior; do not rely on latest state.

If you see a problem: If fuzzing finds a counterexample, save the seed/input and convert it to a deterministic regression test.

If you see a problem: If tests pass locally but fail in CI, compare compiler version, optimizer settings, environment variables and dependency lock state.

If you see a problem: If a deployment script estimates unexpectedly high gas, inspect broadcast trace and constructor behavior rather than only raising limits.

Lab rule: Do not “fix” a failing lab by removing validation or weakening the expected assertion. First identify whether the failure is environment, configuration, integration state or actual code behavior.

Security, Testing, Debugging and Production Readiness

Security review points

- Never expose production private keys in forge command history, CI logs or repository secrets accessible to untrusted workflows.
- Use separate deployer, admin and treasury roles when operational requirements justify separation.
- Forking mainnet does not make calls harmless if your script can accidentally switch to a live RPC; build explicit chain guards.
- Test privileged failure paths as aggressively as normal user paths.
- Maintain regression tests for every security bug you discover.

Required test plan

- Unit tests for all externally callable methods.
- Fuzz tests for value ranges and address combinations.
- At least one stateful invariant.
- Fork test for any third-party protocol integration.
- Deployment postconditions for roles, caps and ownership.

Debugging playbook

1. Reproduce deterministically: pin block, seed, caller, input and contract version.
2. Read the revert/call trace before changing gas limits or code.
3. Check chain ID, deployed address, ABI and dependency version together.
4. Inspect state immediately before the failing call.
5. Reduce the scenario to the smallest failing sequence.
6. Convert the discovered failure into a regression test.
7. Only then refactor or optimize.

Production gate: A feature is not complete when it compiles. It is complete when its permissions, failure modes, tests, deployment configuration, monitoring signals and rollback/emergency assumptions are understood.

Senior Interview Practice, Homework and Portfolio Output

Interview questions - answer aloud first

Q: What is the difference between fuzz and invariant testing?

A: Fuzz tests vary inputs to a specific test flow; invariant testing explores sequences of state-changing actions while checking properties that must always hold.

Q: Why pin a fork block?

A: To make integration tests reproducible instead of depending on moving live state.

Q: Does high coverage mean a contract is secure?

A: No. Coverage measures executed code, not the quality of assertions or whether the design properties are complete.

Q: Why use deployment scripts?

A: They make configuration and state transitions reproducible, reviewable and testable.

Q: What should happen when fuzzing finds a bug?

A: Preserve the counterexample, fix the contract, and add a deterministic regression test in addition to the property test.

Q: What is a useful invariant for a capped token?

A: totalSupply must never exceed cap under any allowed sequence of operations.

Homework

- Create a CI quality checklist for every future portfolio repository.
- Write one invariant for SimpleBank, ProfessionalToken and Treasury.
- Prepare a 3-minute explanation of why fork tests are useful but not sufficient.

Portfolio deliverable

Ship this: Upgrade the Day 5 token repository to include unit, fuzz, invariant, fork, deployment and CI artifacts. This is a major portfolio differentiator.

Official references for this day

<https://www.getfoundry.sh/>

<https://www.getfoundry.sh/forging/testing>

<https://www.getfoundry.sh/guides/invariant-testing>

<https://www.getfoundry.sh/guides/fork-testing>

Day 07 - Smart Contract Security I: Reentrancy, Authorization and External Calls

60-minute goal: Learn to identify and exploit common Solidity vulnerabilities in a controlled local environment, then redesign the contract around explicit invariants and safe interaction patterns.

Learning outcomes

- Explain and apply: security begins with assets and invariants.
- Explain and apply: reentrancy is unexpected control flow.
- Explain and apply: authorization bugs are often simpler than cryptography.
- Explain and apply: tx.origin is not an authentication primitive.
- Complete the guided lab and capture reproducible evidence.
- Answer the senior interview questions using architecture/security trade-offs, not memorized definitions.

Minute-by-minute lecture plan

Time	Teaching focus
0-8 min	Security mindset: assets, actors, trust and invariants
8-20 min	Reentrancy and control-flow attacks
20-30 min	Authorization failures, tx.origin and role mistakes
30-40 min	External-call return handling and denial of service
40-50 min	Front-running, approvals and user-intent risks
50-60 min	Local lab: exploit and repair VulnerableBank

Key terms for today

Term	Term	Term
threat model	reentrancy	CEI
reentrancy guard	tx.origin	authorization
external call	pull payment	front-running
approval risk	DoS	attack surface

Instructor opening script

Today's topic is smart contract security i: reentrancy, authorization and external calls. The senior-level objective is not to memorize API names; it is to understand the underlying trust model, state transitions and failure modes well enough to make architecture decisions and debug unexpected behavior. During the lecture, repeatedly ask: what state is authoritative, who can change it, which external dependencies are trusted, and what property must remain true after every successful transaction?

DAY 07 | SMART CONTRACT SECURITY I: REENTRANCY, AUTHORIZATION AND EXTERNAL CALLS

Deep Theory and Mental Models

Security begins with assets and invariants

Before searching for named vulnerabilities, identify what can be lost or corrupted: native currency, tokens, debt accounting, governance power, upgrade authority or user availability. Then define invariants that protect those assets. Named attack classes are useful, but invariants reveal protocol-specific failures.

Reentrancy is unexpected control flow

An external call can execute arbitrary receiver code before the original function finishes. If internal state still reflects the pre-call condition, the receiver may call back and repeat an operation. Reentrancy can be same-function, cross-function or cross-contract; a single `nonReentrant` modifier is not a substitute for architecture analysis.

Senior lens: Translate this concept into a review question: identify the authoritative state, caller/dependency assumptions, and the invariant that would fail if the concept were implemented incorrectly.

Authorization bugs are often simpler than cryptography

A missing `onlyRole`, an incorrect role admin, an initializer callable twice or a user-supplied privileged address can be catastrophic. Every privileged function should have a clear caller policy and tests proving both allowed and denied cases.

tx.origin is not an authentication primitive

`tx.origin` identifies the original EOA at the root of the call chain, not the immediate caller. A malicious intermediate contract can trick a user into initiating a transaction and then call a victim that authorizes by `tx.origin`. Use `msg.sender` and explicit delegation/signature schemes instead.

Senior lens: Translate this concept into a review question: identify the authoritative state, caller/dependency assumptions, and the invariant that would fail if the concept were implemented incorrectly.

External-call failure semantics must be intentional

Low-level calls return a success boolean and data. Token calls may have nonstandard behavior. Receiver reverts can block push-based payout loops. Senior designs decide whether one failed recipient should revert an entire operation or whether users should claim funds individually.

MEV changes the threat model for public transactions

Pending transactions can reveal intent. Attackers may reorder around swaps, liquidations, mints or governance actions when economically useful. Slippage bounds, deadlines, commit-reveal schemes, private order flow and protocol-specific protections can reduce risk, but each adds trade-offs.

Architecture, Data Flow and Trade-offs

Reference architecture / execution flow

```

Vulnerable withdrawal
check balance
|
call user with ETH <-- attacker fallback reenters here
|
update balance          +--> withdraw again

Safer ordering
check -> update accounting -> external call
|
+-- reentrant call sees reduced balance
  
```

Trade-off analysis

1. CEI is simple and effective for many flows but does not cover every cross-function or cross-contract reentrancy pattern.
2. Pull payments reduce receiver-induced denial of service but require users to make an extra claim transaction.
3. Role-based security improves separation of duties but increases configuration complexity.
4. Private transaction submission may reduce public mempool exposure but adds provider/trust assumptions.
5. Pausing can contain incidents but creates a privileged central control that must be governed.

Failure-mode questions to ask in a design review

- What happens if the external dependency is unavailable, stale or malicious?
- Can one compromised key change user balances, upgrade logic or move treasury assets?
- Does the system fail closed, fail open, or enter an ambiguous partially completed state?
- Which data is authoritative: contract state, emitted events, an indexer, an RPC response or an offchain database?
- How would you detect this component behaving incorrectly in production?
- Can the operation be replayed, reordered, reentered or executed on the wrong chain/context?
- What is the smallest privilege or interface that satisfies the requirement?

Architecture habit: For client work, draw the flow before writing code. A five-minute diagram often reveals missing authorization, wrong trust assumptions or unavailable recovery paths earlier than a compiler can.

DAY 07 | SMART CONTRACT SECURITY I: REENTRANCY, AUTHORIZATION AND EXTERNAL CALLS

Code Walkthrough and Implementation Notes

Solidity: intentionally vulnerable bank for local exploitation

```
// EDUCATIONAL / LOCAL TESTING ONLY
pragma solidity ^0.8.36;

contract VulnerableBank {
    mapping(address => uint256) public balances;

    function deposit() external payable {
        balances[msg.sender] += msg.value;
    }

    function withdraw() external {
        uint256 amount = balances[msg.sender];
        require(amount > 0, "empty");

        // VULNERABILITY: external interaction before state update
        (bool ok, ) = msg.sender.call{value: amount}("");
        require(ok, "send failed");

        balances[msg.sender] = 0;
    }
}

// Repair: set balances[msg.sender] = 0 before the call,
// then add appropriate reentrancy protection and tests.
```

Walkthrough

The bug is not “call is always unsafe.” The bug is that an attacker-controlled external call occurs while the bank still records a withdrawable balance.

A local attacker contract can deposit once, call withdraw, and reenter from receive/fallback before the first frame updates storage.

The educational goal is to observe call traces and broken invariants, then encode a regression test that fails on the vulnerable version and passes on the fixed version.

Do not deploy vulnerable examples to public networks with real value.

Senior implementation checklist

- Pin the compiler/framework/library versions used by the repository.
- Keep configuration such as chain IDs and dependency addresses outside business logic unless immutability is deliberate.
- Use typed interfaces instead of arbitrary calls whenever practical.
- Emit events for operationally important state changes.
- Write the failure-path test at the same time as the success-path implementation.
- Document what this example intentionally omits before anyone reuses it.

Guided Lab: Build, Observe and Break It

Lab steps

1. Create VulnerableBank and fund it only on a local Anvil chain.
2. Write an attacker contract whose receive() reenters while the bank still has sufficient balance.
3. Write a test showing the bank loses more than the attacker's recorded deposit.
4. Inspect the forge -vvvv trace and identify nested call frames.
5. Repair using checks-effects-interactions.
6. Add OpenZeppelin ReentrancyGuard or an equivalent guard and explain what additional protection it provides.
7. Add a second withdrawal-like function and reason about cross-function reentrancy.
8. Write authorization tests proving no admin-only function can be reached by normal users.
9. Create a SECURITY.md that explains the exploit and fix.

Expected evidence

- The vulnerable test demonstrates invariant failure locally.
- The repaired contract passes the same exploit regression test.
- The student can point to the exact line where control leaves the contract.
- The security note explains why both state ordering and a guard can be useful.

Troubleshooting guide

If you see a problem: If the attack does not reenter, make sure the attacker is a contract and its receive/fallback calls withdraw again.

If you see a problem: If arithmetic underflow occurs after reentrancy on modern Solidity, that may cause the whole transaction to revert and hide the intended demo; choose the state update structure carefully for a reproducible local example.

If you see a problem: If a receiver reverts, the bank's call fails; decide whether that denial of service is acceptable for the intended design.

Lab rule: Do not "fix" a failing lab by removing validation or weakening the expected assertion. First identify whether the failure is environment, configuration, integration state or actual code behavior.

Security, Testing, Debugging and Production Readiness

Security review points

- Prefer explicit authorization checks on every privileged entry point.
- Assume external calls can execute hostile code unless the callee is provably constrained.
- Treat token callbacks, ERC-777-style hooks, safe NFT receiver callbacks and proxy fallback behavior as potential reentrancy edges.
- Approve only necessary token amounts and carefully handle allowance changes.
- Threat-model user transaction ordering where price or scarce allocation matters.

Required test plan

- Exploit regression test.
- Fixed-contract success and failure tests.
- Cross-function reentrancy reasoning test or scenario.
- Authorization matrix tests.
- Event and state assertions after failed external calls.

Debugging playbook

1. Reproduce deterministically: pin block, seed, caller, input and contract version.
2. Read the revert/call trace before changing gas limits or code.
3. Check chain ID, deployed address, ABI and dependency version together.
4. Inspect state immediately before the failing call.
5. Reduce the scenario to the smallest failing sequence.
6. Convert the discovered failure into a regression test.
7. Only then refactor or optimize.

Production gate: A feature is not complete when it compiles. It is complete when its permissions, failure modes, tests, deployment configuration, monitoring signals and rollback/emergency assumptions are understood.

DAY 07 | SMART CONTRACT SECURITY I: REENTRANCY, AUTHORIZATION AND EXTERNAL CALLS

Senior Interview Practice, Homework and Portfolio Output

Interview questions - answer aloud first

Q: What makes reentrancy possible?

A: External control transfer before the caller has established a state that is safe to observe if execution returns recursively.

Q: Is nonReentrant enough by itself?

A: No. It is a useful guard, but architecture, cross-contract calls and separate entry points still require reasoning.

Q: Why is tx.origin dangerous?

A: It can remain the victim user even when an attacker-controlled intermediate contract is msg.sender.

Q: What is a pull payment?

A: Funds are credited in storage and recipients explicitly claim them, reducing the chance that one receiver blocks a batch payout.

Q: What should every privileged method have?

A: A defined authorization policy, tests for allowed and denied callers, and operational ownership/governance documentation.

Q: How does MEV affect security?

A: Observable pending intent may be reordered or surrounded for profit, affecting prices, allocations and user outcomes.

Homework

- Write a professional vulnerability finding for the reentrancy bug with title, severity, impact, proof of concept and remediation.
- Audit yesterday's token for authorization and pause-role risks.
- List every external call in your current repositories and classify whether the callee is trusted, semi-trusted or user-controlled.

Portfolio deliverable

Ship this: Publish a local "Smart Contract Security Lab 01" with vulnerable code, exploit test, patched code and an audit-style finding. Clearly label it educational.

Official references for this day

<https://docs.soliditylang.org/en/latest/security-considerations.html>

<https://docs.openzeppelin.com/contracts/5.x/api/utils#ReentrancyGuard>

<https://ethereum.org/en/developers/docs/mev/>

Day 08 - Smart Contract Security II: Auditing, Fuzzing and Invariants

60-minute goal: Perform a structured smart-contract review using threat modeling, manual code analysis, static analysis, fuzzing, invariant tests and professional finding reports.

Learning outcomes

- Explain and apply: an audit starts by reconstructing intent.
- Explain and apply: severity combines impact and exploitability.
- Explain and apply: manual review follows state transitions.
- Explain and apply: static analysis is a force multiplier.
- Complete the guided lab and capture reproducible evidence.
- Answer the senior interview questions using architecture/security trade-offs, not memorized definitions.

Minute-by-minute lecture plan

Time	Teaching focus
0-8 min	Audit scope, assumptions and architecture reconstruction
8-18 min	Assets, actors, trust boundaries and privilege graph
18-30 min	Manual review: state transitions, external calls, accounting and signatures
30-40 min	Static analysis with Slither and interpreting signal versus noise
40-50 min	Fuzz/invariant strategy and economic properties
50-60 min	Write a professional audit finding and executive summary

Key terms for today

Term	Term	Term
scope	threat model	invariant
severity	impact	likelihood
proof of concept	Slither	static analysis
false positive	attack path	economic invariant
audit report		

Instructor opening script

Today's topic is smart contract security ii: auditing, fuzzing and invariants. The senior-level objective is not to memorize API names; it is to understand the underlying trust model, state transitions and failure modes well enough to make architecture decisions and debug unexpected behavior. During the lecture, repeatedly ask: what state is authoritative, who can change it, which external dependencies are trusted, and what property must remain true after every successful transaction?

Deep Theory and Mental Models

An audit starts by reconstructing intent

Reviewers cannot judge correctness without understanding what the protocol is supposed to do. Read specifications, diagrams, deployment scripts and tests before line-by-line review. Build an actor list, privilege map and asset-flow diagram. Missing or contradictory specifications are themselves risk signals.

Severity combines impact and exploitability

A high-impact bug with impossible preconditions is different from a permissionless immediate drain. Severity frameworks vary, so explain assumptions: funds at risk, privilege needed, user interaction, timing, probability and recoverability. Avoid inflating severity to make a report look impressive.

Senior lens: Translate this concept into a review question: identify the authoritative state, caller/dependency assumptions, and the invariant that would fail if the concept were implemented incorrectly.

Manual review follows state transitions

For every externally callable function ask: who can call, what state is read, what state changes, what external calls occur, what assets move, what events are emitted, and what assumptions are made about dependencies? Then connect functions into sequences rather than reviewing each in isolation.

Static analysis is a force multiplier

Tools such as Slither detect patterns, inheritance issues, unused code, dangerous calls and known bug shapes quickly. Tool output is not an audit report. Findings require context validation; false positives should be triaged, and undetected protocol-logic bugs remain possible.

Senior lens: Translate this concept into a review question: identify the authoritative state, caller/dependency assumptions, and the invariant that would fail if the concept were implemented incorrectly.

Economic invariants capture protocol truth

Examples include total shares matching accounting, collateral value remaining above debt under required rules, reserves not becoming negative, and users never withdrawing more claim than they own. These properties are often more valuable than testing individual functions.

Reports should be reproducible and actionable

A strong finding names the issue, identifies affected code, states preconditions, explains impact, provides a minimal proof of concept, and recommends a remediation without redesigning the entire protocol unless necessary. The report should help engineers reproduce the issue quickly.

Architecture, Data Flow and Trade-offs

Reference architecture / execution flow

```

Audit workflow
Scope + commit hash
  v
Architecture reconstruction
  v
Assets / actors / privileges / dependencies
  v
Manual review <-> static analysis
  v
Unit / fuzz / invariant / fork validation
  v
Exploitability + severity assessment
  v
Findings + remediation + regression tests
  v
Final report + residual risk

```

Trade-off analysis

1. Automated scanners increase coverage breadth but can create distraction if reviewers do not triage.
2. A narrow fixed audit scope improves certainty but may miss deployment, frontend or governance risks outside the repository.
3. Invariant testing is powerful but only checks properties the reviewer thought to write.
4. Public bug disclosure improves ecosystem learning but must follow responsible-disclosure rules for live systems.
5. A clean audit does not mean “no bugs”; it means no material issues were identified under the defined scope and effort.

Failure-mode questions to ask in a design review

- What happens if the external dependency is unavailable, stale or malicious?
- Can one compromised key change user balances, upgrade logic or move treasury assets?
- Does the system fail closed, fail open, or enter an ambiguous partially completed state?
- Which data is authoritative: contract state, emitted events, an indexer, an RPC response or an offchain database?
- How would you detect this component behaving incorrectly in production?
- Can the operation be replayed, reordered, reentered or executed on the wrong chain/context?
- What is the smallest privilege or interface that satisfies the requirement?

Architecture habit: For client work, draw the flow before writing code. A five-minute diagram often reveals missing authorization, wrong trust assumptions or unavailable recovery paths earlier than a compiler can.

DAY 08 | SMART CONTRACT SECURITY II: AUDITING, FUZZING AND INVARIANTS

Code Walkthrough and Implementation Notes

Foundry invariant sketch: handler + accounting property

```
contract VaultHandler {
    Vault public vault;
    address[] public actors;

    constructor(Vault vault_) {
        vault = vault_;
    }

    function deposit(uint96 amount) external {
        amount = uint96(bound(amount, 1, 100 ether));
        // In a real handler: choose actor, fund tokens, approve, deposit
    }

    function withdraw(uint96 shares) external {
        // Bound shares by the selected actor's actual balance
    }
}

contract VaultInvariantTest is Test {
    function invariant_assetsCoverClaims() public view {
        assertGe(asset.balanceOf(address(vault)), vault.accountedAssets());
    }
}
```

Walkthrough

The handler constrains random actions into valid domain operations; otherwise the invariant engine may spend most runs on meaningless immediate reverts.

An accounting invariant must match the protocol model. If assets are intentionally deployed into strategies, checking only the vault's local token balance would be wrong.

The best invariant is often stated before code exists. Implementation details may change; the property remains the design contract.

Use ghost variables in test handlers when onchain state does not directly expose enough information to validate a property.

Senior implementation checklist

- Pin the compiler/framework/library versions used by the repository.
- Keep configuration such as chain IDs and dependency addresses outside business logic unless immutability is deliberate.
- Use typed interfaces instead of arbitrary calls whenever practical.
- Emit events for operationally important state changes.
- Write the failure-path test at the same time as the success-path implementation.
- Document what this example intentionally omits before anyone reuses it.

Guided Lab: Build, Observe and Break It

Lab steps

1. Choose one prior contract as the formal audit target and record an exact Git commit hash.
2. Write a one-page scope and architecture summary.
3. Create an actor/privilege table including admin, minter, pauser, user and external token.
4. Run Slither and categorize results: valid issue, informational, false positive, needs manual investigation.
5. Write at least five invariants.
6. Implement one fuzz test and one stateful invariant from those properties.
7. Create a malicious mock dependency to test an external-call assumption.
8. Write one Medium-or-higher hypothetical finding only if the evidence supports it; otherwise write Low/Informational findings honestly.
9. Produce an executive summary that distinguishes discovered issues from residual risks.

Expected evidence

- Audit artifacts identify the exact code version reviewed.
- Every reported finding has a reproducible test or precise reasoning path.
- The privilege table matches actual code.
- Tool output is interpreted rather than copied verbatim.

Troubleshooting guide

If you see a problem: If Slither fails because of compiler/dependency resolution, make the project build cleanly first; analysis tools depend on reproducible compilation.

If you see a problem: If an invariant fails unexpectedly, minimize the sequence and inspect handler assumptions before changing the contract.

If you see a problem: If a finding relies on a privileged malicious admin, severity may be lower if that privilege is explicitly trusted - but centralization risk should still be documented.

Lab rule: Do not “fix” a failing lab by removing validation or weakening the expected assertion. First identify whether the failure is environment, configuration, integration state or actual code behavior.

Security, Testing, Debugging and Production Readiness

Security review points

- Audit deployment and initialization, not only implementation contracts.
- Check signature domains, nonces and replay boundaries whenever offchain signatures authorize actions.
- Review role-admin relationships and ownership-transfer paths.
- Inspect integer rounding direction where shares, fees, interest or prices are involved.
- Model malicious or nonstandard ERC-20 behavior for DeFi integrations.
- Record out-of-scope components so readers do not assume they were reviewed.

Required test plan

- Slither clean/triaged report.
- Five written invariants.
- One fuzz property and one stateful invariant.
- One malicious dependency test.
- Professional finding template completed.

Debugging playbook

1. Reproduce deterministically: pin block, seed, caller, input and contract version.
2. Read the revert/call trace before changing gas limits or code.
3. Check chain ID, deployed address, ABI and dependency version together.
4. Inspect state immediately before the failing call.
5. Reduce the scenario to the smallest failing sequence.
6. Convert the discovered failure into a regression test.
7. Only then refactor or optimize.

Production gate: A feature is not complete when it compiles. It is complete when its permissions, failure modes, tests, deployment configuration, monitoring signals and rollback/emergency assumptions are understood.

DAY 08 | SMART CONTRACT SECURITY II: AUDITING, FUZZING AND INVARIANTS

Senior Interview Practice, Homework and Portfolio Output

Interview questions - answer aloud first

Q: What is the first step in an audit?

A: Understand scope and intended architecture before judging implementation correctness.

Q: Why is static analysis insufficient?

A: It detects known patterns but cannot fully understand protocol-specific economic logic and trust assumptions.

Q: What makes a good proof of concept?

A: It is minimal, reproducible, demonstrates impact and avoids unnecessary destructive complexity.

Q: How do you choose severity?

A: Evaluate concrete impact, required privilege, exploitability, user interaction, timing and recoverability under the protocol assumptions.

Q: What is a ghost variable?

A: Test-only accounting used to track expected properties that are difficult to derive directly from contract state.

Q: What does a clean audit mean?

A: No material issues were identified within the stated scope and effort; it is not a guarantee of bug-free code.

Homework

- Create AUDIT_REPORT.md with executive summary, scope, methodology, findings and residual risks.
- Write a second independent review of another student/open-source toy contract without copying scanner output.
- Prepare a 5-minute mock client explanation of one finding and its remediation.

Portfolio deliverable

Ship this: Publish a sanitized audit portfolio sample showing methodology, a real local proof of concept, regression test and clear severity reasoning.

Official references for this day

<https://docs.soliditylang.org/en/latest/security-considerations.html>

<https://github.com/crytic/slither>

<https://www.getfoundry.sh/guides/invariant-testing>

Day 09 - DeFi Fundamentals, AMMs and Uniswap-Style Market Design

60-minute goal: Understand automated market maker mathematics, liquidity-provider economics, slippage, price impact, routing and the architecture behind modern DEX integrations.

Learning outcomes

- Explain and apply: amms replace order books with deterministic pricing rules.
- Explain and apply: spot price is not execution price.
- Explain and apply: fees compensate liquidity and alter the invariant path.
- Explain and apply: liquidity providers hold path-dependent exposure.
- Complete the guided lab and capture reproducible evidence.
- Answer the senior interview questions using architecture/security trade-offs, not memorized definitions.

Minute-by-minute lecture plan

Time	Teaching focus
0-8 min	DeFi primitives and composability
8-20 min	Constant-product AMM mathematics $x*y=k$
20-30 min	Spot price, slippage, price impact and fees
30-40 min	Liquidity-provider shares and impermanent loss intuition
40-50 min	Uniswap evolution: v2 pairs, v3 concentrated liquidity, v4 pool-manager/hooks concepts
50-60 min	Lab: quote and simulate a swap on a local/fork environment

Key terms for today

Term	Term	Term
AMM	liquidity pool	constant product
reserve	spot price	slippage
price impact	LP	impermanent loss
concentrated liquidity	tick	router
hook		

Instructor opening script

Today's topic is defi fundamentals, amms and uniswap-style market design. The senior-level objective is not to memorize API names; it is to understand the underlying trust model, state transitions and failure modes well enough to make architecture decisions and debug unexpected behavior. During the lecture, repeatedly ask: what state is authoritative, who can change it, which external dependencies are trusted, and what property must remain true after every successful transaction?

Deep Theory and Mental Models

AMMs replace order books with deterministic pricing rules

A constant-product pool holds two reserves x and y and maintains an invariant approximately $x*y=k$ after accounting for fees. Trades move the reserve ratio, so the marginal price changes with trade size. This makes liquidity depth and price impact central to execution quality.

Spot price is not execution price

For a simple pool, reserve ratio gives a local spot-price intuition, but a finite trade walks along the AMM curve and receives an average execution price. Client code must use quotes, slippage bounds and deadlines rather than assuming the displayed spot price is guaranteed.

Senior lens: Translate this concept into a review question: identify the authoritative state, caller/dependency assumptions, and the invariant that would fail if the concept were implemented incorrectly.

Fees compensate liquidity and alter the invariant path

Trading fees are deducted according to pool rules and accrue to liquidity providers or protocol mechanisms. Different fee tiers are suited to different volatility and correlation profiles. A senior integration treats fee configuration as part of the market, not a constant hidden in code.

Liquidity providers hold path-dependent exposure

LP value depends on relative asset prices and fee income. "Impermanent loss" describes underperformance relative to simply holding the assets under particular price movements, before considering fees and other incentives. It is not an onchain accounting error.

Senior lens: Translate this concept into a review question: identify the authoritative state, caller/dependency assumptions, and the invariant that would fail if the concept were implemented incorrectly.

Concentrated liquidity changes capital efficiency

Uniswap v3 allows liquidity to be allocated to price ranges rather than the entire curve. This increases capital efficiency near active prices but introduces range management, tick math and position-specific fee exposure. Integrators should not treat a v3 position like a fungible v2 LP token.

Modern DEX architectures separate routing from pool logic

Routers choose paths and encode user protections; pools implement pricing/accounting. Uniswap v4-style designs introduce a singleton PoolManager and hooks that allow custom logic around pool actions. Hooks increase expressiveness while also expanding the integration and security surface.

Architecture, Data Flow and Trade-offs

Reference architecture / execution flow

Constant product example
 Reserve X = 100 ETH
 Reserve Y = 300,000 USDC
 $k = 30,000,000$

Small spot-price intuition: $Y/X = 3,000$ USDC per ETH

A trader buying ETH removes ETH from X and adds USDC to Y.
 The reserve ratio moves, so later units of ETH cost more than earlier units.
 That difference is price impact, before considering external market movement.

Trade-off analysis

1. Deeper liquidity reduces price impact for the same trade size but requires more capital.
2. Tighter slippage protects users from bad execution but increases revert probability in volatile markets.
3. Multi-hop routing may improve price but increases gas and dependency surface.
4. Concentrated liquidity improves capital efficiency but requires active range-risk management.
5. Hook-based extensibility can implement valuable features but should be treated as third-party execution logic unless thoroughly reviewed.

Failure-mode questions to ask in a design review

- What happens if the external dependency is unavailable, stale or malicious?
- Can one compromised key change user balances, upgrade logic or move treasury assets?
- Does the system fail closed, fail open, or enter an ambiguous partially completed state?
- Which data is authoritative: contract state, emitted events, an indexer, an RPC response or an offchain database?
- How would you detect this component behaving incorrectly in production?
- Can the operation be replayed, reordered, reentered or executed on the wrong chain/context?
- What is the smallest privilege or interface that satisfies the requirement?

Architecture habit: For client work, draw the flow before writing code. A five-minute diagram often reveals missing authorization, wrong trust assumptions or unavailable recovery paths earlier than a compiler can.

Code Walkthrough and Implementation Notes

Solidity: simplified constant-product quote (educational, not production router math)

```
pragma solidity ^0.8.36;

library ConstantProductMath {
    // feeBps = 30 means 0.30%
    function quoteOut(
        uint256 reserveIn,
        uint256 reserveOut,
        uint256 amountIn,
        uint256 feeBps
    ) internal pure returns (uint256 amountOut) {
        require(reserveIn > 0 && reserveOut > 0, "empty pool");
        require(feeBps < 10_000, "fee");

        uint256 amountInAfterFee = amountIn * (10_000 - feeBps);
        uint256 numerator = amountInAfterFee * reserveOut;
        uint256 denominator = reserveIn * 10_000 + amountInAfterFee;
        amountOut = numerator / denominator;
    }
}
```

Walkthrough

The formula includes a basis-point fee adjustment before applying the constant-product relationship.

Integer division rounds down, which matters at small amounts and is part of the security/economic design.

A production integration should use the protocol's official quote interfaces/libraries rather than reimplementing complex pool math casually.

Quotes are state-dependent. Reserves, ticks and routing conditions can change before transaction inclusion.

Senior implementation checklist

- Pin the compiler/framework/library versions used by the repository.
- Keep configuration such as chain IDs and dependency addresses outside business logic unless immutability is deliberate.
- Use typed interfaces instead of arbitrary calls whenever practical.
- Emit events for operationally important state changes.
- Write the failure-path test at the same time as the success-path implementation.
- Document what this example intentionally omits before anyone reuses it.

Guided Lab: Build, Observe and Break It

Lab steps

1. Implement the educational quote function and unit test it with known reserves.
2. Calculate output for three trade sizes and show how effective price worsens as size increases.
3. Calculate a minimum amount out using a 0.5% user slippage tolerance.
4. Fork an EVM chain with a well-known DEX deployment and read pool/token metadata without sending real trades.
5. Compare a direct pair route with a two-hop route conceptually: price versus gas/dependency trade-off.
6. Inspect an actual swap receipt and identify pool/router events.
7. Write a diagram separating UI quote, router, pool and token transfers.
8. Add a test where reserveOut is small and explain why execution price becomes poor.

Expected evidence

- A table showing amountIn, quoted amountOut, effective price and estimated price impact for several sizes.
- A correct minimum-output calculation.
- A README statement that the simplified formula is educational and not a substitute for protocol-specific production libraries.

Troubleshooting guide

If you see a problem: If token decimals differ, normalize human-readable calculations carefully; onchain math remains integer smallest units.

If you see a problem: If a fork quote differs from a web UI, routing, block state, fee tier or concentrated-liquidity state may differ.

If you see a problem: Do not use reserve-ratio spot price as a guaranteed executable price.

Lab rule: Do not “fix” a failing lab by removing validation or weakening the expected assertion. First identify whether the failure is environment, configuration, integration state or actual code behavior.

Security, Testing, Debugging and Production Readiness

Security review points

- Always enforce user-defined or policy-defined minimum output and deadlines where the router supports them.
- Fee-on-transfer or rebasing tokens can break assumptions in routers that expect exact transfer amounts.
- Oracle consumers should not use a manipulable single-pool spot price without robust safeguards.
- Hooks/callbacks can introduce external-call and reentrancy edges.
- Approval to routers is an authorization decision; limit allowances when appropriate and verify spender addresses.

Required test plan

- Known-value AMM math unit tests.
- Fuzz amountIn within reserve-aware bounds and assert amountOut < reserveOut.
- Monotonicity property: larger positive input should not produce lower output under unchanged reserves.
- Rounding-boundary tests for tiny amounts.
- Fork quote comparison with protocol interface.

Debugging playbook

1. Reproduce deterministically: pin block, seed, caller, input and contract version.
2. Read the revert/call trace before changing gas limits or code.
3. Check chain ID, deployed address, ABI and dependency version together.
4. Inspect state immediately before the failing call.
5. Reduce the scenario to the smallest failing sequence.
6. Convert the discovered failure into a regression test.
7. Only then refactor or optimize.

Production gate: A feature is not complete when it compiles. It is complete when its permissions, failure modes, tests, deployment configuration, monitoring signals and rollback/emergency assumptions are understood.

Senior Interview Practice, Homework and Portfolio Output

Interview questions - answer aloud first

Q: What does $x*y=k$ represent?

A: A simplified invariant for constant-product AMMs relating pool reserves before and after trades, subject to fee mechanics.

Q: Why is spot price different from execution price?

A: A finite trade changes reserves as it executes, so later units trade at a different marginal price.

Q: What is slippage tolerance?

A: The user's allowed deviation between expected and minimum acceptable execution, commonly enforced by amountOutMin.

Q: Why is concentrated liquidity different?

A: Liquidity is active only within chosen price ranges, changing capital efficiency, risk and position representation.

Q: Can a DEX spot price be a safe oracle?

A: Not automatically; it may be manipulable within a transaction or short time window depending on liquidity and design.

Q: What is a router responsible for?

A: Path selection/call coordination, token transfers/approvals and user execution protections around pool interactions.

Homework

- Create a spreadsheet or Python notebook comparing constant-product outputs for 1%, 5%, 10% and 25% of pool depth.
- Explain impermanent loss without using the phrase "LPs always lose."
- Research the architectural difference among Uniswap v2, v3 and v4 at a conceptual level.

Portfolio deliverable

Ship this: Publish "AMM Math and DEX Integration Lab" with quote code, tests, price-impact table and architecture diagram.

Official references for this day

<https://docs.uniswap.org/>

<https://docs.uniswap.org/concepts/protocol/concentrated-liquidity>

<https://ethereum.org/en/defi/>

Day 10 - Lending, Aave-Style Flash Loans, Arbitrage and MEV Risk

60-minute goal: Understand collateralized lending, liquidation, health factors, flash-loan execution and the engineering reality of arbitrage after fees, slippage, gas and transaction-ordering risk.

Learning outcomes

- Explain and apply: lending protocols transform collateral into borrowing capacity.
- Explain and apply: health is an oracle-dependent risk metric.
- Explain and apply: liquidation is an incentive mechanism.
- Explain and apply: flash loans make temporary capital an atomic primitive.
- Complete the guided lab and capture reproducible evidence.
- Answer the senior interview questions using architecture/security trade-offs, not memorized definitions.

Minute-by-minute lecture plan

Time	Teaching focus
0-8 min	Lending markets: supply, borrow and utilization
8-20 min	Collateral, LTV, liquidation thresholds and health
20-30 min	Interest accrual and liquidation incentives
30-40 min	Flash loans and atomic repayment
40-50 min	Arbitrage profitability model and MEV competition
50-60 min	Lab: fork-based flash-loan simulation with no real funds

Key terms for today

Term	Term	Term
collateral	LTV	liquidation threshold
health factor	utilization	interest rate
liquidation	flash loan	atomicity
arbitrage	MEV	sandwich risk
profit threshold		

Instructor opening script

Today's topic is lending, aave-style flash loans, arbitrage and mev risk. The senior-level objective is not to memorize API names; it is to understand the underlying trust model, state transitions and failure modes well enough to make architecture decisions and debug unexpected behavior. During the lecture, repeatedly ask: what state is authoritative, who can change it, which external dependencies are trusted, and what property must remain true after every successful transaction?

Deep Theory and Mental Models

Lending protocols transform collateral into borrowing capacity

Users supply assets, designate collateral according to market rules, and borrow other assets up to risk limits. The critical quantities are not just balances but value-adjusted collateral, debt, price feeds, LTV limits and liquidation thresholds.

Health is an oracle-dependent risk metric

A borrower can be safe at one price and liquidatable after a price move. Health-factor formulas differ by protocol, but the engineering pattern is the same: collateral values and risk parameters are compared with debt. Oracle freshness, decimals and asset configuration are therefore security-critical.

Senior lens: Translate this concept into a review question: identify the authoritative state, caller/dependency assumptions, and the invariant that would fail if the concept were implemented incorrectly.

Liquidation is an incentive mechanism

When a position crosses protocol thresholds, third parties may repay debt and receive collateral plus an incentive according to rules. Liquidators compete on execution and must account for gas, liquidity and slippage. The protocol depends on liquidations remaining economically viable during stress.

Flash loans make temporary capital an atomic primitive

A flash loan lends assets with the requirement that principal plus required fee or debt transition conditions are satisfied within the same transaction. If repayment conditions fail, the transaction reverts atomically. Aave V3 documents flashLoan and flashLoanSimple-style flows for different use cases.

Senior lens: Translate this concept into a review question: identify the authoritative state, caller/dependency assumptions, and the invariant that would fail if the concept were implemented incorrectly.

Arbitrage is an execution problem, not just a price-difference problem

A price gap is only the beginning. Real profit must subtract swap fees, flash-loan premium, gas, slippage, price impact, token taxes, failed-transaction cost and competition. By the time a public transaction is included, another searcher may have removed the opportunity.

MEV changes ordering assumptions

Searchers and builders can optimize transaction order around liquidations and arbitrage. Senior engineers model whether a user transaction leaks profitable intent, whether minimum outputs protect users, and whether a strategy depends on being first in a highly competitive environment.

Architecture, Data Flow and Trade-offs

Reference architecture / execution flow

```

Atomic flash-loan flow
Aave Pool
| lends asset
v
Borrower callback
|--> swap / refinance / liquidation / arbitrage step 1
|--> optional step 2...
|--> verify final balance and profitability
|--> approve repayment
v
Pool pulls principal + premium
|
If any requirement fails -> entire transaction reverts

```

Trade-off analysis

1. Flash loans reduce capital requirements but do not remove execution risk or fees.
2. Fork simulation provides realistic interfaces but not guaranteed future profitability.
3. Higher slippage tolerance reduces revert risk while exposing execution to worse prices.
4. Private transaction channels can reduce public mempool competition but add infrastructure/trust dependencies.
5. Complex multi-DEX paths can improve theoretical quotes while increasing gas and failure surface.

Failure-mode questions to ask in a design review

- What happens if the external dependency is unavailable, stale or malicious?
- Can one compromised key change user balances, upgrade logic or move treasury assets?
- Does the system fail closed, fail open, or enter an ambiguous partially completed state?
- Which data is authoritative: contract state, emitted events, an indexer, an RPC response or an offchain database?
- How would you detect this component behaving incorrectly in production?
- Can the operation be replayed, reordered, reentered or executed on the wrong chain/context?
- What is the smallest privilege or interface that satisfies the requirement?

Architecture habit: For client work, draw the flow before writing code. A five-minute diagram often reveals missing authorization, wrong trust assumptions or unavailable recovery paths earlier than a compiler can.

Code Walkthrough and Implementation Notes

Solidity: flash-loan receiver skeleton with explicit profitability guard

```
// EDUCATIONAL SKELETON - adapt only from official protocol interfaces
pragma solidity ^0.8.36;

contract ArbitrageReceiver {
    address public immutable owner;
    error NotProfitable();
    error NotOwner();

    constructor() { owner = msg.sender; }

    function executeStrategy(uint256 borrowed, uint256 premium) internal {
        // 1) perform protocol-approved integrations
        // 2) calculate final asset balance
        // 3) enforce min-profit before repayment approval
        uint256 finalBalance = 0; // replace in a local/fork exercise
        uint256 required = borrowed + premium;
        if (finalBalance <= required) revert NotProfitable();
    }

    function withdraw(address token) external {
        if (msg.sender != owner) revert NotOwner();
        // treasury withdrawal logic
    }
}
```

Walkthrough

The important pattern is to make profitability a contract-level condition rather than relying only on an offchain quote.

Production receivers must validate the lending-pool caller, initiator semantics, token approvals, callback data and protocol-specific repayment rules.

A strategy should use exact, official router/pool interfaces and explicit minimum outputs.

This course intentionally keeps arbitrage execution on a local/fork environment; profitability examples are simulations, not promises of returns.

Senior implementation checklist

- Pin the compiler/framework/library versions used by the repository.
- Keep configuration such as chain IDs and dependency addresses outside business logic unless immutability is deliberate.
- Use typed interfaces instead of arbitrary calls whenever practical.
- Emit events for operationally important state changes.
- Write the failure-path test at the same time as the success-path implementation.
- Document what this example intentionally omits before anyone reuses it.

Guided Lab: Build, Observe and Break It

Lab steps

1. Choose a fork block and record it in your test.
2. Read the Aave Pool address and flash-loan interface from official deployment documentation for the selected network.
3. Create a receiver that can only be initiated by an authorized owner in your local test.
4. Borrow a small test amount on the fork and immediately repay plus premium without trading; prove the callback lifecycle.
5. Add two mocked DEXs with intentionally different prices and simulate an atomic arbitrage.
6. Calculate gross spread, swap fees, flash premium and gas-cost estimate in one report table.
7. Add amountOutMin protection to each mocked swap.
8. Make the trade revert when expected net profit is below a configured threshold.
9. Write a scenario where a 1% apparent price gap becomes unprofitable after costs.

Expected evidence

- A deterministic fork or mock test that demonstrates flash-loan atomicity.
- A profitability equation that includes all material modeled costs.
- The contract reverts on non-profitable simulated routes.
- No real-fund deployment instructions are required to complete the learning objective.

Troubleshooting guide

If you see a problem: If the callback is rejected, verify the exact Aave V3 receiver interface and caller validation for the selected market.

If you see a problem: If repayment fails, check token balance, allowance and premium calculation.

If you see a problem: If a route works on one block but fails on another, pool reserves and market state changed; that is expected.

If you see a problem: If gas estimate dominates profit, the correct result is “do not execute,” not “raise leverage.”

Lab rule: Do not “fix” a failing lab by removing validation or weakening the expected assertion. First identify whether the failure is environment, configuration, integration state or actual code behavior.

Security, Testing, Debugging and Production Readiness

Security review points

- Validate lending-pool caller and callback initiator.
- Use strict token/DEX allowlists and never execute arbitrary user-supplied routers in a privileged receiver.
- Set slippage and minimum-profit checks onchain.
- Model fee-on-transfer/nonstandard token behavior before supporting an asset.
- Protect treasury withdrawal and administrative controls behind strong operational security.
- Do not represent simulated arbitrage as guaranteed income.

Required test plan

- Fork test for borrow + repay.
- Failure test when repayment approval is missing.
- Profitability threshold test.
- Fuzz swap outputs around break-even.
- Authorization tests for strategy initiation and withdrawals.
- Invariant: after every successful operation, lender is repayable and contract accounting is solvent.

Debugging playbook

1. Reproduce deterministically: pin block, seed, caller, input and contract version.
2. Read the revert/call trace before changing gas limits or code.
3. Check chain ID, deployed address, ABI and dependency version together.
4. Inspect state immediately before the failing call.
5. Reduce the scenario to the smallest failing sequence.
6. Convert the discovered failure into a regression test.
7. Only then refactor or optimize.

Production gate: A feature is not complete when it compiles. It is complete when its permissions, failure modes, tests, deployment configuration, monitoring signals and rollback/emergency assumptions are understood.

Senior Interview Practice, Homework and Portfolio Output

Interview questions - answer aloud first

Q: Why can a flash loan exist without collateral?

A: Because repayment is enforced atomically in the same transaction; otherwise the whole state transition reverts.

Q: What makes an arbitrage opportunity executable?

A: Net profit remains positive after fees, slippage, price impact, gas and ordering/competition risk, and the transaction satisfies all protocol constraints.

Q: Why use an onchain minimum-profit guard?

A: Offchain quotes can become stale before inclusion; the contract should refuse execution below an acceptable final-state threshold.

Q: What is liquidation risk based on?

A: Collateral/debt values, protocol risk parameters and oracle prices rather than token counts alone.

Q: How does MEV affect arbitrage?

A: Competitors can observe or infer opportunities and compete for ordering, often eliminating naïve public strategies.

Q: Why pin a fork block for a tutorial?

A: It makes the liquidity and protocol state reproducible for learning and testing.

Homework

- Build a profit calculator that accepts quoted outputs, gas units, gas price and protocol fees and returns break-even spread.
- Write a 1,000-word explanation of why “flash loan = free money” is false.
- Create a risk checklist for a production DeFi integration: caller validation, slippage, oracle, allowances, token behavior, pause and monitoring.

Portfolio deliverable

Ship this: Publish a “Flash Loan and Arbitrage Simulator” using mocks/fork tests, with explicit break-even analysis and security controls rather than profit claims.

Official references for this day

<https://aave.com/docs/aave-v3/guides/flash-loans>

<https://ethereum.org/en/developers/docs/mev/>

<https://www.getfoundry.sh/guides/fork-testing>

Day 11 - Oracles, Price Feeds and External Data Integrity

60-minute goal: Design contracts that consume external data safely, with explicit handling for decimals, staleness, sequencer health, fallback behavior and manipulation resistance.

Learning outcomes

- Explain and apply: oracles bridge two trust domains.
- Explain and apply: price feeds are structured data products, not magic numbers.
- Explain and apply: staleness is application-specific.
- Explain and apply: manipulable spot markets are weak single-source oracles.
- Complete the guided lab and capture reproducible evidence.
- Answer the senior interview questions using architecture/security trade-offs, not memorized definitions.

Minute-by-minute lecture plan

Time	Teaching focus
0-8 min	The oracle problem and why blockchains cannot natively know external prices
8-18 min	Data-feed architecture and aggregation
18-30 min	Decimals, round data, staleness and heartbeat assumptions
30-40 min	Oracle manipulation and safe price-consumer patterns
40-50 min	L2 sequencer-health considerations and fallbacks
50-60 min	Lab: build a robust price consumer with validation

Key terms for today

Term	Term	Term
oracle	data feed	aggregator
round	heartbeat	staleness
decimals	TWAP	manipulation
sequencer uptime	fallback	circuit breaker

Instructor opening script

Today's topic is oracles, price feeds and external data integrity. The senior-level objective is not to memorize API names; it is to understand the underlying trust model, state transitions and failure modes well enough to make architecture decisions and debug unexpected behavior. During the lecture, repeatedly ask: what state is authoritative, who can change it, which external dependencies are trusted, and what property must remain true after every successful transaction?

Deep Theory and Mental Models

Oracles bridge two trust domains

A smart contract can verify onchain state deterministically but cannot independently observe an exchange rate, weather outcome or bank balance. An oracle system introduces a mechanism and trust model for bringing external facts onchain. Senior engineers document that trust model as part of protocol security.

Price feeds are structured data products, not magic numbers

Consumers receive values with decimals, timestamps and round metadata. Contracts must normalize units correctly and decide what data age is acceptable for their risk model. Reading `latestRoundData` without checking freshness or sign can turn a valid feed into an unsafe integration.

Senior lens: Translate this concept into a review question: identify the authoritative state, caller/dependency assumptions, and the invariant that would fail if the concept were implemented incorrectly.

Staleness is application-specific

A feed that is acceptable for a slow vesting calculation may be unacceptable for leveraged lending. The contract should enforce a maximum age consistent with how quickly losses can accumulate, while operations teams monitor feed availability and market conditions.

Manipulable spot markets are weak single-source oracles

A thin DEX pool can often be moved temporarily, especially within atomic transactions. Time-weighted prices, decentralized data feeds, liquidity checks and protocol-specific defenses help, but every oracle choice has latency and trust trade-offs.

Senior lens: Translate this concept into a review question: identify the authoritative state, caller/dependency assumptions, and the invariant that would fail if the concept were implemented incorrectly.

L2 dependencies add another failure mode

On some L2 systems, a sequencer outage changes when users can submit transactions and how stale state may appear. Chainlink documents sequencer-uptime feeds for supported environments. Applications should define grace periods and avoid liquidating users based on unreliable transitional conditions.

Circuit breakers turn data assumptions into explicit policy

A robust protocol can pause, limit actions or switch to a conservative mode when prices are stale, deviate beyond bounds or dependencies fail. Fallbacks should not silently substitute a weaker price source without governance approval and clear risk analysis.

Architecture, Data Flow and Trade-offs

Reference architecture / execution flow

```

External markets / data sources
    |
Independent reporters / aggregation
    v
Onchain feed contract
  answer + decimals + updatedAt
    |
Consumer validation layer
  +-- answer > 0
  +-- age <= maxAge
  +-- expected feed address
  +-- optional sequencer check
    v
Protocol risk logic
  
```

Trade-off analysis

1. Faster data improves responsiveness but may cost more or increase dependency complexity.
2. A conservative maxAge reduces stale-price risk but can halt operations during feed disruption.
3. Fallback price sources improve availability while creating consistency and manipulation questions.
4. TWAPs reduce one-block manipulation but introduce lag during genuine rapid price moves.
5. Manual emergency override can save a protocol but becomes a powerful governance trust assumption.

Failure-mode questions to ask in a design review

- What happens if the external dependency is unavailable, stale or malicious?
- Can one compromised key change user balances, upgrade logic or move treasury assets?
- Does the system fail closed, fail open, or enter an ambiguous partially completed state?
- Which data is authoritative: contract state, emitted events, an indexer, an RPC response or an offchain database?
- How would you detect this component behaving incorrectly in production?
- Can the operation be replayed, reordered, reentered or executed on the wrong chain/context?
- What is the smallest privilege or interface that satisfies the requirement?

Architecture habit: For client work, draw the flow before writing code. A five-minute diagram often reveals missing authorization, wrong trust assumptions or unavailable recovery paths earlier than a compiler can.

Code Walkthrough and Implementation Notes

Solidity: defensive Chainlink-style price consumer

```
// Interface simplified for teaching; use official interfaces in production.
pragma solidity ^0.8.36;

interface IAggregatorV3 {
    function decimals() external view returns (uint8);
    function latestRoundData() external view returns (
        uint80 roundId, int256 answer, uint256 startedAt,
        uint256 updatedAt, uint80 answeredInRound
    );
}

contract PriceConsumer {
    IAggregatorV3 public immutable feed;
    uint256 public immutable maxAge;

    error InvalidPrice();
    error StalePrice(uint256 age);

    constructor(address feed_, uint256 maxAge_) {
        feed = IAggregatorV3(feed_);
        maxAge = maxAge_;
    }

    function read() external view returns (uint256 price, uint8 decimals_) {
        (, int256 answer,, uint256 updatedAt,) = feed.latestRoundData();
        if (answer <= 0) revert InvalidPrice();
        uint256 age = block.timestamp - updatedAt;
        if (age > maxAge) revert StalePrice(age);
        return (uint256(answer), feed.decimals());
    }
}
```

Walkthrough

The consumer validates sign and freshness rather than trusting every returned value.

Production integrations should follow the current feed-specific documentation, including network addresses and any required sequencer checks.

maxAge is a protocol risk parameter and should be justified, monitored and potentially governable under safe procedures.

Never hard-code a feed from a blog or copied repository without verifying the official network address and denomination.

Senior implementation checklist

- Pin the compiler/framework/library versions used by the repository.
- Keep configuration such as chain IDs and dependency addresses outside business logic unless immutability is deliberate.
- Use typed interfaces instead of arbitrary calls whenever practical.
- Emit events for operationally important state changes.
- Write the failure-path test at the same time as the success-path implementation.
- Document what this example intentionally omits before anyone reuses it.

Guided Lab: Build, Observe and Break It

Lab steps

1. Deploy a MockAggregator and test normal, negative, zero and stale values.
2. Add a normalizeTo18 function that converts feed decimals to 18-decimal fixed-point units with explicit overflow reasoning.
3. Add minPrice and maxPrice circuit-breaker bounds for the lab.
4. Fork a network and read one official price feed using a pinned block.
5. Record feed decimals and updatedAt.
6. Simulate stale data in a mock and prove the protocol action reverts.
7. Design an L2 sequencer-outage decision tree on paper: what actions should pause, what can remain enabled, and when is a grace period needed?
8. Write a monitoring alert definition for stale or invalid prices.

Expected evidence

- All invalid-price scenarios fail closed.
- Decimal normalization is tested for feeds below and above 18 decimals if supported by your helper assumptions.
- Fork read uses an official address and documents the chosen network/block.
- Operational response is described, not left as “oracle will always work.”

Troubleshooting guide

If you see a problem: If `block.timestamp < updatedAt` in a malformed mock, subtraction reverts; tests should cover bad metadata assumptions.

If you see a problem: If a fork price seems off by powers of ten, inspect decimals before changing arithmetic.

If you see a problem: If the feed address has no code, verify network/chain ID and current official deployment documentation.

Lab rule: Do not “fix” a failing lab by removing validation or weakening the expected assertion. First identify whether the failure is environment, configuration, integration state or actual code behavior.

Security, Testing, Debugging and Production Readiness

Security review points

- Validate answer sign and data age.
- Use correct denomination: ETH/USD is not interchangeable with token/ETH.
- Do not use a manipulable single-block DEX spot price as a critical lending oracle without safeguards.
- Add sequencer-health logic on applicable L2s.
- Monitor feed freshness and deviation operationally.
- Treat emergency price overrides as privileged actions with strong governance and transparent events.

Required test plan

- Unit tests for positive, zero, negative and stale answers.
- Fuzz decimal-normalization values within safe ranges.
- Fork test against a real feed.
- Property: successful normalized price must be nonzero and within configured bounds.
- Failure test for sequencer-down/grace-period logic in a mock design.

Debugging playbook

1. Reproduce deterministically: pin block, seed, caller, input and contract version.
2. Read the revert/call trace before changing gas limits or code.
3. Check chain ID, deployed address, ABI and dependency version together.
4. Inspect state immediately before the failing call.
5. Reduce the scenario to the smallest failing sequence.
6. Convert the discovered failure into a regression test.
7. Only then refactor or optimize.

Production gate: A feature is not complete when it compiles. It is complete when its permissions, failure modes, tests, deployment configuration, monitoring signals and rollback/emergency assumptions are understood.

DAY 11 | ORACLES, PRICE FEEDS AND EXTERNAL DATA INTEGRITY

Senior Interview Practice, Homework and Portfolio Output

Interview questions - answer aloud first

Q: Why does a blockchain need an oracle?

A: Onchain consensus can verify onchain data but cannot inherently observe external facts such as market prices.

Q: What is staleness?

A: The feed value may be valid historically but too old for the application's current risk assumptions.

Q: Why do decimals matter?

A: Price feeds and tokens use integer values with scaling; a decimal mismatch can create orders-of-magnitude accounting errors.

Q: Why are single-pool spot prices risky?

A: An attacker may temporarily manipulate a thin market, especially atomically, to influence protocol decisions.

Q: What is a circuit breaker?

A: A policy that limits or pauses behavior when data violates safety conditions such as age or deviation bounds.

Q: Why consider sequencer health on L2?

A: Outages can impair transaction submission and create unfair or unsafe liquidation/price-update conditions.

Homework

- Write an Oracle Integration Checklist for future client work.
- Add a stale-price monitor pseudocode script.
- Compare decentralized data feed, DEX TWAP and administrator-set fallback across trust, latency and manipulation dimensions.

Portfolio deliverable

Ship this: Publish "Oracle-Safe Price Consumer" with mocks, fork test, decimal normalization, staleness checks and a risk-parameter README.

Official references for this day

<https://docs.chain.link/data-feeds>

<https://docs.chain.link/data-feeds/price-feeds>

<https://docs.chain.link/data-feeds/l2-sequencer-feeds>

Day 12 - Upgradeable Contracts, Proxies and Storage Safety

60-minute goal: Understand proxy architecture, initialization, UUPS/transparent patterns, storage compatibility, upgrade authorization and operational governance for mutable smart-contract systems.

Learning outcomes

- Explain and apply: upgradeability moves trust from code immutability to governance.
- Explain and apply: delegatecall is the execution mechanism.
- Explain and apply: initializers replace constructor-state assumptions.
- Explain and apply: storage layout is a long-term compatibility contract.
- Complete the guided lab and capture reproducible evidence.
- Answer the senior interview questions using architecture/security trade-offs, not memorized definitions.

Minute-by-minute lecture plan

Time	Teaching focus
0-8 min	Why upgradeability exists and what immutability you give up
8-18 min	Proxy, implementation and delegatecall execution
18-30 min	Initializers versus constructors
30-40 min	Storage layout compatibility and namespaced storage concepts
40-50 min	Transparent versus UUPS operational models
50-60 min	Lab: deploy V1, preserve state, upgrade to V2 and test authorization

Key terms for today

Term	Term	Term
proxy	implementation	delegatecall
initializer	storage layout	UUPS
transparent proxy	ProxyAdmin	upgrade authority
rollback	storage gap	namespaced storage

Instructor opening script

Today's topic is upgradeable contracts, proxies and storage safety. The senior-level objective is not to memorize API names; it is to understand the underlying trust model, state transitions and failure modes well enough to make architecture decisions and debug unexpected behavior. During the lecture, repeatedly ask: what state is authoritative, who can change it, which external dependencies are trusted, and what property must remain true after every successful transaction?

Deep Theory and Mental Models

Upgradeability moves trust from code immutability to governance

A proxy lets users interact with a stable address while logic can change. This enables bug fixes and feature evolution, but the upgrade authority can also alter economic rules or steal assets if compromised. "Upgradeable" is therefore an explicit governance and operational trust choice.

Delegatecall is the execution mechanism

The proxy forwards calldata to implementation code using delegatecall. Code comes from the implementation, but address(this), balances and storage belong to the proxy. This explains why implementation constructors do not initialize proxy state and why storage compatibility is critical.

Senior lens: Translate this concept into a review question: identify the authoritative state, caller/dependency assumptions, and the invariant that would fail if the concept were implemented incorrectly.

Initializers replace constructor-state assumptions

Upgradeable implementations use initializer functions guarded against repeated execution. Deployment must initialize the proxy promptly and disable inappropriate initialization paths on the implementation. An uninitialized proxy or implementation can become an ownership takeover vector.

Storage layout is a long-term compatibility contract

Changing variable order, type width, inheritance layout or struct definitions can reinterpret existing storage. Safe upgrades append compatible fields or use documented storage-management patterns. Modern OpenZeppelin upgradeable systems include mechanisms intended to improve layout safety, but engineers still must understand what state is preserved.

Senior lens: Translate this concept into a review question: identify the authoritative state, caller/dependency assumptions, and the invariant that would fail if the concept were implemented incorrectly.

Transparent and UUPS place upgrade logic differently

Transparent proxies traditionally separate admin routing and implementation calls, often using ProxyAdmin. UUPS places upgrade authorization logic in the implementation while the proxy is a smaller ERC-1967-style shell. The operational choice affects upgrade keys, implementation requirements and failure modes.

An upgrade must be treated like a production migration

Review new logic, compare storage layout, run fork tests, simulate the exact governance transaction, verify implementation bytecode, execute through multisig/timelock, then validate post-upgrade state and critical functions. Rollback plans must be realistic rather than assumed.

Architecture, Data Flow and Trade-offs

Reference architecture / execution flow

```

User -> Proxy address
      |
      | delegatecall(calldata)
      v
Implementation V1 code
      |
      | +-- reads/writes Proxy storage
      |
Upgrade authority changes implementation pointer
      |
      v
Implementation V2 code
      |
      | +-- MUST interpret existing Proxy storage correctly
  
```

Trade-off analysis

1. Immutable contracts minimize governance trust but cannot patch logic without migration.
2. Upgradeable systems improve maintainability but create permanent privileged control and storage-migration obligations.
3. Timelocks provide users time to react to upgrades but slow emergency response.
4. UUPS can reduce proxy complexity but a broken upgrade implementation can lock future upgrades if designed incorrectly.
5. Multiple implementation versions require careful monitoring, verification and incident runbooks.

Failure-mode questions to ask in a design review

- What happens if the external dependency is unavailable, stale or malicious?
- Can one compromised key change user balances, upgrade logic or move treasury assets?
- Does the system fail closed, fail open, or enter an ambiguous partially completed state?
- Which data is authoritative: contract state, emitted events, an indexer, an RPC response or an offchain database?
- How would you detect this component behaving incorrectly in production?
- Can the operation be replayed, reordered, reentered or executed on the wrong chain/context?
- What is the smallest privilege or interface that satisfies the requirement?

Architecture habit: For client work, draw the flow before writing code. A five-minute diagram often reveals missing authorization, wrong trust assumptions or unavailable recovery paths earlier than a compiler can.

Code Walkthrough and Implementation Notes

Solidity: minimal V1/V2 state idea for an upgradeable counter

```
// Illustrative only - use official OpenZeppelin upgradeable contracts/plugins.
pragma solidity ^0.8.36;

contract CounterV1 {
    uint256 public count;
    address public admin;

    function initialize(address admin_) external {
        require(admin == address(0), "initialized");
        admin = admin_;
    }

    function increment() external { count += 1; }
}

contract CounterV2 is CounterV1 {
    // Appended state preserves V1 slots in this simplified inheritance example.
    uint256 public step;

    function setStep(uint256 step_) external {
        require(msg.sender == admin, "not admin");
        step = step_;
    }
}
```

Walkthrough

This manual initializer is intentionally simplistic; production code should use OpenZeppelin Initializable and upgrade plugins.

The central learning point is that V2 keeps V1 variables in compatible positions and appends new state.

Adding a new parent contract with state or reordering V1 fields can change slot allocation and corrupt live data.

Upgrade authorization should be explicit and usually controlled by a multisig/timelock rather than a personal EOA.

Senior implementation checklist

- Pin the compiler/framework/library versions used by the repository.
- Keep configuration such as chain IDs and dependency addresses outside business logic unless immutability is deliberate.
- Use typed interfaces instead of arbitrary calls whenever practical.
- Emit events for operationally important state changes.
- Write the failure-path test at the same time as the success-path implementation.
- Document what this example intentionally omits before anyone reuses it.

Guided Lab: Build, Observe and Break It

Lab steps

1. Create a new project using the current OpenZeppelin Upgrades workflow for your chosen framework.
2. Deploy an upgradeable CounterV1 proxy locally with initializer parameters.
3. Record proxy address, implementation address and initial storage values.
4. Increment state and verify count.
5. Create CounterV2 with an added feature and compatible storage.
6. Run the framework's storage-layout validation before upgrading.
7. Upgrade locally and prove the proxy address and count remain unchanged.
8. Attempt an unauthorized upgrade and prove it fails.
9. Intentionally create an incompatible V2 branch and observe validation/tooling warnings.
10. Write an upgrade runbook with pre-checks, execution and post-checks.

Expected evidence

- State survives the compatible upgrade.
- The implementation address changes while users continue using the proxy address.
- Unauthorized upgrades fail.
- The intentionally incompatible layout is detected by tooling or by your manual analysis.

Troubleshooting guide

If you see a problem: If initialize cannot be called, check whether the proxy was already initialized rather than removing guards.

If you see a problem: If state becomes zero after upgrade, confirm you are reading the proxy address, not the implementation directly.

If you see a problem: If an upgrade tool rejects layout changes, do not bypass validation until you fully understand the storage implications.

Lab rule: Do not "fix" a failing lab by removing validation or weakening the expected assertion. First identify whether the failure is environment, configuration, integration state or actual code behavior.

Security, Testing, Debugging and Production Readiness

Security review points

- Disable implementation initializers where recommended by the framework.
- Protect upgrade authority with robust governance.
- Validate storage layout on every version.
- Never selfdestruct or introduce delegatecall patterns into upgradeable implementations without understanding framework restrictions and modern EVM semantics.
- Emit/monitor upgrade events and implementation bytecode verification.
- Separate emergency pause authority from upgrade authority where practical.

Required test plan

- Initializer can run exactly once.
- V1 state persists through V2.
- Unauthorized upgrade fails.
- Upgrade to incompatible layout is prevented.
- Fork rehearsal against a copy of production proxy state before real upgrades.

Debugging playbook

1. Reproduce deterministically: pin block, seed, caller, input and contract version.
2. Read the revert/call trace before changing gas limits or code.
3. Check chain ID, deployed address, ABI and dependency version together.
4. Inspect state immediately before the failing call.
5. Reduce the scenario to the smallest failing sequence.
6. Convert the discovered failure into a regression test.
7. Only then refactor or optimize.

Production gate: A feature is not complete when it compiles. It is complete when its permissions, failure modes, tests, deployment configuration, monitoring signals and rollback/emergency assumptions are understood.

DAY 12 | UPGRADEABLE CONTRACTS, PROXIES AND STORAGE SAFETY

Senior Interview Practice, Homework and Portfolio Output

Interview questions - answer aloud first

Q: Why do constructors not initialize proxy state?

A: The constructor runs on the implementation deployment, while user calls later execute implementation code via delegatecall against proxy storage.

Q: What is the biggest technical risk of upgrades?

A: Storage incompatibility and privileged upgrade logic can corrupt state or change behavior catastrophically.

Q: What is UUPS?

A: A proxy pattern where upgrade logic/authorization is implemented in the implementation contract while an ERC-1967-style proxy stores the implementation pointer.

Q: Why use a timelock?

A: It creates a delay between governance authorization and execution, giving monitoring systems/users time to review or react.

Q: Why read through the proxy after upgrading?

A: The proxy owns the persistent state and is the address users interact with.

Q: What should an upgrade runbook include?

A: Exact versions, storage validation, fork simulation, governance transaction, bytecode verification, post-state checks and incident/rollback procedures.

Homework

- Build V1->V2 upgrade tests and retain the intentionally incompatible branch for demonstration.
- Create a storage-layout table listing slot intent for your simplified contract.
- Write a client-facing paragraph explaining why “upgradeable” is a governance trade-off, not automatically a best practice.

Portfolio deliverable

Ship this: Publish “Upgradeable Contract Lab” with proxy architecture, V1/V2 tests, storage validation evidence and an upgrade runbook.

Official references for this day

<https://docs.openzeppelin.com/upgrades-plugins/>

<https://docs.openzeppelin.com/upgrades-plugins/writing-upgradeable>

<https://docs.openzeppelin.com/contracts/5.x/api/proxy>

Day 13 - Web3 Backend Engineering, Signing and Transaction Reliability

60-minute goal: Build reliable backend services that read chain state, construct transactions, manage nonces, sign securely, monitor receipts and expose safe APIs without leaking private keys.

Learning outcomes

- Explain and apply: read paths and write paths have different risk.
- Explain and apply: rpc reliability needs explicit policy.
- Explain and apply: abi versioning is application versioning.
- Explain and apply: nonce management becomes a distributed-systems problem.
- Complete the guided lab and capture reproducible evidence.
- Answer the senior interview questions using architecture/security trade-offs, not memorized definitions.

Minute-by-minute lecture plan

Time	Teaching focus
0-8 min	Backend roles: read service, transaction service, indexer and signer
8-18 min	RPC providers, retries, block tags and chain validation
18-30 min	ethers/viem/web3.py contract calls and ABI management
30-40 min	Nonce management, fee policy and transaction replacement
40-50 min	Key management, HSM/KMS/multisig separation and secret hygiene
50-60 min	Lab: REST API for reads + controlled transaction submission

Key terms for today

Term	Term	Term
RPC	ABI	provider
signer	nonce	pending nonce
replacement transaction	receipt	confirmation policy
reorg	KMS	HSM
idempotency	rate limit	

Instructor opening script

Today's topic is web3 backend engineering, signing and transaction reliability. The senior-level objective is not to memorize API names; it is to understand the underlying trust model, state transitions and failure modes well enough to make architecture decisions and debug unexpected behavior. During the lecture, repeatedly ask: what state is authoritative, who can change it, which external dependencies are trusted, and what property must remain true after every successful transaction?

Deep Theory and Mental Models

Read paths and write paths have different risk

Reading chain state can often be horizontally scaled and cached. Writing requires authorization, nonce coordination, gas policy and secure signing. Combining both behind one unrestricted service increases blast radius. Senior architectures separate concerns and use least privilege.

RPC reliability needs explicit policy

Providers can rate-limit, time out, return stale heads or disagree briefly during reorgs. A production service defines retryable errors, timeouts, provider failover, maximum tolerated lag and chain-ID verification. Blindly retrying `sendRawTransaction` can create duplicate operational confusion even if Ethereum nonce semantics prevent duplicate state changes.

Senior lens: Translate this concept into a review question: identify the authoritative state, caller/dependency assumptions, and the invariant that would fail if the concept were implemented incorrectly.

ABI versioning is application versioning

A backend that uses the wrong ABI can encode calls incorrectly or misinterpret return data/events. Contract address, chain ID, implementation/proxy version and ABI should be deployed as a consistent configuration bundle with validation at startup.

Nonce management becomes a distributed-systems problem

Multiple workers sending from one EOA can race for the same nonce. Strategies include a single transaction coordinator, persistent nonce reservation, separate sender accounts or managed signing infrastructure. “Just call `getTransactionCount`” is insufficient at scale.

Senior lens: Translate this concept into a review question: identify the authoritative state, caller/dependency assumptions, and the invariant that would fail if the concept were implemented incorrectly.

Receipt handling needs a state machine

A submitted transaction can be pending, mined-success, mined-reverted, replaced, dropped from a provider view or later affected by a reorg before finality policy is reached. Backend APIs should expose these states instead of returning “success” immediately after broadcast.

Private keys should not live in ordinary application code

Use dedicated secret-management/signing systems, hardware-backed keys, multisigs or external custody depending on authority. A hot backend signer should have narrowly scoped permissions, bounded balances and monitoring. Never log raw signed payloads if doing so exposes sensitive workflow metadata unnecessarily.

Architecture, Data Flow and Trade-offs

Reference architecture / execution flow

```

Client
| HTTPS request
v
API/auth/rate limit
|
+--> Read Service --> RPC pool --> chain
|
+--> Transaction Coordinator
      | validate request + policy
      | reserve nonce
      v
      Signer / KMS
      | signed raw tx
      v
      RPC broadcast
      |
      Receipt monitor -> DB status -> client/webhook
  
```

Trade-off analysis

1. A single sender account simplifies authorization but creates nonce contention and key concentration.
2. Multiple RPC providers improve availability but can disagree at the chain tip and increase cost.
3. Waiting for more confirmations reduces reorg risk but increases user-visible latency.
4. Server-side signing enables automation but creates custodial/security obligations.
5. Client-side signing reduces server key custody but limits some automated workflows.

Failure-mode questions to ask in a design review

- What happens if the external dependency is unavailable, stale or malicious?
- Can one compromised key change user balances, upgrade logic or move treasury assets?
- Does the system fail closed, fail open, or enter an ambiguous partially completed state?
- Which data is authoritative: contract state, emitted events, an indexer, an RPC response or an offchain database?
- How would you detect this component behaving incorrectly in production?
- Can the operation be replayed, reordered, reentered or executed on the wrong chain/context?
- What is the smallest privilege or interface that satisfies the requirement?

Architecture habit: For client work, draw the flow before writing code. A five-minute diagram often reveals missing authorization, wrong trust assumptions or unavailable recovery paths earlier than a compiler can.

Code Walkthrough and Implementation Notes

TypeScript: robust read endpoint and transaction-status model (illustrative)

```
import { createPublicClient, http, parseAbi } from "viem";
import { mainnet } from "viem/chains";

const client = createPublicClient({
  chain: mainnet,
  transport: http(process.env.RPC_URL!),
});

const erc20 = parseAbi([
  "function balanceOf(address) view returns (uint256)",
  "function decimals() view returns (uint8)",
]);

export async function tokenBalance(token: `0x${string}`, user: `0x${string}`) {
  const [raw, decimals] = await Promise.all([
    client.readContract({ address: token, abi: erc20, functionName: "balanceOf", args: [user] }),
    client.readContract({ address: token, abi: erc20, functionName: "decimals" }),
  ]);
  return { raw: raw.toString(), decimals };
}

// Write services should be separated and use a managed signer/policy layer.
```

Walkthrough

The example returns raw integer balance plus decimals instead of converting to JavaScript Number and losing precision.

Addresses should be validated and normalized by the library.

Network/contract configuration should be environment-specific but validated at startup so a production service cannot silently point at the wrong chain.

Write endpoints should validate destination, method, value limits and business authorization before any signing request.

Senior implementation checklist

- Pin the compiler/framework/library versions used by the repository.
- Keep configuration such as chain IDs and dependency addresses outside business logic unless immutability is deliberate.
- Use typed interfaces instead of arbitrary calls whenever practical.
- Emit events for operationally important state changes.
- Write the failure-path test at the same time as the success-path implementation.
- Document what this example intentionally omits before anyone reuses it.

Guided Lab: Build, Observe and Break It

Lab steps

1. Create a small TypeScript backend with GET /chain, GET /balance/:address and GET /token/:token/balance/:address.
2. Return chain ID and latest block number from /chain.
3. Add request validation for addresses.
4. Create an internal transaction status schema: CREATED, SIGNED, BROADCAST, MINED_SUCCESS, MINED_REVERT, REPLACED, FINALIZED, FAILED.
5. Implement receipt polling for a test transaction on a local chain.
6. Simulate a replacement transaction with the same nonce and higher fee on Anvil.
7. Add idempotency key handling to prevent duplicate business requests.
8. Add a startup guard that checks expected chain ID.
9. Write a threat model for the signer endpoint.
10. Ensure no private key appears in source, logs or API responses.

Expected evidence

- API uses bigint-safe serialization.
- Wrong-chain startup fails fast.
- Receipt monitor distinguishes mined success from mined revert.
- Duplicate idempotency key returns the same business operation rather than creating another intent.

Troubleshooting guide

If you see a problem: JSON does not support bigint directly; serialize as decimal strings.

If you see a problem: If a transaction remains pending on a local chain, confirm automining settings.

If you see a problem: If nonce-too-low appears, another transaction with that nonce was already accepted/mined; inspect sender history before retrying.

If you see a problem: If replacement-underpriced appears, the new fee did not satisfy node/network replacement policy.

Lab rule: Do not "fix" a failing lab by removing validation or weakening the expected assertion. First identify whether the failure is environment, configuration, integration state or actual code behavior.

Security, Testing, Debugging and Production Readiness

Security review points

- Never expose arbitrary transaction-signing endpoints to clients.
- Validate chain ID, contract allowlist, function selector and value limits.
- Use rate limiting and authentication separately from blockchain authorization.
- Store secrets in dedicated secret managers; rotate and scope them.
- Monitor abnormal nonce gaps, repeated reverts and unexpected destinations.
- Design withdrawal/treasury actions with stronger approval than ordinary user operations.

Required test plan

- Unit test address/input validation.
- Integration test RPC timeout/retry handling with a mock.
- Test receipt state transitions.
- Test idempotency.
- Test wrong-chain rejection.
- Test signer-policy rejection for unauthorized target/value.

Debugging playbook

1. Reproduce deterministically: pin block, seed, caller, input and contract version.
2. Read the revert/call trace before changing gas limits or code.
3. Check chain ID, deployed address, ABI and dependency version together.
4. Inspect state immediately before the failing call.
5. Reduce the scenario to the smallest failing sequence.
6. Convert the discovered failure into a regression test.
7. Only then refactor or optimize.

Production gate: A feature is not complete when it compiles. It is complete when its permissions, failure modes, tests, deployment configuration, monitoring signals and rollback/emergency assumptions are understood.

Senior Interview Practice, Homework and Portfolio Output

Interview questions - answer aloud first

Q: Why is nonce management hard in a backend?

A: Multiple concurrent workers using one account can race for the same sequential nonce, requiring coordination.

Q: What is the difference between broadcast and success?

A: Broadcast means a node accepted/propagated raw transaction data; success requires inclusion with successful execution and possibly additional finality policy.

Q: Why return bigint values as strings?

A: JavaScript Number cannot exactly represent arbitrary uint256 values.

Q: What should a signer service validate?

A: Business authorization plus chain, target, selector, value/amount, nonce/fee policy and any protocol-specific constraints.

Q: Why separate read and write services?

A: Writes carry key custody and state-changing authority, so separation reduces blast radius and simplifies scaling/security.

Q: What is idempotency?

A: Repeated delivery of the same business request produces one logical operation rather than duplicate effects.

Homework

- Build the REST API and document its transaction-state machine.
- Add a second RPC provider and write failover pseudocode.
- Create a secure-deployment checklist for environment variables, secrets and logging.

Portfolio deliverable

Ship this: Publish "Web3 Transaction Service" with chain guards, status state machine, bigint-safe APIs and signer threat model.

Official references for this day

<https://viem.sh/>

<https://docs.ethers.org/>

<https://web3py.readthedocs.io/>

<https://ethereum.org/en/developers/docs/apis/json-rpc/>

Day 14 - Blockchain Indexing, Events, The Graph and Data Pipelines

60-minute goal: Build reliable offchain data views from onchain events, handle reorgs and backfills, and understand when to use custom indexers versus Subgraphs.

Learning outcomes

- Explain and apply: onchain state is not optimized for application queries.
- Explain and apply: events are the primary ingestion interface.
- Explain and apply: reorgs require reversible ingestion.
- Explain and apply: derived views should be rebuildable.
- Complete the guided lab and capture reproducible evidence.
- Answer the senior interview questions using architecture/security trade-offs, not memorized definitions.

Minute-by-minute lecture plan

Time	Teaching focus
0-8 min	Why applications need indexed data
8-18 min	EVM logs, topics and event design
18-30 min	Backfills, checkpoints and reorg-safe ingestion
30-40 min	Database schema and derived views
40-50 min	The Graph: Subgraphs, manifests, mappings and GraphQL
50-60 min	Lab: index Transfer/Deposit events into a queryable view

Key terms for today

Term	Term	Term
event log	topic	indexed parameter
block number	log index	backfill
checkpoint	reorg	confirmation depth
idempotent ingestion	Subgraph	GraphQL
mapping		

Instructor opening script

Today's topic is blockchain indexing, events, the graph and data pipelines. The senior-level objective is not to memorize API names; it is to understand the underlying trust model, state transitions and failure modes well enough to make architecture decisions and debug unexpected behavior. During the lecture, repeatedly ask: what state is authoritative, who can change it, which external dependencies are trusted, and what property must remain true after every successful transaction?

Deep Theory and Mental Models

Onchain state is not optimized for application queries

Contracts expose current state and transaction-local events, but product questions often need history: all deposits by a user, volume by day, active positions, portfolio value or protocol analytics. Indexers transform canonical chain history into a database optimized for those queries.

Events are the primary ingestion interface

EVM logs include emitting address, topics and data. Indexed event parameters become topics that are efficient to filter, while non-indexed parameters are ABI encoded in data. Good contract event design makes later indexing dramatically easier.

Senior lens: Translate this concept into a review question: identify the authoritative state, caller/dependency assumptions, and the invariant that would fail if the concept were implemented incorrectly.

Reorgs require reversible ingestion

An indexer that immediately treats every observed block as permanent can accumulate incorrect data after a chain reorganization. Store block hashes, define confirmation/finality policy, and make processing idempotent so a block range can be rolled back and replayed safely.

Derived views should be rebuildable

Raw canonical events are valuable as an audit trail. Derived balances, daily aggregates and user summaries should be reproducible from raw events plus known contract state. If a derived table cannot be rebuilt, silent indexer bugs become hard to repair.

Senior lens: Translate this concept into a review question: identify the authoritative state, caller/dependency assumptions, and the invariant that would fail if the concept were implemented incorrectly.

Subgraphs provide a standardized indexing model

The Graph lets developers define data sources, ABIs, event handlers/mappings and a GraphQL schema. Graph Nodes process chain data into entities that clients query efficiently. It is excellent when the supported network/protocol model fits the application.

Custom indexers remain important

Teams may need custom retry policy, proprietary data joins, low-latency websockets, complex offchain enrichment or unsupported chains. Senior engineers choose based on reliability, control, operational cost and query needs rather than assuming one indexing stack fits every project.

Architecture, Data Flow and Trade-offs

Reference architecture / execution flow

```

EVM blocks
  | getLogs / websocket
  v
Ingestion worker
  | verify chain + block hash
  | decode ABI events
  v
Raw events table
  | idempotency key: chain + txHash + logIndex
  v
Derived processors
  +--> user balances
  +--> daily volume
  +--> positions
  v
API / GraphQL / analytics dashboard

Reorg -> remove affected block range -> replay

```

Trade-off analysis

1. More confirmations reduce reorg risk but add data latency.
2. Websocket subscriptions provide low latency but require reconnect/backfill logic.
3. Storing raw events costs more database space but greatly improves recoverability.
4. Subgraphs reduce custom infrastructure work but constrain execution environment and deployment model.
5. Polling is simpler than websockets but can increase RPC requests and detection latency.

Failure-mode questions to ask in a design review

- What happens if the external dependency is unavailable, stale or malicious?
- Can one compromised key change user balances, upgrade logic or move treasury assets?
- Does the system fail closed, fail open, or enter an ambiguous partially completed state?
- Which data is authoritative: contract state, emitted events, an indexer, an RPC response or an offchain database?
- How would you detect this component behaving incorrectly in production?
- Can the operation be replayed, reordered, reentered or executed on the wrong chain/context?
- What is the smallest privilege or interface that satisfies the requirement?

Architecture habit: For client work, draw the flow before writing code. A five-minute diagram often reveals missing authorization, wrong trust assumptions or unavailable recovery paths earlier than a compiler can.

Code Walkthrough and Implementation Notes

TypeScript pseudocode: idempotent event ingestion key

```
type RawLog = {
  chainId: number;
  blockNumber: bigint;
  blockHash: `0x${string}`;
  transactionHash: `0x${string}`;
  logIndex: number;
  address: `0x${string}`;
  topics: readonly `0x${string}`[];
  data: `0x${string}`;
};

function eventId(log: RawLog) {
  return `${log.chainId}:${log.transactionHash}:${log.logIndex}`;
}

async function ingest(log: RawLog) {
  // INSERT ... ON CONFLICT(event_id) DO NOTHING
  // Store raw blockHash so a reorg can be detected and reversed.
}
```

Walkthrough

Transaction hash alone is not enough because one transaction can emit multiple logs.

Chain ID must be part of multi-chain identity because the same transaction-hash bytes could theoretically appear on different chains.

Block hash retention helps detect that a previously processed height has a different canonical block.

Derived tables should update in the same database transaction as raw-event checkpointing when possible.

Senior implementation checklist

- Pin the compiler/framework/library versions used by the repository.
- Keep configuration such as chain IDs and dependency addresses outside business logic unless immutability is deliberate.
- Use typed interfaces instead of arbitrary calls whenever practical.
- Emit events for operationally important state changes.
- Write the failure-path test at the same time as the success-path implementation.
- Document what this example intentionally omits before anyone reuses it.

Guided Lab: Build, Observe and Break It

Lab steps

1. Modify a prior contract to emit Deposit and Withdraw events with indexed user fields.
2. Start Anvil and deploy the contract.
3. Write a script using getLogs to backfill from deployment block to latest.
4. Decode events and store them in SQLite/PostgreSQL or a JSONL file for the lab.
5. Use chainId:txHash:logIndex as an idempotency key.
6. Run the backfill twice and prove no duplicates are created.
7. Create a derived per-user deposit total and compare with contract state where meaningful.
8. Simulate a local chain reset/reorg scenario and describe how your indexer would detect mismatch.
9. Create a simple GraphQL schema for User and Deposit entities.
10. Read The Graph Subgraph docs and map your contract event to a conceptual mapping handler.

Expected evidence

- Backfill is repeatable without duplicate events.
- The derived view can be rebuilt from raw logs.
- Indexer stores checkpoint block number plus block hash.
- README includes reorg-handling policy and expected data latency.

Troubleshooting guide

If you see a problem: If getLogs range is rejected by a provider, chunk block ranges and retry with backoff.

If you see a problem: If decoded values are wrong, verify ABI matches the deployed contract version and event signature.

If you see a problem: If websocket drops, reconnect from the last finalized/checkpointed block rather than assuming no events were missed.

Lab rule: Do not "fix" a failing lab by removing validation or weakening the expected assertion. First identify whether the failure is environment, configuration, integration state or actual code behavior.

Security, Testing, Debugging and Production Readiness

Security review points

- Never treat indexer data as authorization for onchain actions; onchain contracts must enforce their own rules.
- Sanitize any offchain metadata joined to blockchain addresses/events.
- Protect database/API from large unbounded historical queries.
- Monitor indexer lag versus chain head.
- Handle contract upgrades/address changes explicitly in the data-source configuration.
- Do not silently rewrite historical raw data when a derived calculation changes.

Required test plan

- Idempotent duplicate-ingestion test.
- Backfill chunking test.
- ABI decode regression test.
- Reorg rollback/replay test or simulation.
- Derived-view rebuild test.
- Indexer lag alert test with mocked head/checkpoint values.

Debugging playbook

1. Reproduce deterministically: pin block, seed, caller, input and contract version.
2. Read the revert/call trace before changing gas limits or code.
3. Check chain ID, deployed address, ABI and dependency version together.
4. Inspect state immediately before the failing call.
5. Reduce the scenario to the smallest failing sequence.
6. Convert the discovered failure into a regression test.
7. Only then refactor or optimize.

Production gate: A feature is not complete when it compiles. It is complete when its permissions, failure modes, tests, deployment configuration, monitoring signals and rollback/emergency assumptions are understood.

Senior Interview Practice, Homework and Portfolio Output

Interview questions - answer aloud first

Q: Why are events useful for indexing?

A: They are an efficient append-only observation interface emitted during transactions and filterable by address/topics.

Q: Why store logIndex?

A: A transaction may emit multiple logs; txHash + logIndex uniquely identifies a specific log on a chain.

Q: How should an indexer handle reorgs?

A: Track block hashes/checkpoints, roll back affected canonical data and replay from a safe ancestor.

Q: What is a Subgraph?

A: A custom API built from blockchain data using defined data sources, mappings and a GraphQL schema.

Q: Why keep raw events?

A: They provide a rebuildable source for derived views and help recover from processor bugs.

Q: Can an indexer secure a smart contract?

A: No. Offchain data is for applications/analytics; authorization and asset safety must be enforced onchain.

Homework

- Build a small event-indexing service and expose GET /events?user=...
- Write a reorg incident runbook.
- Create one Subgraph schema/mapping sketch for the Day 19 capstone.

Portfolio deliverable

Ship this: Publish “EVM Event Indexer” with idempotent backfill, derived view, reorg policy and a Subgraph design note.

Official references for this day

<https://thegraph.com/docs/en/subgraphs/developing/introduction/>

<https://thegraph.com/docs/en/subgraphs/developing/creating/starting-your-subgraph/>

https://ethereum.org/en/developers/docs/apis/json-rpc/#eth_getlogs

Day 15 - Layer 2, Rollups, Data Availability and Scaling Architecture

60-minute goal: Explain modern Ethereum scaling systems, compare optimistic and ZK rollups, reason about sequencers/bridges/finality, and deploy applications with L2-specific operational awareness.

Learning outcomes

- Explain and apply: scaling separates execution from settlement.
- Explain and apply: optimistic rollups assume batches are valid unless challenged.
- Explain and apply: zk rollups prove state transitions.
- Explain and apply: data availability is essential to independent verification.
- Complete the guided lab and capture reproducible evidence.
- Answer the senior interview questions using architecture/security trade-offs, not memorized definitions.

Minute-by-minute lecture plan

Time	Teaching focus
0-8 min	Why scaling is multi-dimensional: execution, data and settlement
8-20 min	L1 versus L2 security relationships
20-32 min	Optimistic rollups, batches and challenge mechanisms
32-42 min	ZK rollups, validity proofs and verifier contracts
42-52 min	Sequencers, data availability, blobs and bridging UX
52-60 min	Lab: deploy same contract to L1-like local chain and an EVM L2 test environment

Key terms for today

Term	Term	Term
Layer 1	Layer 2	rollup
sequencer	batch	fraud proof
validity proof	data availability	blob
bridge	finality	withdrawal delay
sidechain		

Instructor opening script

Today's topic is layer 2, rollups, data availability and scaling architecture. The senior-level objective is not to memorize API names; it is to understand the underlying trust model, state transitions and failure modes well enough to make architecture decisions and debug unexpected behavior. During the lecture, repeatedly ask: what state is authoritative, who can change it, which external dependencies are trusted, and what property must remain true after every successful transaction?

Deep Theory and Mental Models

Scaling separates execution from settlement

Ethereum L2s process transactions outside Mainnet execution while using Ethereum for varying combinations of data availability, settlement and dispute/proof verification. The key question is not “is it faster?” but which security properties are inherited from L1 and under what failure assumptions.

Optimistic rollups assume batches are valid unless challenged

Transactions execute offchain and data/results are posted to Ethereum. The protocol allows incorrect claims to be challenged according to the rollup's dispute mechanism. This design affects withdrawal finality and operational assumptions around challengers and proof systems.

Senior lens: Translate this concept into a review question: identify the authoritative state, caller/dependency assumptions, and the invariant that would fail if the concept were implemented incorrectly.

ZK rollups prove state transitions

A ZK rollup submits cryptographic validity proofs that an offchain batch transitioned state correctly. Ethereum verifies the proof rather than replaying every transaction. Proving systems add complexity and computation while enabling different finality/withdrawal characteristics.

Data availability is essential to independent verification

A state commitment is not sufficient if users cannot obtain the underlying transaction/state data needed to reconstruct or exit. Rollups that publish required data to Ethereum inherit stronger data-availability properties than systems keeping data elsewhere under additional assumptions.

Senior lens: Translate this concept into a review question: identify the authoritative state, caller/dependency assumptions, and the invariant that would fail if the concept were implemented incorrectly.

Sequencers improve UX but introduce operational dependencies

Many rollups use a sequencer to order transactions and provide rapid confirmations. Users need to understand what happens during sequencer downtime, censorship, forced inclusion and L1 finalization. Fast “soft confirmation” is not necessarily the same as L1-settled finality.

Bridges are part of the application threat model

Deposits/withdrawals cross different execution domains and may involve canonical bridge contracts, messaging delays and token representations. Client applications must show users the correct network, token address and settlement expectations to prevent operational mistakes.

DAY 15 | LAYER 2, ROLLUPS, DATA AVAILABILITY AND SCALING ARCHITECTURE

Architecture, Data Flow and Trade-offs

Reference architecture / execution flow

```

User transaction
|
L2 sequencer / ordering
| fast local confirmation
v
L2 execution + state
| batch / proof data
v
Ethereum L1 contracts
+-- data availability
+-- settlement / dispute or proof verification
v
L1-finalized rollup state

Bridge contracts/messages connect asset state across domains.

```

Trade-off analysis

1. L2s reduce user execution cost but add bridge and sequencer concepts.
2. Optimistic designs can have challenge-related withdrawal delays; liquidity bridges improve UX with additional trust/economic assumptions.
3. ZK proof systems can provide rapid cryptographic finality after proof acceptance but are technologically complex.
4. Sidechains may be EVM-compatible and cheap but do not automatically inherit Ethereum security like rollups.
5. Application monitoring must track both L2 and relevant L1 settlement status.

Failure-mode questions to ask in a design review

- What happens if the external dependency is unavailable, stale or malicious?
- Can one compromised key change user balances, upgrade logic or move treasury assets?
- Does the system fail closed, fail open, or enter an ambiguous partially completed state?
- Which data is authoritative: contract state, emitted events, an indexer, an RPC response or an offchain database?
- How would you detect this component behaving incorrectly in production?
- Can the operation be replayed, reordered, reentered or executed on the wrong chain/context?
- What is the smallest privilege or interface that satisfies the requirement?

Architecture habit: For client work, draw the flow before writing code. A five-minute diagram often reveals missing authorization, wrong trust assumptions or unavailable recovery paths earlier than a compiler can.

Code Walkthrough and Implementation Notes

Solidity: chain-aware deployment guard example

```
pragma solidity ^0.8.36;

contract ChainConfiguredVault {
    uint256 public immutable expectedChainId;
    address public immutable oracle;

    constructor(uint256 expectedChainId_, address oracle_) {
        require(block.chainid == expectedChainId_, "wrong chain");
        require(oracle_ != address(0), "zero oracle");
        expectedChainId = expectedChainId_;
        oracle = oracle_;
    }
}

// Deployment tooling should also verify RPC chain ID, dependency addresses,
// explorer, bridge and oracle configuration before broadcasting.
```

Walkthrough

Contract-level chain guards can prevent accidental deployment using incorrect parameters, though they are not always desired for reusable bytecode.

Most L2 application code is EVM-compatible, but dependency addresses, gas semantics, predeploys, bridge contracts and finality assumptions differ.

Do not copy an Ethereum mainnet oracle/router address into an L2 deployment unless official documentation lists it for that network.

Monitoring should understand both L2 transaction status and any L1 batch/proof settlement relevant to your risk policy.

Senior implementation checklist

- Pin the compiler/framework/library versions used by the repository.
- Keep configuration such as chain IDs and dependency addresses outside business logic unless immutability is deliberate.
- Use typed interfaces instead of arbitrary calls whenever practical.
- Emit events for operationally important state changes.
- Write the failure-path test at the same time as the success-path implementation.
- Document what this example intentionally omits before anyone reuses it.

Guided Lab: Build, Observe and Break It

Lab steps

1. Choose one EVM rollup test environment supported by your tools.
2. Create per-network configuration files with chain ID, RPC, explorer and dependency placeholders.
3. Deploy ChainConfiguredVault locally and to the test environment.
4. Record deployment gas/cost metrics in native gas token units; avoid comparing fiat values unless timestamped.
5. Inspect the L2 explorer and find where available batch/L1 settlement information is shown.
6. Bridge only test assets if needed for the lab.
7. Document user deposit/withdrawal flow and expected finality language.
8. Create an outage scenario: sequencer unavailable but L1 operational. Write what your frontend/backend should display.
9. Compare rollup and sidechain security assumptions in a table.

Expected evidence

- Separate network configuration with no reused unverified dependency addresses.
- A written distinction among soft confirmation, L2 inclusion and L1 settlement.
- A bridge UX checklist that includes network, canonical token and expected waiting time.

Troubleshooting guide

If you see a problem: If deployment succeeds on the wrong chain, your tooling lacks sufficient guards; add explicit expected chain ID checks.

If you see a problem: If a faucet/bridge is unavailable, use local simulation rather than substituting real funds.

If you see a problem: If L2 gas accounting differs from your mainnet expectations, consult that network's current documentation; L1 data costs may be represented separately.

Lab rule: Do not "fix" a failing lab by removing validation or weakening the expected assertion. First identify whether the failure is environment, configuration, integration state or actual code behavior.

Security, Testing, Debugging and Production Readiness

Security review points

- Verify L2-specific oracle and bridge addresses from official sources.
- Model sequencer outage behavior for price-sensitive actions.
- Do not assume a bridge-wrapped token address equals the L1 token address.
- Monitor upgrade/admin controls of L2 dependencies when material to application security.
- Distinguish sidechains from rollups in client claims.
- Use canonical bridge paths for high-value workflows unless an alternative bridge's trust model is deliberately accepted.

Required test plan

- Deployment chain-ID tests.
- Configuration validation tests.
- Oracle/bridge address nonzero and code-existence checks in fork/test contexts.
- Frontend/backend status test for pending versus settled transaction.
- Scenario test for sequencer-down oracle policy.

Debugging playbook

1. Reproduce deterministically: pin block, seed, caller, input and contract version.
2. Read the revert/call trace before changing gas limits or code.
3. Check chain ID, deployed address, ABI and dependency version together.
4. Inspect state immediately before the failing call.
5. Reduce the scenario to the smallest failing sequence.
6. Convert the discovered failure into a regression test.
7. Only then refactor or optimize.

Production gate: A feature is not complete when it compiles. It is complete when its permissions, failure modes, tests, deployment configuration, monitoring signals and rollback/emergency assumptions are understood.

Senior Interview Practice, Homework and Portfolio Output

Interview questions - answer aloud first

Q: What makes an L2 different from a sidechain?

A: An L2 is designed to derive security/settlement properties from the base chain; a sidechain runs separate consensus and does not automatically inherit Ethereum security.

Q: What is an optimistic rollup?

A: An L2 that executes offchain and treats batches as valid unless successfully challenged through its dispute mechanism.

Q: What is a ZK rollup?

A: An L2 that posts validity proofs allowing L1 contracts to verify correct offchain state transitions.

Q: Why is data availability important?

A: Users/verifiers need the underlying data to reconstruct state and verify or exit safely.

Q: What is a sequencer?

A: An L2 component that orders transactions and often provides rapid confirmations before full L1 settlement.

Q: What should an app tell users about bridging?

A: Source/destination network, exact token representation, fees, expected settlement time and the trust model of the chosen bridge.

Homework

- Write a 1,200-word L2 architecture comparison for a client choosing between L1 and an EVM rollup.
- Create a network-configuration validator script.
- Explain how oracle risk changes during sequencer downtime.

Portfolio deliverable

Ship this: Publish "EVM L2 Deployment Lab" with network config, settlement diagram, bridge UX checklist and rollup-vs-sidechain comparison.

Official references for this day

<https://ethereum.org/en/developers/docs/scaling/>

<https://ethereum.org/en/developers/docs/scaling/optimistic-rollups/>

<https://ethereum.org/en/developers/docs/scaling/zk-rollups/>

<https://ethereum.org/en/developers/docs/scaling/sidechains/>

Day 16 - Cross-Chain Messaging, Bridges and Interoperability Security

60-minute goal: Design and review cross-chain systems by separating message verification, token movement, replay protection, finality assumptions, rate limits and emergency controls.

Learning outcomes

- Explain and apply: cross-chain messages cross consensus domains.
- Explain and apply: bridge models move value in different ways.
- Explain and apply: message identity must include domain separation.
- Explain and apply: finality is part of verification.
- Complete the guided lab and capture reproducible evidence.
- Answer the senior interview questions using architecture/security trade-offs, not memorized definitions.

Minute-by-minute lecture plan

Time	Teaching focus
0-8 min	Why cross-chain is not a normal contract call
8-18 min	Lock/mint, burn/mint and liquidity-network bridge models
18-30 min	Messaging lifecycle: source event, verification, destination execution
30-40 min	Replay protection, ordering, finality and destination validation
40-50 min	Bridge attack classes and defense-in-depth
50-60 min	Lab: design a two-chain message receiver with nonce/domain validation

Key terms for today

Term	Term	Term
bridge	message	source chain
destination chain	domain	nonce
replay protection	finality	validator set
light client	lock/mint	burn/mint
liquidity network	rate limit	

Instructor opening script

Today's topic is cross-chain messaging, bridges and interoperability security. The senior-level objective is not to memorize API names; it is to understand the underlying trust model, state transitions and failure modes well enough to make architecture decisions and debug unexpected behavior. During the lecture, repeatedly ask: what state is authoritative, who can change it, which external dependencies are trusted, and what property must remain true after every successful transaction?

Deep Theory and Mental Models

Cross-chain messages cross consensus domains

A contract call on one chain cannot synchronously execute code on another independent chain. Interoperability systems observe or prove a source event/state, transport a message through a verification mechanism, then execute an authorized destination call. Every stage introduces explicit trust and liveness assumptions.

Bridge models move value in different ways

Lock-and-mint bridges escrow an asset on the source and mint a representation on the destination. Burn-and-mint systems coordinate canonical supply across supported chains. Liquidity networks use inventory owned by liquidity providers and rebalance later. The accounting invariant differs for each model.

Senior lens: Translate this concept into a review question: identify the authoritative state, caller/dependency assumptions, and the invariant that would fail if the concept were implemented incorrectly.

Message identity must include domain separation

A destination receiver should know not only the payload but the authenticated source chain/domain, source sender and message identifier or nonce. Otherwise a valid message from one context may be replayed or interpreted in another context for which it was never intended.

Finality is part of verification

A bridge that acts on a source event too early can be exposed to reorgs. Different chains have different finality characteristics; the interoperability protocol must define when a source event is safe enough to relay. Faster bridging usually means additional liquidity or trust assumptions.

Senior lens: Translate this concept into a review question: identify the authoritative state, caller/dependency assumptions, and the invariant that would fail if the concept were implemented incorrectly.

Destination execution should be narrowly authorized

A generic cross-chain executor with arbitrary target and calldata can become protocol-wide remote root access. Prefer message schemas with explicit action types, destination allowlists, amount limits and validation of expected source senders.

Rate limits buy time during incidents

Bridges are attractive targets because they aggregate value and authority. Rate limits, per-asset caps, pauses, monitoring and delayed large transfers can reduce blast radius while teams investigate anomalies. They do not fix invalid verification, but they can make failures survivable.

DAY 16 | CROSS-CHAIN MESSAGING, BRIDGES AND INTEROPERABILITY SECURITY

Architecture, Data Flow and Trade-offs

Reference architecture / execution flow

```

Source Chain A
User -> Bridge Sender
      | lock/burn + emit message
      v
Verification layer
  validator network / proof / light-client mechanism
      | authenticated source domain + sender + payload
      v
Destination Chain B Receiver
+-- verify approved messenger
+-- verify source sender/domain
+-- verify nonce not processed
+-- enforce amount/action limits
      v
Destination action / token mint / app call
  
```

Trade-off analysis

1. More decentralized verification may improve trust minimization while increasing cost/latency/complexity.
2. Liquidity bridges offer fast UX but rely on liquidity providers and economic rebalancing.
3. Canonical bridges may have strongest alignment with a rollup but can have slower withdrawals.
4. Generic messaging improves flexibility but increases destination authorization surface.
5. Strict rate limits reduce catastrophic loss but can temporarily block legitimate high-volume users.

Failure-mode questions to ask in a design review

- What happens if the external dependency is unavailable, stale or malicious?
- Can one compromised key change user balances, upgrade logic or move treasury assets?
- Does the system fail closed, fail open, or enter an ambiguous partially completed state?
- Which data is authoritative: contract state, emitted events, an indexer, an RPC response or an offchain database?
- How would you detect this component behaving incorrectly in production?
- Can the operation be replayed, reordered, reentered or executed on the wrong chain/context?
- What is the smallest privilege or interface that satisfies the requirement?

Architecture habit: For client work, draw the flow before writing code. A five-minute diagram often reveals missing authorization, wrong trust assumptions or unavailable recovery paths earlier than a compiler can.

DAY 16 | CROSS-CHAIN MESSAGING, BRIDGES AND INTEROPERABILITY SECURITY

Code Walkthrough and Implementation Notes

Solidity: destination message receiver skeleton with replay protection

```
pragma solidity ^0.8.36;

contract MessageReceiver {
    address public immutable messenger;
    uint256 public immutable trustedSourceDomain;
    address public immutable trustedSourceSender;

    mapping(bytes32 => bool) public processed;

    error UnauthorizedMessenger();
    error InvalidSource();
    error Replay();

    constructor(address m, uint256 domain, address sender) {
        messenger = m;
        trustedSourceDomain = domain;
        trustedSourceSender = sender;
    }

    function receiveMessage(
        uint256 sourceDomain,
        address sourceSender,
        uint256 nonce,
        bytes calldata payload
    ) external {
        if (msg.sender != messenger) revert UnauthorizedMessenger();
        if (sourceDomain != trustedSourceDomain || sourceSender != trustedSourceSender)
            revert InvalidSource();

        bytes32 id = keccak256(abi.encode(sourceDomain, sourceSender, nonce, payload));
        if (processed[id]) revert Replay();
        processed[id] = true;

        // Decode a narrowly defined message schema and execute allowed action.
    }
}
```

Walkthrough

This skeleton does not implement a real bridge protocol; it demonstrates destination-side validation layers.

Real protocols define messenger interfaces, source-sender encoding and message IDs differently. Always use official integration libraries/contracts.

Replay state should be marked before any untrusted external action so reentrant execution cannot process the same message twice.

A payload should decode into an explicit versioned schema rather than arbitrary target + calldata unless the governance model deliberately grants that power.

Senior implementation checklist

- Pin the compiler/framework/library versions used by the repository.
- Keep configuration such as chain IDs and dependency addresses outside business logic unless immutability is deliberate.
- Use typed interfaces instead of arbitrary calls whenever practical.
- Emit events for operationally important state changes.
- Write the failure-path test at the same time as the success-path implementation.
- Document what this example intentionally omits before anyone reuses it.

Guided Lab: Build, Observe and Break It

Lab steps

1. Draw a two-chain token bridge architecture and mark which component guarantees supply accounting.
2. Implement the MessageReceiver with a mock messenger.
3. Test wrong messenger, wrong source domain, wrong source sender and duplicate nonce/payload.
4. Change the message schema to Action{kind,recipient,amount} instead of arbitrary calldata.
5. Add a per-day or per-epoch transfer limit in the lab and test the boundary.
6. Write a pause path controlled by a separate emergency role.
7. Model a source-chain reorg: identify which component should delay or invalidate relay.
8. Read a current interoperability protocol's official documentation and map its verification model at a high level.
9. Create an incident checklist for abnormal bridge volume.

Expected evidence

- Every message has a domain-separated identity.
- Duplicate processing fails.
- Only authenticated source sender/domain is accepted.
- Rate-limit and pause behavior is covered by tests and emits operational events.

Troubleshooting guide

If you see a problem: If addresses differ in representation across chains/protocols, use the official canonical encoding instead of assuming EVM address formatting everywhere.

If you see a problem: If the mock passes but real integration fails, compare actual messenger callback signature and source-sender semantics.

If you see a problem: If a message is delivered twice by a retrying relayer, application-level replay protection should make processing idempotent.

Lab rule: Do not "fix" a failing lab by removing validation or weakening the expected assertion. First identify whether the failure is environment, configuration, integration state or actual code behavior.

Security, Testing, Debugging and Production Readiness

Security review points

- Verify messenger contract and authenticated source sender separately.
- Use domain separation and nonces/message IDs.
- Avoid unrestricted arbitrary cross-chain execution.
- Apply per-token/action rate limits and monitoring.
- Document source finality assumptions.
- Use emergency pause with carefully governed authority.
- Treat bridge upgrades and validator-set changes as part of application dependency risk.

Required test plan

- Replay test.
- Source-domain/sender authentication tests.
- Rate-limit boundary tests.
- Pause/unpause tests.
- Property: each valid message ID changes destination accounting at most once.
- Mock malicious payload and ensure schema validation rejects it.

Debugging playbook

1. Reproduce deterministically: pin block, seed, caller, input and contract version.
2. Read the revert/call trace before changing gas limits or code.
3. Check chain ID, deployed address, ABI and dependency version together.
4. Inspect state immediately before the failing call.
5. Reduce the scenario to the smallest failing sequence.
6. Convert the discovered failure into a regression test.
7. Only then refactor or optimize.

Production gate: A feature is not complete when it compiles. It is complete when its permissions, failure modes, tests, deployment configuration, monitoring signals and rollback/emergency assumptions are understood.

Senior Interview Practice, Homework and Portfolio Output

Interview questions - answer aloud first

Q: Why is a cross-chain call asynchronous?

A: Independent chains do not share one synchronous execution environment; a source event/proof must be relayed and verified before destination execution.

Q: What is domain separation?

A: Binding authorization to a specific source chain/context so a valid message cannot be reused in another domain.

Q: Why is replay protection essential?

A: Relayers may retry and attackers may resubmit valid data; destination state must ensure each message is processed at most once.

Q: What is the danger of arbitrary target + calldata messaging?

A: It can create remote root-level authority if source authentication or governance is compromised.

Q: Why do bridges wait for finality?

A: Acting on a source event that is later reorged can create unbacked destination state.

Q: What does a rate limit accomplish?

A: It limits the amount of value/authority that can move during a time window, buying time to detect and respond to attacks.

Homework

- Write a bridge threat model covering source chain, messenger/verifier, relayer, destination receiver and admin keys.
- Compare canonical bridge, liquidity bridge and general messaging trust assumptions.
- Create a one-page "cross-chain integration acceptance checklist" for client work.

Portfolio deliverable

Ship this: Publish "Cross-Chain Security Lab" with receiver tests, replay/domain checks, rate limits, threat model and incident-response checklist.

Official references for this day

<https://docs.chain.link/ccip>

<https://docs.openzeppelin.com/contracts/5.x/api/interfaces>

<https://ethereum.org/en/developers/docs/bridges/>

Day 17 - Solana, Rust and Anchor: Accounts, PDAs, CPI and Program Security

60-minute goal: Build a second blockchain mental model beyond the EVM by learning Solana's account model, Rust program execution, Anchor validation, PDAs, SPL tokens and cross-program invocation.

Learning outcomes

- Explain and apply: solana separates executable programs from state accounts.
- Explain and apply: accounts declare access up front.
- Explain and apply: pdas create deterministic program-controlled addresses.
- Explain and apply: anchor converts many checks into declarative constraints.
- Complete the guided lab and capture reproducible evidence.
- Answer the senior interview questions using architecture/security trade-offs, not memorized definitions.

Minute-by-minute lecture plan

Time	Teaching focus
0-8 min	Solana architecture and how it differs from EVM contracts
8-18 min	Accounts, owners, data, lamports and executable programs
18-30 min	Instructions, transactions, signers and account lists
30-40 min	PDAs, seeds and deterministic authority
40-50 min	Anchor account validation, CPI and SPL Token concepts
50-60 min	Lab: Anchor counter/vault program with PDA state

Key terms for today

Term	Term	Term
program	account	owner
lamports	instruction	transaction
signer	PDA	seed
bump	CPI	Anchor
IDL	SPL Token	ATA

Instructor opening script

Today's topic is solana, rust and anchor: accounts, pdas, cpi and program security. The senior-level objective is not to memorize API names; it is to understand the underlying trust model, state transitions and failure modes well enough to make architecture decisions and debug unexpected behavior. During the lecture, repeatedly ask: what state is authoritative, who can change it, which external dependencies are trusted, and what property must remain true after every successful transaction?

Deep Theory and Mental Models

Solana separates executable programs from state accounts

A deployed Solana program is executable code, while mutable application state lives in separate accounts passed to instructions. This is a major conceptual shift from the EVM pattern where contract code and storage are associated with one contract address.

Accounts declare access up front

Instructions specify which accounts they will read/write and which must sign. This explicit account list enables runtime parallelism when transactions do not contend for the same writable state. It also creates security obligations: programs must verify that each supplied account has the expected owner, seeds, relationships and authority.

Senior lens: Translate this concept into a review question: identify the authoritative state, caller/dependency assumptions, and the invariant that would fail if the concept were implemented incorrectly.

PDAs create deterministic program-controlled addresses

Program Derived Addresses are deterministically derived from seeds plus a program ID and do not have normal private keys. A program can sign for its PDA during CPI using the same seeds. PDAs are widely used for vault authorities, user state and protocol configuration.

Anchor converts many checks into declarative constraints

Anchor's `#[derive(Accounts)]` constraints can validate signers, mutability, owners, PDA seeds, `has_one` relationships and token-account properties. These macros reduce boilerplate but do not eliminate the need to understand the underlying checks and trust boundaries.

Senior lens: Translate this concept into a review question: identify the authoritative state, caller/dependency assumptions, and the invariant that would fail if the concept were implemented incorrectly.

CPI is Solana's composability mechanism

One program can invoke another program through cross-program invocation. For example, an application program can ask the SPL Token program to transfer tokens, with a PDA signing as authority. The called program is an external dependency and account constraints must ensure the correct program/account relationships.

Program upgrade authority is operational power

Solana programs may be upgradeable depending on deployment authority configuration. Teams should know who controls upgrade authority, how it is protected, and when making a program immutable is appropriate. Verifiable builds help users compare onchain bytecode with public source.

Architecture, Data Flow and Trade-offs

Reference architecture / execution flow

```
Transaction
+-- Instruction 1: Program A
|   Accounts: user(signer), state PDA(writable), system program
|
+-- Instruction 2: Program B
    Accounts: token accounts, authority PDA, token program

Program A executes against supplied accounts
|
+-- validates PDA seeds / owners / signers
+-- may invoke Program B via CPI
    and sign for PDA using seeds
```

Trade-off analysis

1. Explicit account lists enable parallelism but make account-validation bugs critical.
2. Anchor increases developer velocity while adding framework conventions that auditors must understand.
3. PDAs remove private-key custody for program authority but seed design becomes part of authorization logic.
4. Upgradeable programs support maintenance but create upgrade-authority trust.
5. CPI enables composability while expanding the set of external programs/accounts that must be validated.

Failure-mode questions to ask in a design review

- What happens if the external dependency is unavailable, stale or malicious?
- Can one compromised key change user balances, upgrade logic or move treasury assets?
- Does the system fail closed, fail open, or enter an ambiguous partially completed state?
- Which data is authoritative: contract state, emitted events, an indexer, an RPC response or an offchain database?
- How would you detect this component behaving incorrectly in production?
- Can the operation be replayed, reordered, reentered or executed on the wrong chain/context?
- What is the smallest privilege or interface that satisfies the requirement?

Architecture habit: For client work, draw the flow before writing code. A five-minute diagram often reveals missing authorization, wrong trust assumptions or unavailable recovery paths earlier than a compiler can.

Rust/Anchor: PDA-backed counter sketch

Walkthrough

PDA seeds should be stable, unique to the intended namespace and documented because changing seeds changes derived addresses.

- Pin the compiler/framework/library versions used by the repository.
- Keep configuration such as chain IDs and dependency addresses outside business logic unless immutability is deliberate.
- Use typed interfaces instead of arbitrary calls whenever practical.
- Emit events for operationally important state changes.
- Write the failure-path test at the same time as the success-path implementation.
- Document what this example intentionally omits before anyone reuses it.

Guided Lab: Build, Observe and Break It

Lab steps

1. Install Solana/Rust/Anchor using current official instructions; on Windows use the supported WSL path described by Solana docs.
2. Create an Anchor project and run the default test.
3. Implement a PDA-backed user counter.
4. Derive the PDA in the client and compare with the onchain account address.
5. Add an authority constraint so another signer cannot increment a user-owned counter.
6. Write a failing test for an incorrect PDA.
7. Create a second exercise using an SPL token account and an associated token account.
8. Draw a CPI flow where your PDA authorizes an SPL Token transfer.
9. Inspect program deployment authority with Solana CLI on a local/devnet program.
10. Document how you would make a program immutable after governance approval.

Expected evidence

- Anchor tests derive the same PDA as the program.
- Unauthorized signer test fails.
- Student can explain account owner versus application “authority” as separate concepts.
- Upgrade-authority state is documented.

Troubleshooting guide

If you see a problem: If PDA derivation differs client/server, compare seed byte encoding and program ID.

If you see a problem: If account space is too small, serialization will fail; calculate discriminator plus every field.

If you see a problem: If Windows tooling fails natively, follow official WSL installation rather than mixing incompatible binaries.

If you see a problem: If CPI reports incorrect program/account owner, validate that the passed Token/System program and token accounts are canonical for the selected cluster.

Lab rule: Do not “fix” a failing lab by removing validation or weakening the expected assertion. First identify whether the failure is environment, configuration, integration state or actual code behavior.

Security, Testing, Debugging and Production Readiness

Security review points

- Validate every externally supplied account relationship.
- Use signer and PDA constraints rather than trusting client-provided authority fields.
- Avoid unchecked accounts unless the program validates them manually and documentation explains why.
- Use checked arithmetic or deliberate overflow semantics.
- Protect program upgrade authority and consider multisig/governance.
- Validate token mint, token-account owner and authority on SPL flows.

Required test plan

- Initialization and PDA uniqueness tests.
- Unauthorized authority test.
- Wrong seed/bump test.
- Overflow/error-boundary test.
- CPI mock/devnet test for token transfer.
- Property: a user cannot mutate another user's PDA-backed state without required authority.

Debugging playbook

1. Reproduce deterministically: pin block, seed, caller, input and contract version.
2. Read the revert/call trace before changing gas limits or code.
3. Check chain ID, deployed address, ABI and dependency version together.
4. Inspect state immediately before the failing call.
5. Reduce the scenario to the smallest failing sequence.
6. Convert the discovered failure into a regression test.
7. Only then refactor or optimize.

Production gate: A feature is not complete when it compiles. It is complete when its permissions, failure modes, tests, deployment configuration, monitoring signals and rollback/emergency assumptions are understood.

Senior Interview Practice, Homework and Portfolio Output

Interview questions - answer aloud first

Q: What is the biggest conceptual difference from EVM storage?

A: Solana program state lives in separate accounts supplied to instructions rather than contract-local storage at the program address.

Q: What is a PDA?

A: A deterministic address derived from seeds and a program ID, controlled by the program without a normal private key.

Q: Why are account constraints critical?

A: Clients supply accounts; the program must prove they have the expected owner, signer, seeds and relationships.

Q: What is CPI?

A: Cross-program invocation, where one Solana program invokes another program during the same transaction.

Q: What does Anchor provide?

A: Rust macros/framework conventions for account validation, serialization, IDLs and reduced boilerplate.

Q: What is program upgrade authority?

A: The authority permitted to deploy updated program code while the program remains upgradeable.

Homework

- Create a comparison table: EVM contract storage versus Solana accounts.
- Extend the counter to store a last-updated slot/timestamp field and handle space/versioning.
- Write a security checklist for Anchor account validation.

Portfolio deliverable

Ship this: Publish “Solana Anchor PDA Lab” with Rust code, tests, EVM-vs-Solana architecture note and upgrade-authority checklist.

Official references for this day

<https://solana.com/docs/core/programs>

<https://solana.com/docs/programs/rust>

<https://solana.com/docs/intro/installation>

<https://solana.com/docs/core/cpi>

Day 18 - Senior Blockchain System Architecture, Governance and Production Operations

60-minute goal: Design end-to-end blockchain systems that integrate contracts, wallets, backends, indexers, oracles, governance, monitoring and deployment controls with explicit trust boundaries.

Learning outcomes

- Explain and apply: the product is larger than its contracts.
- Explain and apply: put trust-sensitive rules where they can be enforced.
- Explain and apply: privilege should be decomposed by consequence.
- Explain and apply: monitoring is part of security.
- Complete the guided lab and capture reproducible evidence.
- Answer the senior interview questions using architecture/security trade-offs, not memorized definitions.

Minute-by-minute lecture plan

Time	Teaching focus
0-8 min	From smart contract to socio-technical system
8-18 min	Onchain versus offchain responsibility decisions
18-30 min	Privilege architecture: multisigs, timelocks, RBAC and emergency roles
30-40 min	Monitoring, alerting, incident response and circuit breakers
40-50 min	Deployment pipelines, configuration, verification and runbooks
50-60 min	Architecture exercise: institutional DeFi vault/lending platform

Key terms for today

Term	Term	Term
architecture	trust boundary	multisig
timelock	RBAC	guardian
circuit breaker	monitoring	runbook
deployment pipeline	configuration management	SLO
incident response		

Instructor opening script

Today's topic is senior blockchain system architecture, governance and production operations. The senior-level objective is not to memorize API names; it is to understand the underlying trust model, state transitions and failure modes well enough to make architecture decisions and debug unexpected behavior. During the lecture, repeatedly ask: what state is authoritative, who can change it, which external dependencies are trusted, and what property must remain true after every successful transaction?

Deep Theory and Mental Models

The product is larger than its contracts

A production dapp includes wallet UX, RPC, indexers, databases, oracles, cloud services, DNS, signing infrastructure, governance and human operators. A contract audit cannot cover all of these. Senior engineers map the entire system and state which components are authoritative for each decision.

Put trust-sensitive rules where they can be enforced

Asset ownership, withdrawal authorization, share accounting and protocol limits generally belong onchain if users expect trust-minimized enforcement. Search, analytics, notifications and heavy computation often belong offchain. The decision depends on verifiability, cost, latency, privacy and upgrade needs.

Senior lens: Translate this concept into a review question: identify the authoritative state, caller/dependency assumptions, and the invariant that would fail if the concept were implemented incorrectly.

Privilege should be decomposed by consequence

Upgrade, pause, treasury movement, parameter tuning and oracle emergency actions do not need the same authority. Use separate roles and stronger controls as potential impact increases. A 2-of-3 operational multisig may be reasonable for pause while upgrades require a larger governance multisig plus timelock.

Monitoring is part of security

You cannot respond to abnormal minting, oracle staleness, bridge volume, admin changes or repeated reverts if nobody measures them. Define alerts from contract events, balances, role changes, RPC health and external dependencies. Each alert should map to an owner and a runbook.

Senior lens: Translate this concept into a review question: identify the authoritative state, caller/dependency assumptions, and the invariant that would fail if the concept were implemented incorrectly.

Runbooks reduce improvisation during incidents

A runbook states how to verify an alert, who can pause, how communications occur, which keys are needed, what evidence to preserve, and how to restore safely. Emergency functions without practiced operational procedures create false confidence.

Architecture reviews should expose failure modes

For every component ask: what if unavailable, stale, compromised, malicious, upgraded or misconfigured? Then decide whether the system fails open, fails closed, degrades safely or requires human intervention. This is the core of senior-level system thinking.

Reference architecture / execution flow



1. More onchain logic improves verifiability but costs gas and is harder to change.
2. A timelock improves transparency/reaction time but can slow urgent patches.
3. A guardian can pause quickly but becomes a centralized availability control.
4. Multiple service providers reduce single points of failure but increase configuration complexity.
5. Automated incident actions can respond quickly but must avoid false-positive catastrophic actions.

Failure-mode questions to ask in a design review

- What happens if the external dependency is unavailable, stale or malicious?
- Can one compromised key change user balances, upgrade logic or move treasury assets?
- Does the system fail closed, fail open, or enter an ambiguous partially completed state?
- Which data is authoritative: contract state, emitted events, an indexer, an RPC response or an offchain database?
- How would you detect this component behaving incorrectly in production?
- Can the operation be replayed, reordered, reentered or executed on the wrong chain/context?
- What is the smallest privilege or interface that satisfies the requirement?

Architecture habit: For client work, draw the flow before writing code. A five-minute diagram often reveals missing authorization, wrong trust assumptions or unavailable recovery paths earlier than a compiler can.

Code Walkthrough and Implementation Notes

Architecture policy pseudocode: role matrix as code/data

```
roles:
  upgrade_admin:
    holder: governance_multisig
    delay: 48h_timelock
    permissions: [upgrade_proxy]
  guardian:
    holder: operations_multisig
    delay: none
    permissions: [pause]
  treasury:
    holder: treasury_multisig
    delay: 12h_for_large_transfers
    permissions: [withdraw_fees]

alerts:
  - oracle_stale > maxAge
  - proxy_upgraded
  - role_granted_or_revoked
  - daily_outflow > threshold
  - contract_balance_drop > threshold
  - indexer_lag > blocks_threshold
```

Walkthrough

Treat role/configuration files as reviewable infrastructure artifacts even if execution happens through multisig UIs.

Every permission should have an expected holder and operational process. Unknown or obsolete roles are governance debt.

Alerts should be actionable; a noisy alert that operators ignore is not a control.

Large-value protocols often use different controls for routine maintenance, emergency response and irreversible changes.

Senior implementation checklist

- Pin the compiler/framework/library versions used by the repository.
- Keep configuration such as chain IDs and dependency addresses outside business logic unless immutability is deliberate.
- Use typed interfaces instead of arbitrary calls whenever practical.
- Emit events for operationally important state changes.
- Write the failure-path test at the same time as the success-path implementation.
- Document what this example intentionally omits before anyone reuses it.

Guided Lab: Build, Observe and Break It

Lab steps

1. Choose the Day 19 capstone and draw a full system context diagram.
2. Create an actor list: user, strategist, pauser, upgrader, treasury, oracle, DEX, RPC, indexer operator.
3. Create a privilege matrix with impact and recommended holder for each role.
4. Define five onchain and five offchain alerts.
5. Write one incident runbook for stale oracle and one for suspicious upgrade/role change.
6. Create a deployment checklist: compiler, commit, chain ID, addresses, constructor/initializer, verification, role transfer.
7. Define finality/confirmation policy for user deposits and backend accounting.
8. Document secrets and keys by storage location and rotation owner.
9. Perform a "dependency failure review" for RPC, oracle, DEX and indexer.
10. Present the architecture aloud in five minutes as if to a client CTO.

Expected evidence

- Architecture diagram covers data plane, governance plane and observability plane.
- Every privileged action has an owner and delay policy.
- Every critical dependency has an outage/compromise response.
- Deployment checklist includes post-deploy verification, not only broadcast.

Troubleshooting guide

If you see a problem: If responsibilities overlap ambiguously, assign one authoritative source for each decision: contract state, indexer, backend database or governance.

If you see a problem: If an alert has no defined response owner, it is incomplete.

If you see a problem: If the same EOA controls upgrade, treasury and pause, document the concentration and redesign unless justified.

Lab rule: Do not "fix" a failing lab by removing validation or weakening the expected assertion. First identify whether the failure is environment, configuration, integration state or actual code behavior.

Security, Testing, Debugging and Production Readiness

Security review points

- Use multisig/timelock for high-impact administration.
- Separate emergency pause from unrestricted upgrade/treasury authority.
- Monitor all role and implementation changes.
- Maintain tested incident runbooks.
- Validate configuration by chain ID and code hash where practical.
- Use least privilege for backend signers.
- Record exact deployed versions and source verification evidence.

Required test plan

- Architecture review checklist completed.
- Privilege-matrix review.
- Incident tabletop scenario.
- Deployment rehearsal on a fork/test environment.
- Monitoring test with synthetic event/threshold breach.

Debugging playbook

1. Reproduce deterministically: pin block, seed, caller, input and contract version.
2. Read the revert/call trace before changing gas limits or code.
3. Check chain ID, deployed address, ABI and dependency version together.
4. Inspect state immediately before the failing call.
5. Reduce the scenario to the smallest failing sequence.
6. Convert the discovered failure into a regression test.
7. Only then refactor or optimize.

Production gate: A feature is not complete when it compiles. It is complete when its permissions, failure modes, tests, deployment configuration, monitoring signals and rollback/emergency assumptions are understood.

Senior Interview Practice, Homework and Portfolio Output

Interview questions - answer aloud first

Q: What makes someone a senior blockchain engineer rather than a Solidity coder?

A: They can reason across contracts, dependencies, keys, governance, data pipelines, operations, failure modes and client requirements.

Q: Why separate guardian and upgrade roles?

A: Pause needs rapid limited authority; upgrades have broader irreversible power and should usually require stronger governance.

Q: What is a runbook?

A: A predefined operational procedure for verifying and responding to incidents or maintenance events.

Q: What is a system of record?

A: The authoritative source for a particular fact; e.g., contract state for balances rather than an eventually consistent indexer.

Q: Why monitor role events?

A: A role change can alter protocol authority even if no asset moves immediately.

Q: What should architecture review ask about every dependency?

A: What happens if it is unavailable, stale, compromised, malicious, upgraded or misconfigured?

Homework

- Write a 3-5 page architecture decision record for the capstone.
- Create an incident severity matrix and communication flow.
- Prepare a client discovery checklist for new blockchain projects.

Portfolio deliverable

Ship this: Publish “Senior Blockchain Architecture Pack” with system diagram, privilege matrix, alerts, runbooks, ADR and deployment checklist.

Official references for this day

<https://docs.openzeppelin.com/contracts/5.x/access-control>

<https://docs.openzeppelin.com/contracts/5.x/governance>

<https://ethereum.org/en/developers/docs/>

Day 19 - Capstone: Production-Style DeFi Vault from Requirements to Deployment

60-minute goal: Integrate Solidity, ERC-4626 thinking, testing, oracles, access control, backend/indexing and production documentation into one portfolio-grade project.

Learning outcomes

- Explain and apply: start with invariants, not features.
- Explain and apply: ERC-4626 provides a common vault vocabulary.
- Explain and apply: strategy boundaries should be narrow.
- Explain and apply: fees are accounting logic and security logic.
- Complete the guided lab and capture reproducible evidence.
- Answer the senior interview questions using architecture/security trade-offs, not memorized definitions.

Minute-by-minute lecture plan

Time	Teaching focus
0-8 min	Requirements and invariant definition
8-18 min	Vault/share accounting and ERC-4626 interface concepts
18-30 min	Roles, fees, pause, strategy boundaries and oracle policy
30-40 min	Testing plan: unit/fuzz/invariant/fork
40-50 min	Backend/indexer/frontend integration contracts
50-60 min	Capstone execution plan and portfolio acceptance rubric

Key terms for today

Term	Term	Term
vault	share	asset
ERC-4626	deposit	mint
withdraw	redeem	strategy
fee	high-water mark	solvency
pause	capstone	

Instructor opening script

Today's topic is capstone: production-style defi vault from requirements to deployment. The senior-level objective is not to memorize API names; it is to understand the underlying trust model, state transitions and failure modes well enough to make architecture decisions and debug unexpected behavior. During the lecture, repeatedly ask: what state is authoritative, who can change it, which external dependencies are trusted, and what property must remain true after every successful transaction?

DAY 19 | CAPSTONE: PRODUCTION-STYLE DEFI VAULT FROM REQUIREMENTS TO DEPLOYMENT

Deep Theory and Mental Models

Start with invariants, not features

The capstone vault must define what a share represents, how assets are valued, who may move funds into strategies, how fees are calculated, and what happens during pause or strategy failure. Features should be rejected if they cannot be reconciled with clear accounting invariants.

ERC-4626 provides a common vault vocabulary

The standard defines relationships among assets, shares, deposits, mints, withdrawals and redemptions. It improves integration but does not solve strategy risk, oracle design or every rounding issue. OpenZeppelin's ERC-4626 documentation specifically highlights security considerations around share pricing and initial deposits.

Senior lens: Translate this concept into a review question: identify the authoritative state, caller/dependency assumptions, and the invariant that would fail if the concept were implemented incorrectly.

Strategy boundaries should be narrow

The vault should not grant arbitrary strategy contracts unlimited ability to move every asset unless the governance model deliberately accepts that risk. Use approved strategy interfaces, debt/asset caps, withdrawal paths and emergency revocation. External protocol integrations should be tested on forks.

Fees are accounting logic and security logic

Management, performance or withdrawal fees affect share value and user expectations. Rounding direction must be explicit. Fee recipients and update authority are privileged parameters. A tiny rounding bias repeated many times can become economically material.

Senior lens: Translate this concept into a review question: identify the authoritative state, caller/dependency assumptions, and the invariant that would fail if the concept were implemented incorrectly.

Capstone quality comes from evidence

A portfolio project earns credibility through tests, threat model, deployment reproducibility, role matrix, screenshots/diagrams and known limitations. Large code volume without test coverage or documentation does not demonstrate seniority.

Integration contracts should be stable and observable

Emit events for deposits, withdrawals, fee updates, strategy changes and emergencies. Backends/indexers should consume these events, but the contract remains authoritative for user rights. Frontends should preview operations and surface slippage/fee/paused states clearly.

Architecture, Data Flow and Trade-offs

Reference architecture / execution flow

```

User
| deposit asset
v
ERC-4626-style Vault
+-- share accounting
+-- deposit/withdraw caps
+-- pause/emergency rules
+-- fee accounting
|
+--> Approved Strategy Adapter -> external DeFi protocol
|
+--> Oracle / valuation dependency (only if strategy needs it)

Offchain:
Indexer -> API -> dashboard
Monitoring -> alerts on role/strategy/price/outflow changes
Governance -> multisig/timelock for upgrades and sensitive parameters

```

Trade-off analysis

1. A simple custody-only vault is easier to secure but produces no strategy yield.
2. Strategies improve capital utility but add protocol, oracle and liquidity risk.
3. Upgradeable vaults ease maintenance but add governance trust.
4. Deposit caps limit growth/blast radius but can reduce product scalability.
5. Emergency pause protects against active exploits but may temporarily block user actions; emergency withdrawal policy must be explicit.

Failure-mode questions to ask in a design review

- What happens if the external dependency is unavailable, stale or malicious?
- Can one compromised key change user balances, upgrade logic or move treasury assets?
- Does the system fail closed, fail open, or enter an ambiguous partially completed state?
- Which data is authoritative: contract state, emitted events, an indexer, an RPC response or an offchain database?
- How would you detect this component behaving incorrectly in production?
- Can the operation be replayed, reordered, reentered or executed on the wrong chain/context?
- What is the smallest privilege or interface that satisfies the requirement?

Architecture habit: For client work, draw the flow before writing code. A five-minute diagram often reveals missing authorization, wrong trust assumptions or unavailable recovery paths earlier than a compiler can.

DAY 19 | CAPSTONE: PRODUCTION-STYLE DEFI VAULT FROM REQUIREMENTS TO DEPLOYMENT

Code Walkthrough and Implementation Notes

Solidity: capstone contract skeleton emphasizing interfaces and controls

```
// PSEUDOCODE-LIKE SKELETON: use OpenZeppelin ERC4626 in the actual project.
pragma solidity ^0.8.36;

interface IStrategy {
    function totalAssets() external view returns (uint256);
    function deposit(uint256 assets) external;
    function withdraw(uint256 assets, address receiver) external returns (uint256 received);
}

contract VaultDesignNotes {
    // Core invariants to implement/test:
    // 1. totalAssets() accurately includes vault + strategy assets.
    // 2. A user can never redeem more claim than owned shares represent.
    // 3. total strategy debt <= configured strategy cap.
    // 4. paused state blocks risk-increasing operations.
    // 5. fee accounting cannot mint arbitrary value to fee recipient.
    // 6. only approved governance can change strategy / limits / fee params.
}
```

Walkthrough

Using a skeleton prevents false confidence from an incomplete copied vault implementation. The actual capstone should import the current OpenZeppelin ERC4626 base and implement only needed custom logic.

Write invariants before the code so tests remain a specification during refactors.

External strategy totalAssets can itself be an assumption. Decide whether the vault trusts strategy accounting, independently values positions or enforces withdrawal-based checks.

Capstone scope should be achievable: one asset, one strategy adapter and a mocked/known protocol integration is enough to demonstrate engineering quality.

Senior implementation checklist

- Pin the compiler/framework/library versions used by the repository.
- Keep configuration such as chain IDs and dependency addresses outside business logic unless immutability is deliberate.
- Use typed interfaces instead of arbitrary calls whenever practical.
- Emit events for operationally important state changes.
- Write the failure-path test at the same time as the success-path implementation.
- Document what this example intentionally omits before anyone reuses it.

DAY 19 | CAPSTONE: PRODUCTION-STYLE DEFI VAULT FROM REQUIREMENTS TO DEPLOYMENT

Guided Lab: Build, Observe and Break It

Lab steps

1. Create a repository with contracts, test, script, backend, indexer, docs and audit folders.
2. Write REQUIREMENTS.md with supported asset, roles, fee policy, strategy model and emergency behavior.
3. Write INVARIANTS.md with at least ten properties.
4. Implement a minimal ERC-4626-based vault using current OpenZeppelin code.
5. Add AccessControl/Ownable-style governance appropriate to your design and a pause control.
6. Implement one strategy adapter against a mock first; later use a fork for a real integration read/write test if appropriate.
7. Add deposit cap and strategy debt cap.
8. Write unit tests for all roles and boundary conditions.
9. Add fuzz tests for deposit/withdraw round trips within rounding tolerance.
10. Add stateful invariants for solvency/share claims.
11. Create a fork integration test.
12. Add deployment script and post-deployment role assertions.
13. Build a simple event indexer and API endpoint for user history.
14. Create ARCHITECTURE.md, THREAT_MODEL.md, RUNBOOK.md and AUDIT_NOTES.md.

Expected evidence

- Clean build/test from a fresh checkout.
- No hardcoded private keys or production secrets.
- At least ten documented invariants, with the most important encoded as tests.
- Role matrix and upgrade/emergency policy are explicit.
- Fork integration uses pinned block/dependency addresses.
- README contains architecture diagram, setup, limitations and screenshots or example outputs.

Troubleshooting guide

If you see a problem: If share math produces unexpected rounding, compare preview methods and ERC-4626 rounding semantics before adding ad hoc compensating arithmetic.

If you see a problem: If invariant tests fail only after many operations, minimize the sequence; the bug may be cumulative rounding or accounting drift.

If you see a problem: If strategy withdrawal returns less than requested, the vault must have a defined loss/liquidity policy rather than assuming full return.

If you see a problem: If frontend history differs from state, treat contract state as authority and debug indexer reorg/backfill logic.

Lab rule: Do not “fix” a failing lab by removing validation or weakening the expected assertion. First identify whether the failure is environment, configuration, integration state or actual code behavior.

Security, Testing, Debugging and Production Readiness

Security review points

- Use SafeERC20 for asset transfers.
- Protect strategy changes, fee changes and upgrades with high-assurance governance.
- Enforce caps and pause risk-increasing actions.
- Avoid arbitrary strategy call execution from user input.
- Validate oracle freshness if valuation depends on external prices.
- Test donation/inflation and first-depositor edge cases relevant to ERC-4626 design.
- Monitor large outflows, role changes, strategy changes and paused state.

Required test plan

- Unit suite for every external method.
- Fuzz deposit/redeem properties.
- Stateful solvency/accounting invariants.
- Fork strategy integration test.
- Role/pausing tests.
- Donation/rounding edge-case tests.
- Deployment postcondition tests.
- Indexer rebuild/reorg test.

Debugging playbook

1. Reproduce deterministically: pin block, seed, caller, input and contract version.
2. Read the revert/call trace before changing gas limits or code.
3. Check chain ID, deployed address, ABI and dependency version together.
4. Inspect state immediately before the failing call.
5. Reduce the scenario to the smallest failing sequence.
6. Convert the discovered failure into a regression test.
7. Only then refactor or optimize.

Production gate: A feature is not complete when it compiles. It is complete when its permissions, failure modes, tests, deployment configuration, monitoring signals and rollback/emergency assumptions are understood.

DAY 19 | CAPSTONE: PRODUCTION-STYLE DEFI VAULT FROM REQUIREMENTS TO DEPLOYMENT

Senior Interview Practice, Homework and Portfolio Output

Interview questions - answer aloud first

Q: Why choose ERC-4626 for a vault?

A: It standardizes asset/share operations and improves composability, while still requiring careful accounting and security design.

Q: What is the most important capstone deliverable?

A: Evidence of correct reasoning: invariants, tests, threat model, architecture and reproducible deployment - not merely code volume.

Q: How should strategy risk be bounded?

A: Approved adapters, debt/asset caps, explicit withdrawal behavior, governance control, monitoring and fork-tested dependencies.

Q: Why are rounding rules security-relevant?

A: Repeated rounding can transfer value between users/protocol and can be exploited at edge cases if share math is biased.

Q: What should pause block?

A: At minimum risk-increasing operations defined by the design; emergency withdrawals may follow a separate carefully specified path.

Q: What is the source of truth for user share ownership?

A: The onchain vault/token state, not the offchain indexer or UI cache.

Homework

- Complete the capstone to portfolio-ready quality over the next 7-14 practice days.
- Write a self-audit using Day 8 methodology.
- Record a 5-minute screen-share explanation of architecture, invariants and tests as interview practice.

Portfolio deliverable

Ship this: Your primary portfolio project: a production-style DeFi Vault repository that demonstrates contracts, tests, architecture, security, indexing and operational maturity.

Official references for this day

<https://docs.openzeppelin.com/contracts/5.x/erc4626>

<https://docs.openzeppelin.com/contracts/5.x/api/token/ERC20#ERC4626>

<https://www.getfoundry.sh/guides/invariant-testing>

Day 20 - Senior Blockchain Career: Upwork, LinkedIn, Interviews and Client Delivery

60-minute goal: Turn technical capability into professional credibility through evidence-based positioning, discovery, scoping, proposals, estimates, interview reasoning and long-term skill development.

Learning outcomes

- Explain and apply: senior positioning must be evidence-backed.
- Explain and apply: clients buy reduced risk and completed outcomes.
- Explain and apply: discovery questions protect both sides.
- Explain and apply: milestones should produce verifiable artifacts.
- Complete the guided lab and capture reproducible evidence.
- Answer the senior interview questions using architecture/security trade-offs, not memorized definitions.

Minute-by-minute lecture plan

Time	Teaching focus
0-8 min	What “senior” means in client outcomes, not titles
8-18 min	Portfolio and GitHub evidence strategy
18-30 min	Upwork job triage, discovery and proposal structure
30-40 min	Estimating scope, milestones, risks and acceptance criteria
40-50 min	Senior technical interview system-design practice
50-60 min	90-day continuation roadmap and personal specialization plan

Key terms for today

Term	Term	Term
portfolio	case study	discovery
scope	acceptance criteria	milestone
risk register	estimate	technical proposal
system design	trade-off	postmortem

Instructor opening script

Today’s topic is senior blockchain career: upwork, linkedin, interviews and client delivery. The senior-level objective is not to memorize API names; it is to understand the underlying trust model, state transitions and failure modes well enough to make architecture decisions and debug unexpected behavior. During the lecture, repeatedly ask: what state is authoritative, who can change it, which external dependencies are trusted, and what property must remain true after every successful transaction?

Deep Theory and Mental Models

Senior positioning must be evidence-backed

A senior title is credible when you can own ambiguous requirements, architecture, security, testing, deployment and incident considerations. Your portfolio should show decision-making artifacts and trade-offs, not only token contracts and tutorial clones.

Clients buy reduced risk and completed outcomes

A strong proposal translates a vague request into an engineering plan: target chains, contract responsibilities, integrations, admin model, test strategy, deployment, deliverables and assumptions. This demonstrates that you understand what can go wrong before billing begins.

Senior lens: Translate this concept into a review question: identify the authoritative state, caller/dependency assumptions, and the invariant that would fail if the concept were implemented incorrectly.

Discovery questions protect both sides

For a staking, token, bridge or marketplace project, clarify custody, upgradeability, token standards, supported networks, expected TVL/value, admin powers, oracle dependencies, existing audits, frontend/backend scope and deployment ownership. Missing answers become explicit assumptions, not silent guesses.

Milestones should produce verifiable artifacts

Examples: architecture/specification approved; contracts implemented with unit tests; fork integration complete; security review complete; testnet/UAT deployment; production deployment and handover. Each milestone should have acceptance criteria that a client can verify.

Senior lens: Translate this concept into a review question: identify the authoritative state, caller/dependency assumptions, and the invariant that would fail if the concept were implemented incorrectly.

Interviews reward structured reasoning

When asked to design a \$50M protocol, do not jump to code. State assumptions, define actors/assets/invariants, choose onchain/offchain boundaries, describe permissions, dependencies, testing, monitoring and incident response, then discuss trade-offs. The structure demonstrates seniority even if implementation details differ.

Specialization accelerates reputation

You can remain broad while developing a clear market wedge: DeFi/Foundry security, EVM protocol engineering, Solana/Anchor, smart-contract audits, tokenization, blockchain backend/indexing or cross-chain integrations. Publish artifacts in that niche consistently for 90 days.

Architecture, Data Flow and Trade-offs

Reference architecture / execution flow

```

Client request
  v
Discovery questions
  v
Requirements + assumptions
  v
Architecture + threat model
  v
Milestones + estimate + acceptance criteria
  v
Implementation + testing + review
  v
Testnet/UAT
  v
Production deployment + verification
  v
Handover docs + monitoring/runbook
  v
Case study / lessons learned (sanitized)

```

Trade-off analysis

1. Broad "I do blockchain" profiles reach more postings but make differentiation difficult.
2. Narrow specialization improves credibility but can reduce the number of directly relevant jobs; maintain adjacent skills.
3. Fixed-price work rewards clear scope but exposes you to ambiguity; use milestones and change-control language.
4. Hourly work handles uncertainty better but clients still expect progress visibility and artifact-based reporting.
5. Public portfolio depth builds trust but never expose client secrets, keys or proprietary code without permission.

Failure-mode questions to ask in a design review

- What happens if the external dependency is unavailable, stale or malicious?
- Can one compromised key change user balances, upgrade logic or move treasury assets?
- Does the system fail closed, fail open, or enter an ambiguous partially completed state?
- Which data is authoritative: contract state, emitted events, an indexer, an RPC response or an offchain database?
- How would you detect this component behaving incorrectly in production?
- Can the operation be replayed, reordered, reentered or executed on the wrong chain/context?
- What is the smallest privilege or interface that satisfies the requirement?

Architecture habit: For client work, draw the flow before writing code. A five-minute diagram often reveals missing authorization, wrong trust assumptions or unavailable recovery paths earlier than a compiler can.

Code Walkthrough and Implementation Notes

Proposal structure: concise technical template

1. Restate the client outcome in one sentence.
2. Show 2-3 directly relevant capabilities/evidence.
3. Identify the most important technical risks/decisions.
4. Propose a short milestone plan.
5. Ask 3-5 discovery questions that affect architecture or scope.
6. Offer a concrete first deliverable (architecture/spec, audit pass, PoC).

Example opening:

"I can build the staking/vault contracts and the integration layer, but before implementation I would lock down custody, reward source, upgrade/admin model, supported assets and withdrawal rules. I would deliver the contract spec + threat model first, then implement with Foundry unit/fuzz/invariant tests..."

Walkthrough

Avoid generic claims such as "I am expert in all blockchains." Use evidence: capstone link, audit sample, invariant tests, deployment experience or relevant system diagram.

Do not promise audit/security guarantees you cannot substantiate. Say what review/testing is included and whether an independent audit is recommended.

Estimate unknown integrations separately and identify assumptions that can change price/schedule.

For sensitive client work, share sanitized case studies rather than proprietary repositories.

Senior implementation checklist

- Pin the compiler/framework/library versions used by the repository.
- Keep configuration such as chain IDs and dependency addresses outside business logic unless immutability is deliberate.
- Use typed interfaces instead of arbitrary calls whenever practical.
- Emit events for operationally important state changes.
- Write the failure-path test at the same time as the success-path implementation.
- Document what this example intentionally omits before anyone reuses it.

Guided Lab: Build, Observe and Break It

Lab steps

1. Create a LinkedIn headline focused on senior blockchain protocol engineering and your chosen specialization.
2. Rewrite About section around outcomes, stack, security mindset and portfolio evidence.
3. Pin 4-6 strongest GitHub repositories and improve README screenshots/architecture.
4. Collect five realistic blockchain job descriptions and create a skills-gap matrix.
5. For each job, write a 150-250 word custom proposal using the template.
6. Create a reusable discovery questionnaire for DeFi, token and Web3 backend projects.
7. Write a sample milestone plan for a staking platform.
8. Practice the interview prompt: design an upgradeable DeFi staking system handling significant TVL. Record yourself for five minutes.
9. Create a 90-day plan: 30 days portfolio depth, 30 days applications/interviews, 30 days specialization/open-source contributions.
10. Set weekly metrics: quality proposals, interview practice, shipped tests/features, public technical writing.

Expected evidence

- Profile language matches evidence in repositories.
- Proposals mention client-specific requirements and risks rather than copy-paste introductions.
- Every milestone has an artifact and acceptance criterion.
- The 90-day plan includes learning, shipping, public proof and job-market feedback.

Troubleshooting guide

If you see a problem: If applications get views but no replies, improve relevance and opening lines before simply increasing volume.

If you see a problem: If interviews expose weak system design, practice architecture explanations instead of memorizing more Solidity syntax.

If you see a problem: If clients request unrealistic timelines, respond with scope trade-offs and a smaller verifiable first milestone.

If you see a problem: Never request or store client production private keys casually; define secure deployment/handover responsibilities.

Lab rule: Do not “fix” a failing lab by removing validation or weakening the expected assertion. First identify whether the failure is environment, configuration, integration state or actual code behavior.

Security, Testing, Debugging and Production Readiness

Security review points

- Do not overstate audits, certifications, years or project results.
- Use testnet/local evidence when production experience is limited and label it accurately.
- Protect client secrets and code.
- Put security assumptions and independent-audit recommendations in high-value DeFi proposals.
- Use written acceptance criteria to reduce disputes.
- Maintain professional change control when scope expands.

Required test plan

- Portfolio review against a rubric: clarity, tests, security, deployment, docs.
- Mock proposal review for specificity.
- Mock discovery call questions.
- 45-minute system-design interview practice.
- Code-review exercise explaining a security bug and remediation.

Debugging playbook

1. Reproduce deterministically: pin block, seed, caller, input and contract version.
2. Read the revert/call trace before changing gas limits or code.
3. Check chain ID, deployed address, ABI and dependency version together.
4. Inspect state immediately before the failing call.
5. Reduce the scenario to the smallest failing sequence.
6. Convert the discovered failure into a regression test.
7. Only then refactor or optimize.

Production gate: A feature is not complete when it compiles. It is complete when its permissions, failure modes, tests, deployment configuration, monitoring signals and rollback/emergency assumptions are understood.

Senior Interview Practice, Homework and Portfolio Output

Interview questions - answer aloud first

Q: What does senior mean in blockchain work?

A: Owning ambiguous requirements and delivering secure, testable, operable systems across contracts and surrounding infrastructure.

Q: What should an Upwork proposal prove quickly?

A: You understand the client's outcome, the key risks, a credible delivery plan and have relevant evidence.

Q: How do you estimate an ambiguous project?

A: Perform discovery, separate assumptions, create milestone-based scope and price uncertainty rather than pretending it does not exist.

Q: What belongs in a strong portfolio README?

A: Problem, architecture, setup, tests, security assumptions, deployment/example usage, limitations and evidence.

Q: How should you answer a system-design interview?

A: State assumptions, define actors/assets/invariants, architecture, permissions, dependencies, testing, monitoring, failure modes and trade-offs.

Q: What should you specialize in?

A: A market-relevant area where you can repeatedly publish deep evidence while retaining broad fundamentals.

Homework

- Finalize your GitHub portfolio and publish one capstone case study.
- Prepare three proposal templates: protocol build, smart-contract debugging/audit, Web3 backend/indexing.
- Run two mock interviews each week for the next 90 days.
- Choose one specialization and publish one technical artifact per week.

Portfolio deliverable

Ship this: Create a polished professional package: LinkedIn profile, Upwork profile, GitHub portfolio index, capstone case study, audit sample and system-design notes.

Official references for this day

<https://docs.soliditylang.org/en/latest/>

<https://www.getfoundry.sh/>

<https://docs.openzeppelin.com/contracts/5.x>

<https://solana.com/docs/>

APPENDIX A - 90-DAY SENIOR BLOCKCHAIN CONTINUATION ROADMAP

Why the next 90 days matter

The 20-day handbook builds the map. The next 90 days convert that map into judgment through repetition. The schedule below is designed around shipping evidence, not accumulating more passive tutorials.

Days 1-30 after the course: deepen the capstone

- Finish the DeFi Vault capstone with clean tests, deployment scripts and architecture documentation.
- Add one real fork integration and one deliberately malicious mock dependency.
- Run Slither, triage every result and write a self-audit report.
- Create a reorg-aware event indexer and simple dashboard/API.
- Rehearse an upgrade/incident on a local fork.
- Publish two technical articles derived from your hardest debugging lessons.

Days 31-60: specialize and contribute

- Choose one specialization: DeFi/security, EVM protocol engineering, Solana/Anchor, backend/indexing, or cross-chain.
- Read and test at least two open-source protocols in that specialization.
- Make one documentation/test/open-source contribution if appropriate.
- Build one smaller specialized project that solves a real integration problem.
- Perform two timed mock audits or code reviews each week.
- Practice explaining architecture to a non-blockchain CTO/client.

Days 61-90: market feedback loop

- Apply to carefully selected roles/contracts where your portfolio maps directly to the requirements.
- Track proposal-to-reply and interview-stage conversion rather than only number of applications.
- Turn interview misses into specific study tasks and repository improvements.
- Publish one new evidence artifact per week.
- Ask clients/interviewers which proof would have increased confidence, then improve portfolio accordingly.

Success metric: At Day 90, you should be able to show a stranger exactly where in your public work you demonstrated testing, security, architecture, deployment and debugging - not merely tell them you know those topics.

APPENDIX B - CLIENT DISCOVERY CHECKLIST

Use before estimating blockchain work

Area	Discovery question
Business outcome	What user/business outcome must the system create? What is explicitly out of scope?
Networks	Which chains/L2s are required? Testnet and production? Any cross-chain behavior?
Assets	Which native/ERC/SPL assets? Any nonstandard token behavior, rebasing or transfer fees?
Custody	Who can move user funds? Is the system non-custodial? Are admin recovery powers required?
Upgradeability	Immutable, upgradeable or migration-based? Who controls upgrades and with what delay?
Roles	Who can pause, mint, change fees, update oracles, manage strategies, withdraw treasury funds?
Oracles	Which external prices/data are required? Freshness and outage policy?
Integrations	DEX, lending, bridge, wallet, indexer, custody, KMS, analytics?
Value at risk	Expected TVL/transaction sizes? This changes review, governance and monitoring expectations.
Security	Independent audit required? Bug bounty? Threat model? Incident response?
Frontend/backend	Which components are included? Who signs automated transactions?
Data	What historical queries/analytics are required? Reorg/finality policy?
Deployment	Who owns production keys? Multisig setup? Source verification? Handover process?
Acceptance	What tests, documentation and behaviors must pass for each milestone?
Operations	Who monitors the system after launch and how are alerts handled?

Red flags before accepting a fixed-price scope

- Client cannot specify supported chains/assets but expects a final fixed bid.
- “Admin should be able to do anything” with no governance model.
- Production private key is expected to be shared in chat/email.
- Profit/return guarantees requested for arbitrage, trading or yield strategies.
- No time allocated for testing, testnet/UAT or independent review on high-value contracts.
- Bridge/oracle/router addresses supplied from unverified third-party sources.
- Scope includes “audit” but expectations actually mean a guarantee against all future exploits.

APPENDIX C - SENIOR SMART CONTRACT REVIEW CHECKLIST

Requirements and state model

- List actors and assets.
- Write invariants before code review.
- Identify lifecycle states and impossible/terminal transitions.
- Identify every external dependency and source of price/data.

Authorization

- List every external/public state-changing entry point.
- Map roles to functions and role-admin relationships.
- Review ownership transfer, initializer and upgrade paths.
- Look for arbitrary target/call capabilities.

Accounting

- Track units/decimals.
- Check rounding direction.
- Compare local balance versus accounted balance.
- Check caps, debt/share/fee calculations and donation edge cases.

External calls

- Locate calls, callbacks, token transfers and delegatecalls.
- Classify callees as trusted/semi-trusted/user-controlled.
- Check CEI/reentrancy assumptions and return values.
- Test malicious receivers/tokens.

Signatures

- Check domain separator, chain/context binding, signer recovery, nonce and replay protection.
- Review expiration/deadline and typed-data UX.

Oracles

- Verify feed denomination, decimals, staleness and L2 sequencer policy.
- Model manipulation and outage.

Upgradeability

- Validate storage layout.
- Protect initializer and upgrade authority.
- Review timelock/multisig operations and post-upgrade checks.

Testing

- Unit success/failure paths.
- Fuzz numeric/address inputs.
- Stateful invariants.
- Fork integrations.
- Regression tests for every finding.

APPENDIX D - PORTFOLIO QUALITY RUBRIC

Score each important repository from 0-3 in every category. A senior-track portfolio should not rely on one polished README hiding weak tests; quality should be visible throughout the repository.

Category	0	1	2	3
Problem definition	No context	Basic description	Clear user/problem scope	Requirements + assumptions + non-goals
Architecture	None	Simple diagram	Components + data flow	Trust boundaries + trade-offs + failure modes
Code quality	Tutorial copy	Works locally	Modular + documented	Clear interfaces + minimal privilege + maintainable
Testing	Few examples	Unit tests	Unit + fuzz/fork	Invariants + malicious mocks + regression discipline
Security	No section	Generic checklist	Threat model + controls	Attack tests + residual risks + runbook
Deployment	Manual notes	Basic script	Network config + verify	Postconditions + role handover + chain guards
Operations	None	Events	Monitoring suggestions	Alert ownership + incident procedures
Professional proof	Claims only	Screenshots	Reproducible outputs	Case study explaining decisions and trade-offs

Recommended flagship repositories

- Production ERC-20 / role system
- Security exploit + remediation lab
- Audit report sample
- AMM math / DEX integration lab
- Flash-loan simulator
- Oracle-safe consumer
- Upgradeable proxy lab
- Web3 transaction service
- Reorg-aware indexer
- Solana Anchor PDA program
- DeFi Vault capstone

APPENDIX E - GLOSSARY (PART 1)

- ABI:** Application Binary Interface: schema used to encode/decode contract calls, returns and events.
- Account abstraction:** Techniques that make account authorization/execution programmable beyond a simple EOA key model.
- Allowance:** ERC-20 delegated spending limit from owner to spender.
- AMM:** Automated market maker: onchain market using deterministic pool pricing/accounting rules.
- Anvil:** Foundry local EVM node.
- Archive node:** Node retaining historical state needed for some old-block queries.
- Atomicity:** Either the entire transaction state transition succeeds or all changes revert.
- Base fee:** EIP-1559 protocol-determined gas base fee burned for included execution.
- Block reorg:** Replacement of recently considered canonical blocks by another valid chain branch.
- Bridge:** System moving messages/value representations across chains.
- Calldata:** Read-only external call input data in the EVM.
- Chain ID:** Identifier used to distinguish blockchain networks and prevent some replay contexts.
- Circuit breaker:** Control that pauses/limits actions when safety conditions are violated.
- CPI:** Solana cross-program invocation.
- Delegatecall:** EVM call executing another address's code in caller storage/context.
- Finality:** Confidence/protocol condition that a state transition will not be reverted.
- Fork test:** Local test using copied state from a live chain at a chosen block.
- Function selector:** First 4 bytes of Keccak-256 over a canonical Solidity function signature.
- Fuzz testing:** Property-based testing with automatically generated inputs.
- Gas:** EVM resource-metering unit consumed by computation/state access.

APPENDIX E - GLOSSARY (PART 2)

Health factor: Lending-protocol metric relating risk-adjusted collateral to debt.

Hook: Extensibility callback around protocol actions; exact semantics depend on protocol.

Invariant: Property that must remain true under all allowed state transitions.

Indexer: Offchain service transforming blockchain history/events into query-optimized data.

Initializer: Upgrade-safe replacement for constructor state initialization through a proxy.

KMS/HSM: Managed/hardware-backed key systems used to isolate signing secrets.

L1: Base blockchain layer used for execution/settlement/data depending on architecture.

L2: Scaling system operating above a base chain while deriving specified security properties from it.

Liquidity: Assets available to execute trades/withdrawals/loans without excessive price or availability impact.

MEV: Value obtainable through transaction inclusion, exclusion or ordering choices.

Nonce: Sequential/replay-prevention value whose exact semantics depend on account/message system.

Oracle: Mechanism that supplies external facts/data to onchain applications.

PDA: Solana Program Derived Address controlled by program logic through deterministic seeds.

Proxy: Contract forwarding calls to implementation logic, often using delegatecall.

Reentrancy: Unexpected nested execution caused by an external call invoking back before state is safely established.

RPC: Remote Procedure Call interface exposed by blockchain nodes/providers.

Sequencer: Component used by many L2s to order transactions and provide rapid confirmations.

Slippage: Difference between expected quote/reference and actual/minimum acceptable execution.

Storage: Persistent EVM contract state across transactions.

Subgraph: The Graph data-source/mapping/schema package that builds a queryable blockchain-data API.

Additional terms to master

- ERC-20 / ERC-721 / ERC-1155 / ERC-4626
- Permit / EIP-712
- Timelock / multisig / guardian
- TWAP / tick / concentrated liquidity
- Fraud proof / validity proof / data availability
- Lock-mint / burn-mint / liquidity bridge
- SPL Token / ATA
- Revert / panic / custom error
- Staticcall / memory / stack
- Role admin / least privilege

APPENDIX F - FINAL SYSTEM-DESIGN INTERVIEW RUBRIC

Prompt

Practice prompt: Design a secure, upgradeable DeFi staking/vault platform expected to manage significant TVL across an EVM L2. It needs deposits, withdrawals, rewards/yield integration, an oracle dependency, backend analytics and an emergency process.

A strong 15-minute answer should cover

1. Clarify assumptions: asset(s), custody, yield source, upgrade requirement, expected scale, network, oracle need and withdrawal liquidity.
2. Define actors and assets: users, governance, guardian, strategy, oracle, treasury, backend signer if any.
3. State invariants: solvency, share/claim limits, caps, authorization and accounting.
4. Draw contracts and external dependencies.
5. Explain permissions, multisig/timelock and emergency pause.
6. Explain oracle freshness/manipulation/L2 sequencer policy.
7. Explain strategy/DEX integration boundaries and slippage.
8. Describe unit/fuzz/invariant/fork tests.
9. Describe indexing/backend as non-authoritative data plane.
10. Describe monitoring, deployment, verification and incident runbooks.
11. Close with trade-offs: upgradeability, centralization, withdrawal liquidity, operational complexity.

What weak answers sound like

- Jumping immediately into Solidity function names without clarifying system requirements.
- Saying “use Chainlink/OpenZeppelin” as a substitute for explaining trust and failure modes.
- Ignoring admin/upgrader keys.
- Treating an indexer/database as authoritative for user funds.
- Claiming an audit guarantees security.
- Ignoring L2 finality/sequencer and oracle outage policy.
- No monitoring or incident response.

APPENDIX G - OFFICIAL REFERENCE LIBRARY

Blockchain tooling changes quickly. Before production work, re-check the current official documentation for compiler versions, APIs, network addresses, deployment tooling and protocol-specific integration requirements. The links below were selected as primary references for this edition.

Solidity - <https://docs.soliditylang.org/en/latest/>

Solidity ABI - <https://docs.soliditylang.org/en/latest/abi-spec.html>

Ethereum Developer Docs - <https://ethereum.org/en/developers/docs/>

Ethereum Scaling - <https://ethereum.org/en/developers/docs/scaling/>

Foundry - <https://www.getfoundry.sh/>

Foundry Testing - <https://www.getfoundry.sh/forging/testing>

Foundry Invariants - <https://www.getfoundry.sh/guides/invariant-testing>

Foundry Fork Testing - <https://www.getfoundry.sh/guides/fork-testing>

OpenZeppelin Contracts 5.x - <https://docs.openzeppelin.com/contracts/5.x>

OpenZeppelin Access Control - <https://docs.openzeppelin.com/contracts/5.x/access-control>

OpenZeppelin ERC-4626 - <https://docs.openzeppelin.com/contracts/5.x/erc4626>

OpenZeppelin Upgrades - <https://docs.openzeppelin.com/upgrades-plugins/>

Aave V3 Flash Loans - <https://aave.com/docs/aave-v3/guides/flash-loans>

Chainlink Data Feeds - <https://docs.chain.link/data-feeds>

Chainlink CCIP - <https://docs.chain.link/ccip>

The Graph Subgraphs - <https://thegraph.com/docs/en/subgraphs/developing/introduction/>

Uniswap Docs - <https://docs.uniswap.org/>

Solana Programs - <https://solana.com/docs/core/programs>

Solana Rust Programs - <https://solana.com/docs/programs/rust>

Solana Installation - <https://solana.com/docs/intro/installation>

FINAL COURSE COMPLETION CHECKLIST

You are ready to present yourself for senior-track work when you can demonstrate most of the following

- ☐ Explain EVM transaction execution and gas from first principles.
- ☐ Write Solidity without relying on copied templates for basic state/authorization.
- ☐ Use OpenZeppelin intentionally and understand the authority its extensions create.
- ☐ Write unit, fuzz, invariant and fork tests.
- ☐ Reproduce and explain a security vulnerability locally.
- ☐ Write a professional audit finding with proof and remediation.
- ☐ Explain AMM slippage/price impact and lending/oracle risk.
- ☐ Build and test a safe oracle consumer.
- ☐ Perform a storage-safe proxy upgrade in a test environment.
- ☐ Build a Web3 backend transaction state machine without key leakage.
- ☐ Build a reorg-aware event indexer.
- ☐ Explain optimistic vs ZK rollups and L2 finality.
- ☐ Threat-model cross-chain message replay and source validation.
- ☐ Build/test a Solana Anchor program with PDA authorization.
- ☐ Create a privilege matrix, monitoring plan and incident runbook.
- ☐ Ship the DeFi Vault capstone with architecture and security docs.
- ☐ Answer system-design interview prompts in a structured way.
- ☐ Write client discovery questions before estimating work.
- ☐ Show evidence in GitHub instead of relying on title claims.
- ☐ Maintain a 90-day continuing practice plan.

Next action: Choose Day 1 and begin. Do not wait until you have read the entire book. The course is designed to create skill through a daily build-test-explain loop.